浙江大学

# 本科实验报告

| | |
|---|---|
| 课程名称： | 实验 30 基于 RV32I 指令集的 RISC-V 微处理器设计 |
| 姓　　名： | |
| 学　　院： | 信息与电子工程学院 |
| 系： | |
| 专　　业： | 信息工程 |
| 学　　号： | |
| 指导教师： | 屈民军，唐奕 |

2025 年 12 月 19 日

# 目录

一、 实验目的 ..... 4

二、 实验任务 ..... 4

三、 实验设计原理与 Verilog 代码 ..... 5

（一） 总体设计 ..... 5

1. 流水线中的控制信号 ..... 5

2. 数据相关与数据转发 ..... 7

3. 数据冒险与数据转发 ..... 9

（二） RISC-V：指令译码模块（ID）的设计 ..... 9

1. 寄存器堆(Registers)子模块的设计 ..... 10

2. 指令译码(包含立即数产生电路)子模块的设计 ..... 12

3. 分支检测(BranchTest)电路的设计 ..... 18

4. 冒险检测功能电路(HazardDetector)的设计 ..... 20

5. 顶层代码 ID 的设计： ..... 20

（三） RISC-V：执行模块(EX)的设计 ..... 22

1. ALU 子模块的设计 ..... 23

2. 数据前推电路的设计 ..... 28

3. 顶层代码 EX 设计 ..... 29

（四） RISC-V：数据存储器模块(DataRAM)的设计 ..... 31

1. 原理： ..... 31

2. 过程： ..... 32

3. Verilog 代码：在 Risc5CPU 顶层代码中，作为 MEM 级使用 ..... 32

（五） RISC-V：取指令级模块(IF)的设计 ..... 32
1. 原理： ..... 33
2. Verilog 代码： ..... 34
3. 仿真波形： ..... 35
（六） RISC-V：流水线寄存器的设计 ..... 36
1. 原理： ..... 36
2. Verilog 代码： ..... 37
（七） RISC-V：顶层文件的设计 ..... 39
1. 原理： ..... 39
2. Verilog 代码： ..... 40
3. 仿真波形与结果分析： ..... 43
四、 Vivado 工程实现和下载 ..... 52
1. vivado 工程文件建立 ..... 52
2. 开发板测试： ..... 53
五、 拓展功能 ..... 55
六、 实验过程中遇到的问题和解决方案 ..... 62
1. 原 CPU 代码编写时： ..... 62
2. 编写拓展功能： ..... 63
七、 思考题 ..... 63
八、 实验总结 ..... 64

专业：信息工程
姓名：
学号：
日期：2025.12.6
地点：玉泉校区

# 浙江大学实验报告

课程名称：计算机设计与组成
指导老师：屈民军，唐奕
成绩：
实验名称：实验 30 基于 RV32I 指令集的 RISC-V 微处理器设计

## 一、实验目的

1．熟悉 RISC-V 指令系统；

2．了解提高 CPU 性能的方法；

3．掌握流水线 RISC-V 微处理器的工作原理；

4．理解数据冒险、控制冒险的概念以及流水线冲突的解决方法；

5．掌握流水线 RISC-V 微处理器的测试方法；

6．了解用软件实现数字系统的方法。

## 二、实验任务

### 1．基本要求：

设计一个流水线 RISC-V 微处理器，具体要求如下所述

（1）至少运行下列 RV32I 核心指令：

①算术运算指令：add、sub、addi

②逻辑运算指令：and、or、xor、slt、sltu、andi、ori、xori、slti、sltiu

③移位指令：sll、srl、sra、slli、srli、srai

④条件分支指令：beq、bne、blt、bge、bltu、bgeu

⑤无条件跳转指令：jal、jalr

⑥数据传送指令：lw、sw、lui、auipc

⑦空指令：nop

（2）采用 5 级流水线技术，对数据冒险实现转发或阻塞功能；

（3）在 Nexys Video 开发系统中实现 RISC-V 微处理器，要求 CPU 的运行速度大于 25MHz。

### 2．拓展要求：

（1）要求设计的微处理器还能运行 lb、lh、ld、lbu、lhu、lwu、sb、sh 或 sd 等字节、半

字 和双字数据传送指令；

（2）要求设计的 CPU 增加异常（exception）、自陷（trap）、中断（interrupt）等处理方案。

# 三、实验设计原理与 Verilog 代码

## （一）总体设计

流水线是数字系统中一种提高系统稳定性和工作速度的方法，广泛应用于高档 CPU 的架构中。根据 RISC-V 处理器指令的特点，将指令整体的处理过程分为取指令(IF)、指令译码(ID)、执行(EX)、存储器访问(MEM)和寄存器回写(WB)五级。如图 30.2 示，一个指令的执行需要 5 个时钟周期，每个时钟周期的上升沿来临时，此指令所代表的一系列数据和控制信息将转移到下一级处理。

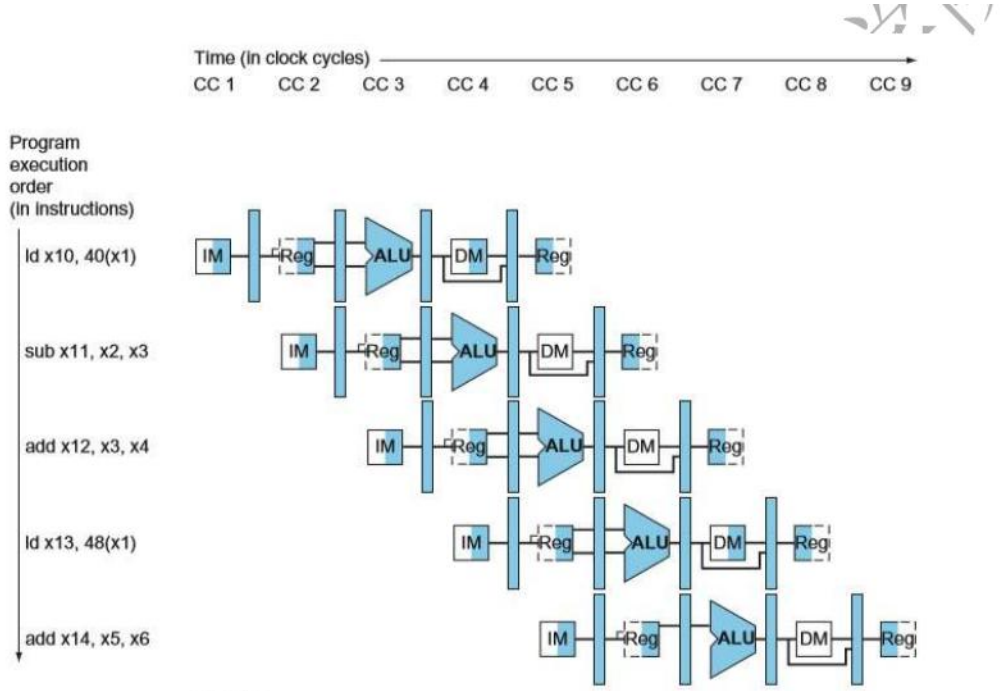



图 30. 2 流水线流水作业示意图

图 30.3 所示为符合设计要求的流水线 RISC-V 微处理器的原理框图，采用五级流水线。由于在流水线中，数据和控制信息将在时钟周期的上升沿转移到下一级，所以规定流水线转移的变量命名遵守:名称-流水线级名称。

### 1．流水线中的控制信号

（1）IF 级：取指令级。从 ROM 中读取指令，并在下一个时钟沿到来时把指令送到 ID 级的指令缓冲器中。该级共有三个控制信号：

①PCSource：决定下一条指令指针的控制信号，当 PCSource=0 时，顺序执行下一条指令；当 PCSource=1 时，跳转执行。

②IFWrite：当 IFWrite=0 时，阻塞 IF/ID 流水线，同时暂停读取下一条指令。

③IF_flush：当 IF_flush=1 时，清空 IF/ID 寄存器。

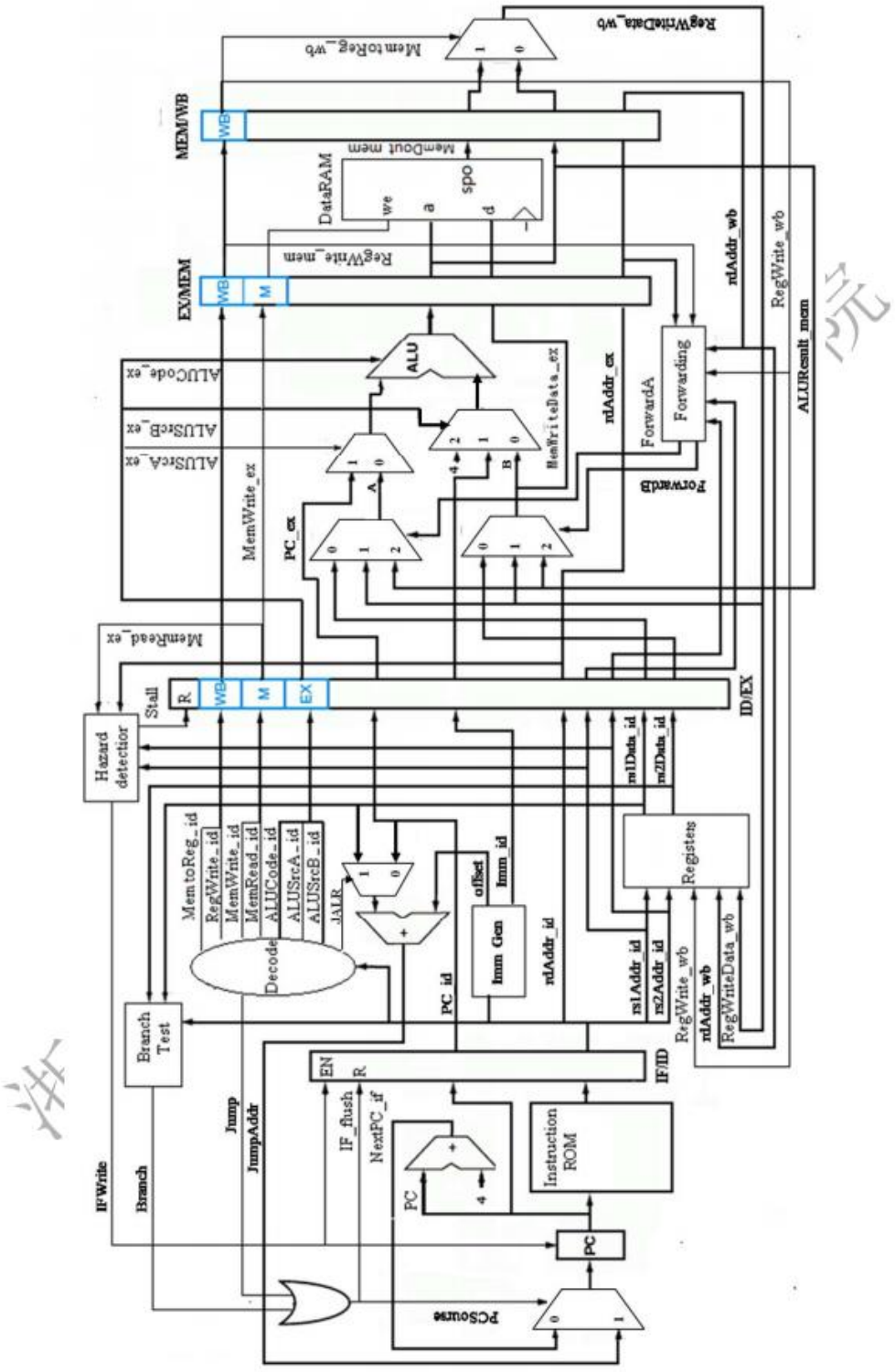



图 30.3 流水线 RISC-V 微处理器的原理框图

（2）ID 级：指令译码级。对来自 IF 级的指令进行译码，并产生相应的控制信号。整个 CPU 的控制信号基本都是在 ID 级上产生。该级自身不需任何控制信号。

流水线冒险检测也在该级进行，即当流水线冒险条件成立时，冒险检测电路产生 Stall 信号清空 ID/EX 寄存器，同时冒险检测电路产生低电平 IFWrite 信号阻塞 IF/ID 流水线，即插入一个流水线气泡)

（3）EX 级：执行级。此级进行算术或逻辑操作；此外数据传送指令所用的 RAM 访问地址也是在本级上实现。控制信号有 ALUCode、ALUSrcA 和 ALUSrcB，根据这些信号确定 ALU 操作、并选择两个 ALU 操作数 ALU_A、ALU_B。

另外，数据转发也在该级完成。数据转发控制电路产生 ForwardA 和 ForwardB 两组转发控制信号。

（4）MEM 级：存储器访问级。只有在执行数据传送指令时才对存储器进行读写，对其它指令只起到缓冲一个时钟周期的作用。该级只需存储器写操作允许信号 MemWrite。

（5）WB 级：回写级。此级把指令执行的结果回写到寄存器堆中。该级设置信号 MemtoReg 和寄存器写操作允许信号 RegWrite。其中 MemtoReg 决定写入寄存器的数据来源： 当 MemtoReg=0 时，回写数据来自于 ALU 运算结果；而当 MemtoReg=1 时，回写数 据来自于存储器。

## 2．数据相关与数据转发

如果上一条指令的结果还没有写入到寄存器中，而下一条指令的源操作数又恰恰是此寄存器的数据，那么，它所获得的将是原来的数据，而不是更新后的数据。这样的相关问题称为数据相关。在设计当中，我们采用数据转发和插入流水线气泡的方法解决此类相关问题。

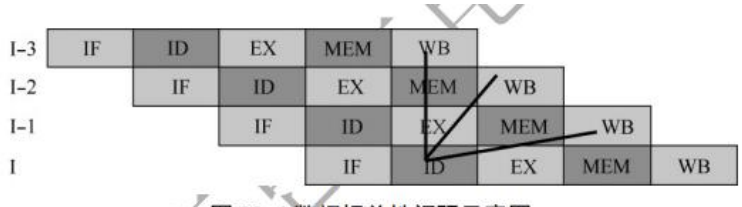


图 30.4 数据相关性问题示意图

（1）一阶数据相关与转发（EX 冒险）

如图 30.5 所示，如果源操作寄存器与第 I-1 条指令的目标操作寄存器相重，将导致一阶数据相关。此时，第 I 条指令的 EX 级与第 I-1 条指令的 MEM 级处于同一时钟周期，且数据转发必须在第 I 条指令的 EX 级完成。

因此，导致操作数 A 的一阶数据相关成立的条件为：MEM 级阶段必须是写操作

（RegWrite_mem=1）；目标寄存器不是 X0 寄存器（rdAddr_mem≠0）；两条指令读写同一个寄存器（rdAddr_mem=rs1Addr_ex）

导致操作数 B 的一阶数据相关成立的条件为：MEM 级阶段必须是写操作（RegWrite_mem=1）；目标寄存器不是 X0 寄存器（rdAddr_mem≠0）；两条指令读写同一个寄存器（rdAddr_mem=rs2Addr_ex）

解决方法是将第 I-1 条指令的 MEM 级的 ALUResult_mem 转发至第 I 条 EX，

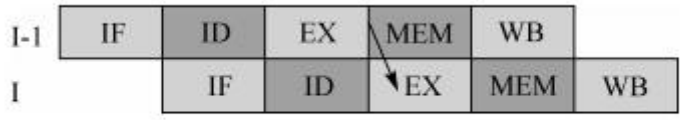



图 30.5 一阶前推网络示意图

（2）二阶数据相关与转发（MEM 冒险）

如图 30.6 所示，如果第 I 条指令的源操作寄存器与第 I-2 条指令的目标寄存器相重，将导致二阶数据相关。

导致操作数 A 的二阶数据相关必须满足下列条件：寄存器必须是写操作（RegWrite_wb =1）；目标寄存器不是 X0 寄存器（rdAddr_wb≠0）；读写同一个寄存器（rdAddr_wb =rs1Addr_id）

导致操作数 B 的三阶数据相关必须满足下列条件：寄存器必须是写操作（RegWrite_wb =1）；目标寄存器不是 X0 寄存器（rdAddr_wb≠0）；读写同一个寄存器（rdAddr_wb =rs2Addr_id）

解决方法是将第 I-2 条指令的回写数据 RegWriteData 转发至 I 条指令的 EX

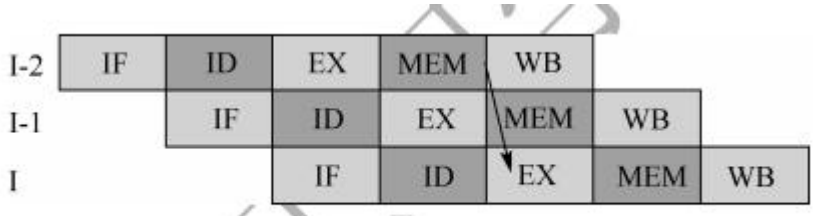



图 30.6 二阶前推网络示意图

（3）三阶数据相关

如图 30.7 所示，如果第 I 条指令的源操作寄存器与第 I-2 条指令的目标寄存器相重，将导致二阶数据相关。

导致操作数 A 的二阶数据相关必须满足下列条件：WB 级阶段必须是写操作（RegWrite_wb=1）；目标寄存器不是 X0 寄存器（rdAddr_wb≠0）；一阶数据相关条件不

成立（rdAddr_mem≠rs1Addr_ex）；两条指令读写同一个寄存器（rdAddr_wb=rs1Addr_ex）。

导致操作数 B 的二阶数据相关必须满足下列条件：WB 级阶段必须是写操作（RegWrite_wb=1）；目标寄存器不是 X0 寄存器（rdAddr_wb≠0）；一阶数据相关条件不成立（rdAddr_mem≠rs2Addr_ex）；两条指令读写同一个寄存器（rdAddr_wb=rs2Addr_ex）

解决方法是将第 I-2 条指令的回写数据 RegWriteData 转发至 I 条指令的 EX

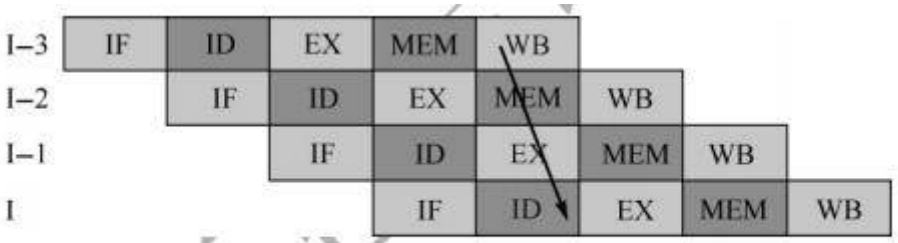



图 30.7 三阶前推网络示意图

### 3．数据冒险与数据转发

如前分析可知，当第 I 条指令读取一个寄存器，而第 I-1 条指令为 lw，且与 lw 写入为同一个寄存器时，定向转发是无法解决问题的。因此，当 lw 指令后跟一条需要读取它结果的指令时，必须采用相应的机制来阻塞流水线，即还需要增加一个冒险检测单元（Hazard Detector）。它工作在 ID 级，当检测到上述情况时，在 lw 指令和后一条指令之间插入气泡，使后一条指令延迟一个周期执行，这样可将一阶数据冒险问题变成二阶数据冒险问题，就可用转发解决。

冒险检测工作在 ID 级，前一条指令已处在 EX 级，冒险成立的条件为：上一条指令必须是 lw 指令（MemRead_ex=1）；两条指令读写同一个寄存器（rdAddr_ex=rs1Addr_id 或 rdAddr_ex=rs2Addr_id）。

当上述条件满足时，指令将被阻塞一个周期，Hazard Detector 电路输出的 Stall 信号将清空 ID/EX 寄存器，另外一个输出低电平有效的 IFWrite 信号将阻塞流水线 ID 级、IF 级，即插入一个流水线气泡。

## （二）RISC-V：指令译码模块（ID）的设计

指令译码模块的主要作用是从机器码中解析出指令，并根据解析结果输出各种控制信号。其主要由指令译码（Decode）、寄存器堆（Registers）、冒险检测、分支检测和加法器等组成，接口信息如表 30.3 所示。

表 30.3 ID 模块的输入/输出引脚说明

| 引脚名称 | 方向 | 说明 |
|---|---|---|
| clk | Input | 系统时钟 |
| Instruction_id[31:0] | | 指令机器码 |
| PC_id[31:0] | | 指令指针 |
| RegWrite_wb | | 寄存器写允许信号，高电平有效 |
| rdAddr_wb[4:0] | | 寄存器的写地址。 |
| RegWriteData_wb[31:0] | | 写入寄存器的数据 |
| MemRead_ex | | 冒险检测的输入 |
| rdAddr_ex[4:0] | | |
| MemtoReg_id | Output | 决定回写的数据来源（0：ALU；1：存储器） |
| RegWrite_id | | 寄存器写允许信号，高电平有效 |
| MemWrite_id | | 存储器写允许信号，高电平有效 |
| MemRead_id | | 存储器读允许信号，高电平有效 |
| ALUCode_id[3:0] | | 决定 ALU 采用何种运算 |
| ALUSrcA_id | | 决定 ALU 的 A 操作数的来源（0：rs1；1：pc） |
| ALUSrcB_id[1:0] | | 决定 ALU 的 B 操作数的来源(2'b00：rs2；2'b01：imm；2'b10：常数 4) |
| Stall | | ID/EX 寄存器清空信号，高电平表示插入一个流水线气泡 |
| Branch | | 条件分支指令的判断结果，高电平有效 |
| Jump | | 无条件分支指令的判断结果，高电平有效 |
| IFWrite | | 阻塞流水线的信号，低电平有效 |
| BranchAddr[31:0] | | 分支地址 |
| Imm_id[31:0] | | 立即数 |
| rdAddr_id[4:0] | | 回写寄存器地址 |
| rs1Addr_id[4:0] | | 两个数据寄存器地址 |
| rs2Addr_id[4:0] | | |
| rs1Data_id[31:0] | | 寄存器两个端口输出数据 |
| rs2Data_id[31:0] | | |

## 1．寄存器堆(Registers)子模块的设计

（1）Registers 模块

①原理：

寄存器堆由 32 个 32 位寄存器组成，这些寄存器通过寄存器号进行读写存取。寄存器堆的原理框图如图 30.8 所示。因为读取寄存器不会更改其内容，故只需提供寄存号即可读出该寄存器内容。读取端口采用数据选择器即可实现读取功能。应注意的是，“0”号寄存器为常数 0。

对于往寄存器里写数据，需要目标寄存器号(WriteRegister)、待写入数据(WriteData)、写允许信号(RegWrite)三个变量。图 30.8 中 5 位二进制译码器完成地址译码，其输出控制目标寄存器的写使能信号 EN，决定将数据 WriteData 写入哪个寄存器。

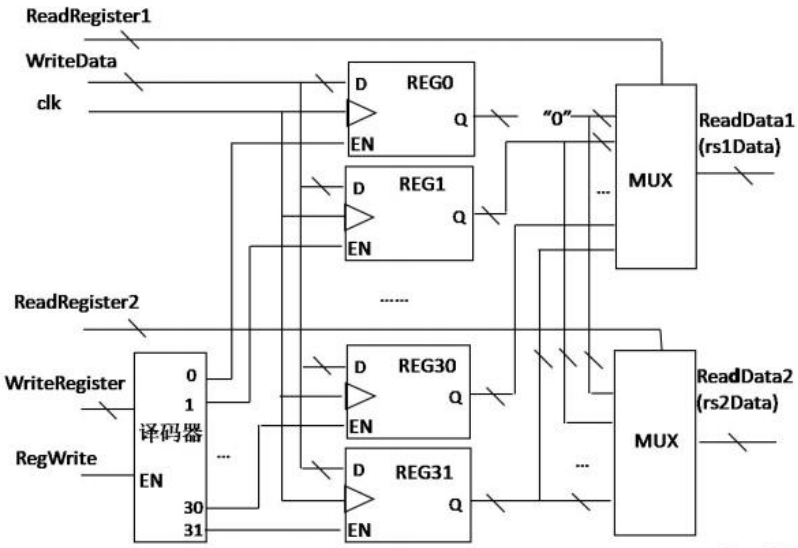



图 30.8 寄存器堆的原理框图

②Verilog 代码：

```
module Registers(clk,RegWrite,ReadRegister1,ReadRegister2,WriteRegister,WriteData,ReadData1,ReadData2);
    input clk;
    input RegWrite; //允写信号
    input [4:0] ReadRegister1,ReadRegister2; //rs1, rs2
    input [4:0] WriteRegister; //rd
    input [31:0] WriteData;
    output [31:0] ReadData1,ReadData2;

    reg[31:0]regs[31:0]; //32*32存储器
        assign ReadData1=(ReadRegister1 == 5'b0)?32'b0:regs[ReadRegister1]; //端口1，判断是否为reg0
        assign ReadData2=(ReadRegister2 == 5'b0)?32'b0:regs[ReadRegister2]; //端口2
        always@(posedge clk) if(RegWrite) regs[WriteRegister]<=WriteData;

endmodule
```

（2）RAW_Registers 模块

①原理：

在流水线型 CPU 设计中，寄存器堆设计还应解决三阶数据相关的数据转发问题。当满足三阶数据相关条件时，寄存器具有 ReadAfer Write 特性。设计时，只需要在图 30.8 设计寄存器堆的基础上添加少量电路就可实现 Read Afer Write 特性

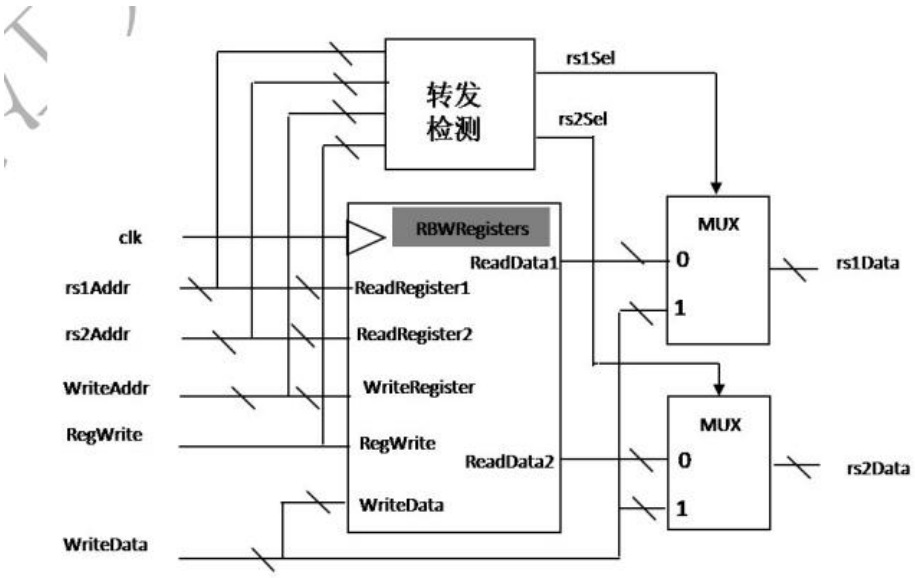



图 30.9 具有 Read After Write 特性寄存器堆的原理框图

②verilog 代码：

```
module RAWRegisters(clk,RegWrite,rs1Addr,rs2Addr,WriteAddr,WriteData,rs1Data,rs2Data);
    input clk;
    input RegWrite; //允写信号
    input [4:0] rs1Addr,rs2Addr;//rs1, rs2
    input [4:0] WriteAddr;//rd
    input [31:0] WriteData;
    output [31:0] rs1Data,rs2Data;

    //转发检测模块
    wire rs1Sel,rs2Sel;
    assign rs1Sel=RegWrite&&(WriteAddr!=0)&&(WriteAddr==rs1Addr);
    assign rs2Sel=RegWrite&&(WriteAddr!=0)&&(WriteAddr==rs2Addr);

    //RBW模块
    wire [31:0] ReadData1,ReadData2;
    Registers RAW_Registers(
        .clk(clk),
        .ReadRegister1(rs1Addr),
        .ReadRegister2(rs2Addr),
        .WriteRegister(WriteAddr),
        .RegWrite(RegWrite),
        .WriteData(WriteData),
        .ReadData1(ReadData1),
        .ReadData2(ReadData2)
    );
    mux_2to1 #(.n(32))muxrs1(.out(rs1Data), .in0(ReadData1), .in1(WriteData), .addr(rs1Sel));
    mux_2to1 #(.n(32))muxrs2(.out(rs2Data), .in0(ReadData2), .in1(WriteData), .addr(rs2Sel));
endmodule
```

## 2．指令译码(包含立即数产生电路)子模块的设计

（1）根据指令确定各个控制信号的值模块

①原理：

RISC-V 将指令分为 R、I、S、SB、U、UJ 等六类。仔细分析表 30.2 指令表，从电路设计角度看，根据操作数的来源和立即数构成方式不同，再次细分指令。具体做法:

- R type 类: 操作码(opcode)为 7'h33，R 类的所有指令，两个操作数分别为 rs1 和 rs2
- Itype 类:操作码 7'h13，I 类的算术逻辑运算指令和移位指令，两个操作数分别为 rs1 和立即数 imm
- LW 指令:操作码 7’h03，I 类的数据传送指令 lw，两个操作数分别为 rs1 和立即数 imm
- JALR 指令:操作码 7’h67，I 类的无条件分支指令 jalr，两个操作数分别为 PC 和常数 4
- SW 指令:操作码 7 h23，S 类的数据传送指令 sw，两个操作数分别为 rs1 和立即数 imm
- SB type 类:操作码 7’h63，SB 类的所有指令，两个操作数分别为 PC 和立即数 imm
- LUI 指令:操作码 7’h37，U 类的数据传送指令 li，只有一个操作数(立即数 imm)
- AUIPC 指令: 操作码 7h17,U 类的数据传送指令 auipc,两个操作数分别为 PC 和立即数 imm
- JAL 指令:操作码 7’h6F，UJ 类的无条件分支指令 jal，两个操作数分别为 PC 和常数 4。

因此，设置 R type、I type、SB type、LW、JALR、SW、UIAUIPC 和 JAL, 等变量来表示指令类型，各变量的值由式 30.3 决定。

$$\begin{cases} R_type = (op == R_type_op) \\ I_type = (op == I_type_op) \\ SB_type = (op == SB_type_op) \\ LW = (op == LW_op) \\ JALR = (op == JALR_op) \\ SW = (op == SW_op) \\ LUI = (op == LUI_op) \\ AUIPC = (op == AUIPC_op) \\ JAL = (op == JAL_op) \end{cases} \tag{30.3}$$

1）只有 LW 指令读取存储器且回写数据取自存储器，所以 MemtoReg=LW，MemRead=LW

2）只有 Sw 指令会对存储器写数据，所以有 MemWrite _SW

3）需要进行回写的指令类型有 R type、I type、LW、JALR、LUI、AUIPC 和 JAL。所以有 RegWrite=R_type || I_type || LW || JALR || LUI || AUIPC || JAL

4）只有 JALR 和 JAL 两条无条件分支指令，所以有 Jump=JALR || JAL

5）操作数 A 和 B 的选择信号的确定：

表 30.4 操作数选择信号的功能表

| 类型 | ALUSrcA_id | ALUSrcB_id[1:0] | 说明 |
|---|---|---|---|
| R_type | 0 | 2'b00 | rd=rs1 op rs2 |
| I_type | 0 | 2'b 01 | rd=rs1 op imm |
| LW | 0 | 2'b 01 | rs1 + imm |
| SW | 0 | 2'b 01 | rs1 + imm |
| JALR | 1 | 2'b 10 | rd=pc + 4 |
| JAL | 1 | 2'b 10 | rd=pc + 4 |
| LUI | 1'bx | 2'b 01 | rd= imm |
| AUIPC | 1 | 2'b 01 | rd=pc + imm |

从表 30.4 可获得 ALUSrcA_id 和 ALUSrcB_id[1:0]表达式。ALUSrcA =JALR || JAL || AUIPC，ALUSrcB[1]= JAL || JALR，ALUSrcB[0]=~(R_type || JAL || JALR)

6）ALUCode 的确定：除了条件分支指令，其它指令都需要 ALU 执行运算，共有 种不同运算，ALUCode 信号需用 4 位二进制表示。最主要为加法运算,设为默认算法,ALUCode 的功能表如表 30.5 所示

表 30.5　ALUCode 的功能表

| R_type | I_type | LUI | funct3 | funct7[6] (funct6[5]) | ALUCode | 备注 |
|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 3'o0 | 0 | 4'd 0 | 加 |
| 1 | 0 | 0 | 3'o0 | 1 | 4'd 1 | 减 |
| 1 | 0 | 0 | 3'o1 | 0 | 4'd 6 | 左移 A <<B |
| 1 | 0 | 0 | 3'o2 | 0 | 4'd9 | A<B?1:0 |
| 1 | 0 | 0 | 3'o3 | 0 | 4'd10 | A<B?1:0（无符号数) |
| 1 | 0 | 0 | 3'o4 | 0 | 4'd 4 | 异或 |
| 1 | 0 | 0 | 3'o5 | 0 | 4'd 7 | 右移 A >>B |
| 1 | 0 | 0 | 3'o5 | 1 | 4'd 8 | 算术右移 A >>>B |
| 1 | 0 | 0 | 3'o6 | 0 | 4'd 5 | 或 |
| 1 | 0 | 0 | 3'o7 | 0 | 4'd 3 | 与 |
| 0 | 1 | 0 | 3'o0 | x | 4'd 0 | 加 |
| 0 | 1 | 0 | 3'o1 | x | 4'd 6 | 左移 |
| 0 | 1 | 0 | 3'o2 | x | 4'd9 | A<B?1:0 |
| 0 | 1 | 0 | 3'o3 | x | 4'd10 | A<B?1:0（无符号数) |
| 0 | 1 | 0 | 3'o4 | x | 4'd 4 | 异或 |
| 0 | 1 | 0 | 3'o5 | 0 | 4'd 7 | 右移 A >>B |
| 0 | 1 | 0 | 3'o5 | 1 | 4'd 8 | 算术右移 A >>>B |
| 0 | 1 | 0 | 3'o6 | x | 4'd 5 | 或 |
| 0 | 1 | 0 | 3'o7 | x | 4'd 3 | 与 |
| 0 | 0 | 1 | x | x | 4'd 2 | 送数:ALUResult=B |
| 其它 | | | | | 4'd 0 | 加 |

②Verilog 代码：

```
module Decode(
    // Outputs
    MemtoReg, RegWrite, MemWrite, MemRead,ALUCode,ALUSrcA,ALUSrcB,Jump,JALR,Imm,offset,
    // Inputs
    Instruction);
    input [31:0]    Instruction;    // current instruction
    output          MemtoReg;       // use memory output as data to write into register
    output          RegWrite;       // enable writing back to the register
    output          MemWrite;       // write to memory
    output          MemRead;
    output [3:0]    ALUCode;        // ALU operation select
    output          ALUSrcA;
    output [1:0]    ALUSrcB;
    output          Jump;
    output          JALR;
    output[31:0]    Imm,offset;

//*************************************************************************
//  instruction type decode
//*************************************************************************
    parameter  R_type_op=   7'b0110011;
    parameter  I_type_op=   7'b0010011;
    parameter  SB_type_op=  7'b1100011;
    parameter  LW_op=       7'b0000011;
    parameter  JALR_op=     7'b1100111;
    parameter  SW_op=       7'b0100011;
    parameter  LUI_op=      7'b0110111;
    parameter  AUIPC_op=    7'b0010111;
    parameter  JAL_op=      7'b1101111;
//
  //
    parameter  ADD_funct3 =     3'b000 ;
    parameter  SUB_funct3 =     3'b000 ;
    parameter  SLL_funct3 =     3'b001 ;
    parameter  SLT_funct3 =     3'b010 ;
    parameter  SLTU_funct3 =    3'b011 ;
    parameter  XOR_funct3 =     3'b100 ;
    parameter  SRL_funct3 =     3'b101 ;
    parameter  SRA_funct3 =     3'b101 ;
    parameter  OR_funct3 =      3'b110 ;
    parameter  AND_funct3 =     3'b111;
    //
```

```
    parameter  ADDI_funct3 =     3'b000 ;
    parameter  SLLI_funct3 =     3'b001 ;
    parameter  SLTI_funct3 =     3'b010 ;
    parameter  SLTIU_funct3 =    3'b011 ;
    parameter  XORI_funct3 =     3'b100 ;
    parameter  SRLI_funct3 =     3'b101 ;
    parameter  SRAI_funct3 =     3'b101 ;
    parameter  ORI_funct3 =      3'b101 ;
    parameter  ANDI_funct3 =     3'b111;
    //
    parameter     alu_add=  4'b0000;
    parameter     alu_sub=  4'b0001;
    parameter     alu_lui=  4'b0010;
    parameter     alu_and=  4'b0011;
    parameter     alu_xor=  4'b0100;
    parameter     alu_or =  4'b0101;
    parameter     alu_sll=  4'b0110;
    parameter     alu_srl=  4'b0111;
    parameter     alu_sra=  4'b1000;
    parameter     alu_slt=  4'b1001;
    parameter     alu_sltu= 4'b1010;

//*****************************************************************************
// instruction field
//*****************************************************************************
    wire [6:0]      op;
    wire            funct6_7;
    wire [2:0]      funct3;
    assign op           = Instruction[6:0];
    assign funct6_7     = Instruction[30];
    assign funct3       = Instruction[14:12];

    //指令类型
    wire R_type, I_type, SB_type, LW, SW, LUI, AUIPC, JAL;
    assign R_type = (op == R_type_op);
    assign I_type = (op == I_type_op);
    assign SB_type = (op == SB_type_op);
    assign LW = (op == LW_op);
    assign JALR = (op == JALR_op);
    assign SW = (op == SW_op);
    assign LUI = (op == LUI_op);
    assign AUIPC = (op == AUIPC_op);
    assign JAL = (op == JAL_op);

    //指令部分操作

    //只有LW读取存储器，并写回数据
    assign MemtoReg = LW;
    assign MemRead = LW;

    //只有SW写入存储器
    assign MemWrite = SW;

    //需要回写的指令
    assign RegWrite = R_type || I_type || LW || JALR || LUI || AUIPC || JAL;
```

```
    //只有SW写入存储器
    assign MemWrite = SW;

    //需要回写的指令
    assign RegWrite = R_type || I_type || LW || JALR || LUI || AUIPC || JAL;

    //无条件分支指令
    assign Jump = JAL || JALR;

    //操作数A和B的选择
    // A:rs or pc
    assign ALUSrcA = JALR || JAL || AUIPC;
    // B:rs or imm or 4
    assign ALUSrcB[1]=JAL || JALR;
    assign ALUSrcB[0]=~(R_type || JAL || JALR);

    //ALUCode
    reg [3:0] ALUCode_temp;
    always @(*) begin
      casez ({R_type,I_type,LUI,funct3,funct6_7})
          7'b100_000_0: ALUCode_temp = 4'd0; //加
          7'b100_000_1: ALUCode_temp = 4'd1; //减
          7'b100_001_0: ALUCode_temp = 4'd6; //左移
          7'b100_010_0: ALUCode_temp = 4'd9; //小于
          7'b100_011_0: ALUCode_temp = 4'd10; //无符号小于
          7'b100_100_0: ALUCode_temp = 4'd4; //异或
          7'b100_101_0: ALUCode_temp = 4'd7; //右移
          7'b100_101_1: ALUCode_temp = 4'd8; //算术右移
          7'b100_110_0: ALUCode_temp = 4'd5; //或
          7'b100_111_0: ALUCode_temp = 4'd3; //与
          7'b010_000_?: ALUCode_temp = 4'd0; //立即数加
          7'b010_001_?: ALUCode_temp = 4'd6; //立即数左移
          7'b010_010_?: ALUCode_temp = 4'd9; //立即数小于
          7'b010_011_?: ALUCode_temp = 4'd10; //立即数无符号小于
          7'b010_100_?: ALUCode_temp = 4'd4; //立即数异或
          7'b010_101_0: ALUCode_temp = 4'd7; //立即数右移
          7'b010_101_1: ALUCode_temp = 4'd8; //立即数算术右移
          7'b010_110_?: ALUCode_temp = 4'd5; //立即数或
          7'b010_111_?: ALUCode_temp = 4'd3; //立即数与
          7'b001_???_?: ALUCode_temp = 4'd2; //送数
          default: ALUCode_temp = 4'd0; //其他为加
      endcase
    end
    //输出ALUCode
    assign ALUCode = ALUCode_temp;
```

（2）立即数产生电路 ImmGen 模块设计：

①原理：

I_type、SB_type、LW、JALR、SW、LUI、AUIPC 和 JAL 这几类指令均用到立即数。由于 I_type 的算术逻辑运算与移位运算指令的立即数构成方法不同，这里再设定一个变量 Shift 来区分两者。shift=1 表示移位运算，否为则算术逻辑运算。

立即数构造和拓展方法如表 30.6 所示，inst 表示 instruction

表 30.6（a）立即数 imm 产生方法

| 类别 | Shift | Imm | 说明 |
|---|---|---|---|
| I_type | 1 | {27'd0,inst[24:20]} | 无符号数 0 扩展 |
| I_type | 0 | {{20{inst[31]}},inst[31:20]} | 有符号数符号扩展 |
| LW | x | | |
| SW | x | {{20{inst[31]}},inst[31:25],inst[11:7]} | |
| LUI | x | {inst[31:12], 12'd0} | |
| AUIPC | x | | |
| other | x | - | |

表 30.6（b）立即数 offset 产生方法

| 类别 | offset | 说明 |
|---|---|---|
| JALR | {{20{inst[31]}},inst[31:20]} | 有符号数符号扩展 |
| JAL | {{11{inst[31]}}, inst[31], inst[19:12] , inst[20], inst[30:21], 1'b0} | |
| SB_type | {{19{inst[31]}}, inst[31], inst[7],inst[30:25] , inst[11:8], 1'b0} | |
| other | - | |

②Verilog 代码：

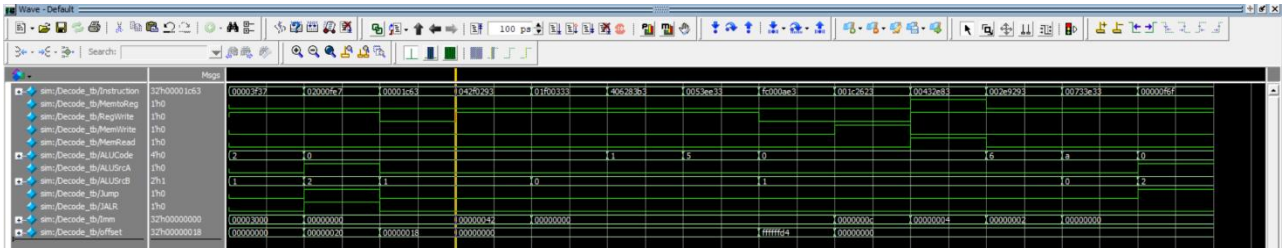
```
//立即数产生
wire Shift;
assign Shift=(funct3==1)||(funct3==5);
reg [31:0] Imm_temp;
//Imm 需要拓展位对齐运算
always @(*) begin
    if (I_type==1 && Shift==1)begin
      Imm_temp = {27'd0, Instruction[24:20]};
    end
    else if (I_type==1 && Shift==0)begin
      Imm_temp = {{20{Instruction[31]}}, Instruction[31:20]};
    end
    else if (LW==1) begin
      Imm_temp = {{20{Instruction[31]}}, Instruction[31:20]};
    end
    else if (SW==1)begin
      Imm_temp = {{20{Instruction[31]}}, Instruction[31:25], Instruction[11:7]};
    end
    else if (LUI==1)begin
      Imm_temp = {Instruction[31:12], 12'd0};
    end
    else if (AUIPC==1)begin
      Imm_temp = {Instruction[31:12], 12'd0};
    end
    else begin
      Imm_temp = 32'b0;//其他
    end
end
//输出Immt
assign Imm=Imm_temp;
```

```
    //offset
    reg [31:0] offset_temp;
    always @(*) begin
        if (JALR==1) begin
          offset_temp = {{20{Instruction[31]}}, Instruction[31:20]};
        end
        else if (JAL==1) begin
          offset_temp = {{11{Instruction[31]}}, Instruction[31], Instruction[19:12], Instruction[20], Instruction[30:21], 1'b0};
        end
        else if (SB_type==1) begin
          offset_temp = {{19{Instruction[31]}}, Instruction[31], Instruction[7], Instruction[30:25], Instruction[11:8], 1'b0};
        end
        else begin
          offset_temp = 32'b0;//default
        end
    end

    //输出offset
    assign offset=offset_temp;
```

（3）仿真分析

①仿真结果：



②波形分析：

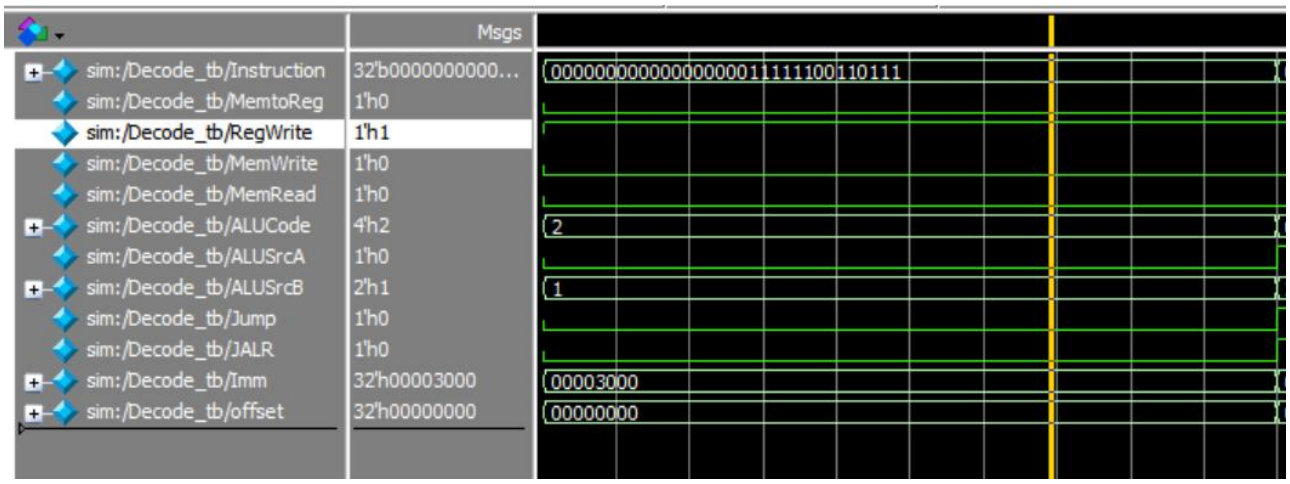
Decode_tb：

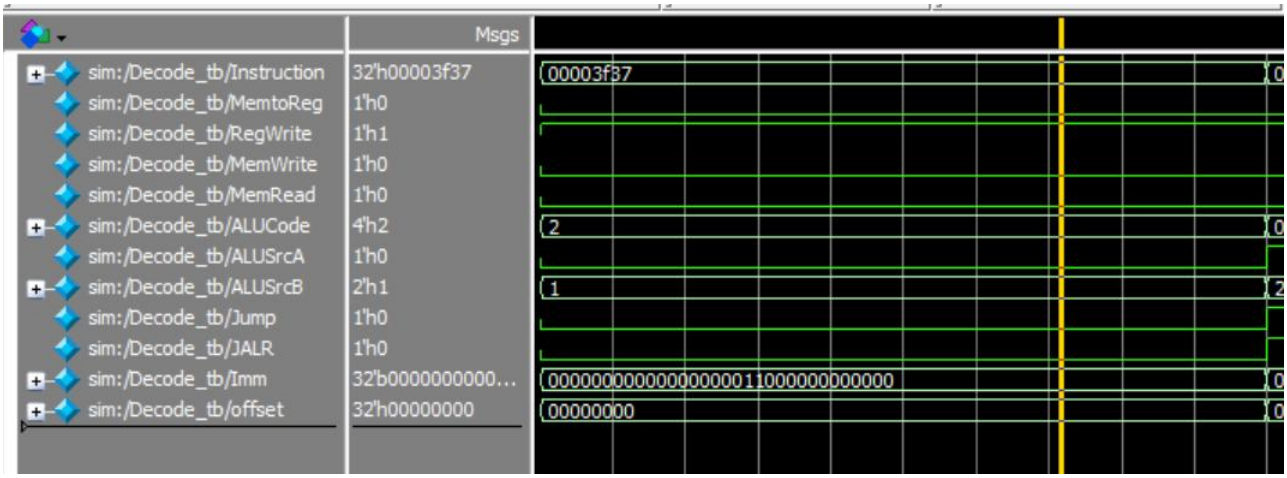
```
initial begin
    // Initialize Inputs
    Instruction = 32'h00003f37;//lui  X30, 0x3000

    // Add stimulus here
    #200 Instruction = 32'h02000fe7;//jalr X31, later(X0)
    #200 Instruction = 32'h00001c63;//bne  X0, X0, end
    #200 Instruction = 32'h042f0293;//addi X5, X30, 42
    #200 Instruction = 32'h01f00333;//add  X6, X0, X31
    #200 Instruction  = 32'h406283b3;//sub  X7, X5, X6
    #200 Instruction = 32'h0053ee33;//or     X28, X7, X5
    #200 Instruction = 32'hfc000ae3;//beq X0, X0, earlier
    #200 Instruction = 32'h001c2623;//sw  X28, 0C(X0)
    #200 Instruction = 32'h00432e83;//lw X29, 04(X6)
    #200 Instruction = 32'h002e9293;//sll X5, X29,2
    #200 Instruction = 32'h00733e33;//sltu X28, X6,X7
    #200 Instruction = 32'h0000f6f;//jal  X31, done

    #200 $stop;
end
```

以第一条指令为例分析，后面指令同理：






第一条指令为 lui　x30, 0x3000，属于 U_type 指令，对应的 instruction 为：

Imm[11:0]　　rs1　　func3　　rd　　opcode

000000000000　00000　000　11111　0110111

对应的指令译码为：

| | | |
|---|---|---|
| MemtoReg | MemtoReg=LW | 非 LW 指令，为 0 |
| RegWrite | RegWrite=I_type \|\| LUI \|\| | 为 LUI 指令，为 1 |
| MemWrite | MemWrite=SW | 非 SW 指令，为 0 |
| ALUCode | R_type,I_type,LUI,funct3,funct6_7 | LUI=1,ALUCode=2 |
| ALUSrcA | ALUSrcA=JALR \|\| JAL \|\| AUIPC | 不属于这三种指令，为 0 |
| ALUSrcB | ALUSrcB[1]= JAL \|\| JALR，<br>ALUSrcB[0]=~(R_type \|\| JAL \|\| JALR) | ALUSrcB[1]=0<br>ALUSrcB[0]=1 |
| Jump | Jump=JALR \|\| JAL | Jump=0 |
| JALR | --- | 不属于 JALR，为 0 |
| Imm | 为 LUI，{inst[31:12],12'd0} | 000...11+000..00 |
| offset | JALR、JAL、SB_Type | Other，其他为 0 |
| 和实际波形对应！因此该模块符合要求 | | |

## 3．分支检测(BranchTest)电路的设计

（1）原理：

分支检测电路主要用于判断分支条件是否成立，在 VerilogHDL 可以用比较运算符号“>”、

“-”和“<”描述，但要注意符号数和无符号数的处理方法不同。在这里，我们用加法器来实现。

①用一个 32 位加法器完成 rsIData+(~rs2Data)+1(即 rslData-rs2Data)，设结果为 sum [31:0]。

②确定比较运算的结果。对于比较运算来说，如果最高位不同，即 rsIData[31]≠rs2Data [31]，可根据 rs1Data[31]、rs2Data[31]决定比较结果，但是应注意符号数、无符号数的最高位 rslData [31]、rs2Data [31]代表意义不同。若两数最高位相同，则两数之差不会溢出，所以比较运算结果可由两个操作数之差的符号位 sum[31]决定。

在符号数比较运算中，rs1Data<rs2Data 有以下两种情况:

a)rs1Data 为负数、rs2Data 为 0 或正数:rs1Data[31]&&(~rs2Data[31])

b)rslData、rs2Data 符号相同,sům 为负:(rslData[31]~^rs2Data[31])&& sum[31]

无符号数比较运算中，rs1Data<rs2Data 有以下两种情况:

a) rs1Data 最高位为 0、rs2Data 最高位为 1:(~rslData[31])&& rs2Data[31]

b)rslData、rs2Data 最高位相同，sum 为负:(rslData[31]~^rs2Data[31])&& sum31]

最后用数据选择器完成式(30.14)即可完成分支检测。

$$Branch = \begin{cases} \sim(|sum[31:0]); & SB_type \ \&\& \ (funct3 == beq_funct3) \\ |sum[31:0]; & SB_type \ \&\& \ (funct3 = bne_funct3) \\ isLT; & SB_type \ \&\& (funct3 = blt_funct3) \\ \sim isLT; & SB_type \ \&\& \ (funct3 = bge_funct3) \\ isLTU; & SB_type \ \&\& \ (funct3 = bltu_funct\ 3) \\ \sim isLTU; & SB_type \ \&\& \ (funct3 = bgeu_funct\ 3) \\ 0 & others \end{cases} \tag{30.14}$$

（2）Verilog 代码：

```
module BranchTest(rs1Data,rs2Data,SB_type,funct3,Branch);
    input [31:0] rs1Data;
    input [31:0] rs2Data;
    input SB_type;
    input [2:0] funct3;
    output Branch;

    wire [31:0] sum;
    adder_32bits adder(.a(rs1Data),.b(~rs2Data),.ci(1'b1),.s(sum),.co());

    wire isLT, isLTU;
    assign isLT = rs1Data[31] && (~rs2Data[31]) || (rs1Data[31] ~^ rs2Data[31]) && sum[31];
    assign isLTU = (~rs1Data[31]) && rs2Data[31] || (rs1Data[31] ~^ rs2Data[31]) && sum[31];

    // funct3
    parameter beq_funct3 = 3'b000;
    parameter bne_funct3 = 3'b001;
    parameter blt_funct3 = 3'b100;
    parameter bge_funct3 = 3'b101;
    parameter bltu_funct3 = 3'b110;
    parameter bgeu_funct3 = 3'b111;
```

```
    reg Branch_temp;
    always @(*) begin
        if (SB_type == 1 && funct3 == beq_funct3) begin
            Branch_temp = ~(|sum[31:0]);
        end
        else if (SB_type == 1 && funct3 == bne_funct3) begin
            Branch_temp = |sum[31:0];
        end
        else if (SB_type == 1 && funct3 == blt_funct3) begin
            Branch_temp = isLT;
        end
        else if (SB_type == 1 && funct3 == bge_funct3) begin
            Branch_temp = ~isLT;
        end
        else if (SB_type == 1 && funct3 == bltu_funct3) begin
            Branch_temp = isLTU;
        end
        else if (SB_type == 1 && funct3 == bgeu_funct3) begin
            Branch_temp = ~isLTU;
        end
        else begin
            Branch_temp = 0;
        end
    end
    // 输出Branch
    assign Branch = Branch_temp;
endmodule
```

## 4. 冒险检测功能电路(HazardDetector)的设计

（1）原理：

冒险成立的条件为:①上一条指令必须是 lw 指令(MemRead ex1);②两条指令读写同一个寄存器(rdAddr ex=rslAddr id 或 rdAddr ex=rs2Addr id)。冒险检测功能电路(Hazard Detector)等功能单元比较简单，直接在 ID 顶层描述。

（2）Verilog 代码：

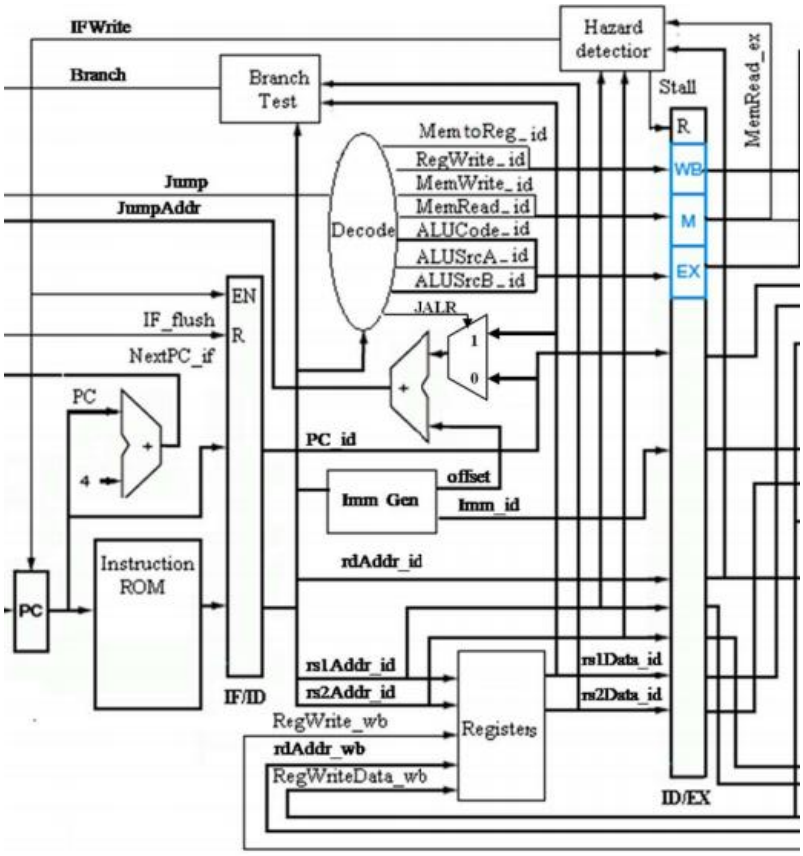
```
//冒险检测
assign Stall=((rdAddr_ex==rs1Addr_id)||(rdAddr_ex==rs2Addr_id))&&MemRead_ex;
assign IFWrite=~Stall;
```

## 5. 顶层代码 ID 的设计：

（1）原理框图：

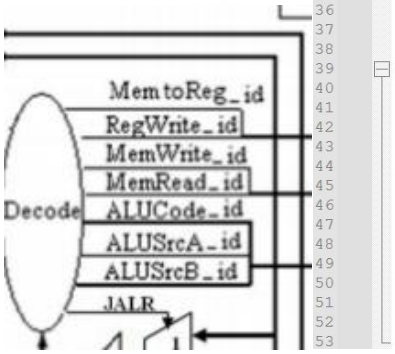
输入端口： clk,Instruction_id, PC_id, RegWrite_wb, rdAddr_wb, RegWriteData_wb, MemRead_ex, rdAddr_ex

输出端口： MemtoReg_id, RegWrite_id, MemWrite_id, MemRead_id, ALUCode_id, ALUSrcA_id, ALUSrcB_id, Stall, Branch, Jump, IFWrite, JumpAddr, Imm_id,rs1Data_id, rs2Data_id,rs1Addr_id,rs2Addr_id,rdAddr_id



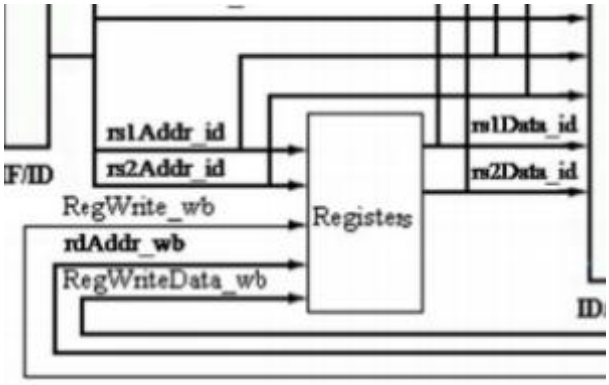

（2）Verilog 代码：

①Decode+Imm 模块：




```
36      //Decode+Imm模块
37      wire JALR;
38      wire [31:0]offset_id;
39      Decode Decode_id(
40          .Instruction(Instruction_id),
41
42          .JALR(JALR),
43          .MemtoReg(MemtoReg_id),
44          .RegWrite(RegWrite_id),
45          .MemWrite(MemWrite_id),
46          .MemRead(MemRead_id),
47          .ALUCode(ALUCode_id),
48          .ALUSrcA(ALUSrcA_id),
49          .ALUSrcB(ALUSrcB_id),
50          .Imm(Imm_id),
51          .Jump(Jump),
52          .offset(offset_id)
53      );
```

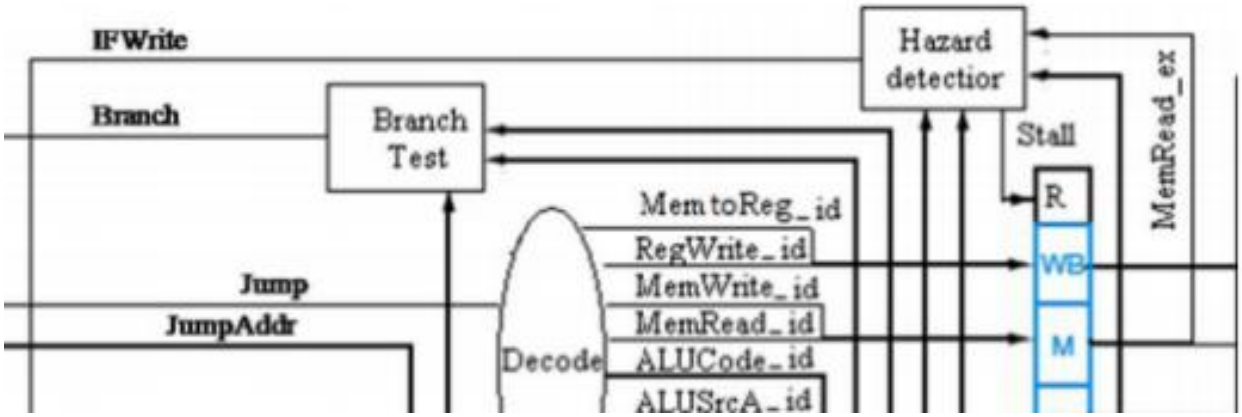
②寄存器堆 Registers




```
//寄存器地址
assign rs1Addr_id = Instruction_id[19:15];
assign rs2Addr_id = Instruction_id[24:20];
assign rdAddr_id = Instruction_id[11:7];
RAWRegisters RAWRegisters_id(
    .clk(clk),
    .rs1Addr(rs1Addr_id),
    .rs2Addr(rs2Addr_id),
    .WriteAddr(rdAddr_wb),
    .RegWrite(RegWrite_wb),
    .WriteData(RegWriteData_wb),

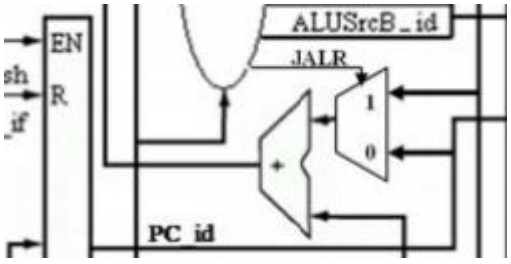
    .rs1Data(rs1Data_id),
    .rs2Data(rs2Data_id)
);
```

③分支检测+冒险检测




```
70      // 分支检测
71      wire SB_type;
72      assign SB_type=(Instruction_id[6:0]==7'b1100011);
73      wire [2:0] funct3_id;
74      assign funct3_id=Instruction_id[14:12];
75      BranchTest BranchTest_id(
76          .rs1Data(rs1Data_id),
77          .rs2Data(rs2Data_id),
78          .SB_type(SB_type),
79          .funct3(funct3_id),
80
81          .Branch(Branch)
82      );
83      //冒险检测
84      assign Stall=((rdAddr_ex==rs1Addr_id)||(rdAddr_ex==rs2Addr_id))&&MemRead_ex;
85      assign IFWrite=~Stall;
```

④JALR 跳转判断




```
    //跳转判断
    mux_2to1 #(.n(32)) mux_JALR_id(.out(JumpAddr), .in0(PC_id+offset_id), .in1(rs1Data_id+offset_id), .addr(JALR));
endmodule
```

## （三）RISC-V：执行模块(EX)的设计

执行模块主要由 ALU 子模块、数据前推电路(Forwarding)及若干数据选择器组成。执行模块的接口信息如表 30.7 所示。

表 30.7　EX 模块的输入/输出引脚说明

| 引脚名称 | 方向 | 说明 |
|---|---|---|
| ALUCode_ex[3:0] | Input | 决定 ALU 采用何种运算 |
| ALUSrcA_ex | | 决定 ALU 的 A 操作数的来源（rs1、PC） |
| ALUSrcB_ex[1:0] | | 决定 ALU 的 B 操作数的来源(rs2、imm 和常数 4) |
| Imm_ex[31:0] | | 立即数 |
| rs1Addr_ex[4:0] | | rs1 寄存器地址 |
| rs2Addr_ex[4:0] | | rs2 寄存器地址 |
| rs1Data_ex[31:0] | | rs1 寄存器数据 |
| rs2Data_ex[31:0] | | rs2 寄存器数据 |
| PC_ex[31:0] | | 指令指针 |
| RegWriteData_wb[31:0] | | 写入寄存器的数据 |
| ALUResult_mem[31:0] | | ALU 输出数据 |
| rdAddr_mem[4:0] | | 寄存器的写地址 |
| rdAddr_wb[4:0] | | |
| RegWrite_mem | | 寄存器写允许信号 |
| RegWrite_wb | | |
| ALUResult_ex[31:0] | Output | ALU 运算结果 |
| MemWriteData_ex[31:0] | | 存储器的回写数据 |
| ALU_A [31:0] | | ALU 操作数，测试时使用 |
| ALU_B [31:0] | | |

## 1．ALU 子模块的设计

（1）原理：

算术逻辑运算单元(ALU)提供 CPU 的基本运算能力，如加、减、与、或、比较、移位等。具体而言,ALU 输入为两个操作数 A、B 和控制信号 ALUCode，由控制信号 ALUCode 决定采用何种运算，运算结果为 ALUResult。整理表 30.5 所示的 ALUCode 的功能表，可得到 ALU 的功能表，如表 30.8 所示。

表 30.8　ALU 的功能表

| ALUCode | ALUResult |
|---|---|
| 4'b0000 | A + B |
| 4'b0001 | A-B |
| 4'b0010 | B |
| 4'b0011 | A&B |
| 4'b0100 | A ^ B |
| 4'b0101 | A \| B |
| 4'b0110 | A << B |
| 4'b0111 | A >> B |
| 4'b1000 | A>>>B |
| 4'b1001 | A<B? 1:0，其中 A、B 为有符号数 |
| 4'b1010 | A<B? 1:0，其中 A、B 为无符号数 |

ALU 需执行多种运算，为了提高运算速度，本设计可同时进行各种运算，再根据 ALUCode 信号选出所需结果。

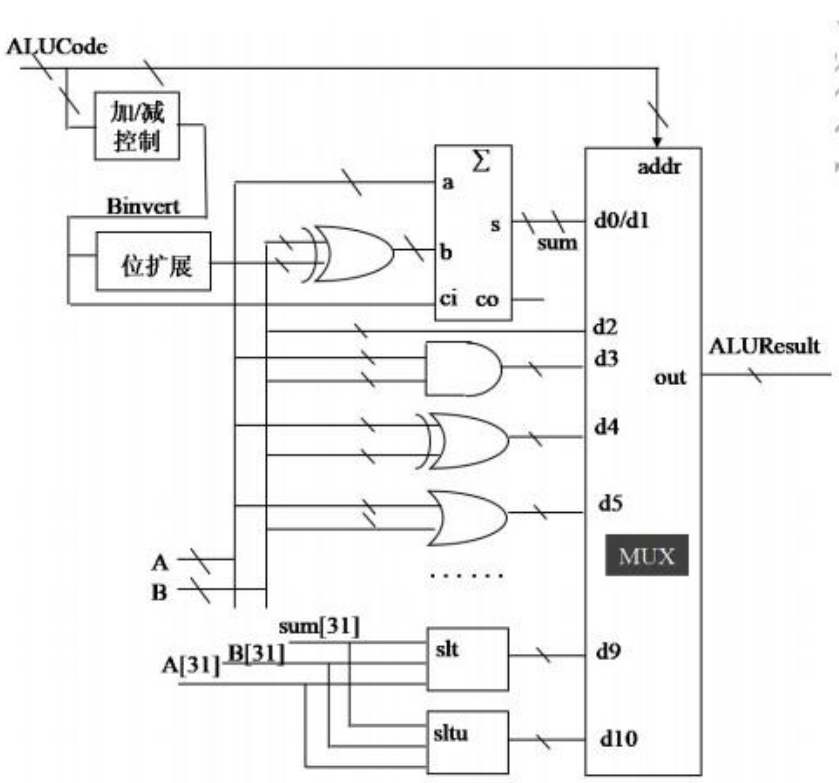



图 30.10 ALU 结构框图

①加、减电路的设计考虑

减法、比较(slt、sltu)均可用加法器和必要辅助电路来实现。图 30.10 中的 Binven 信号控制加减运算:若 Binvert 信号为低电平，则实现加法运算:sum=A+B;若 Binvert 信号为高电平，则电路为减法运算 sum=A-B。除加法外，减法、比较和分支指令都应使电路工作在减法状态，所以:Binvert=~(ALUCode==0)

最后要强调的是，32 位加法器的运算速度决定了 RISC-V 微处理器的时钟信号频率的高低，因此设计一个高速的 32 位加法器尤为重要。32 位加法器可采用进位选择加法器

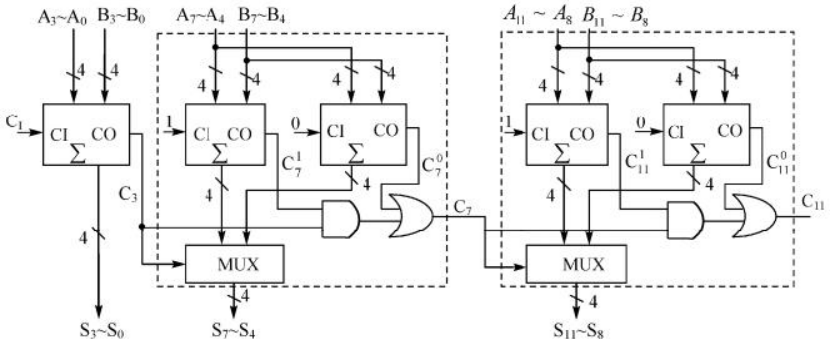



图 5.6 12 位进位选择加法器原理图

②比较电路的设计

考虑比较电路的设计方法已在分支检测电路介绍，参考式(30.12)和式(30.13)可确定 stt 和 sltu 两条件的比较结果。可以详见 Branch 分支电路的设计

③算术右移运算电路的设计

考虑算术右移对有符号数而言，移出的高位补符号位而不是 0。每右移一位相当于除以 2。Verilog HDL,的算术右移的运算符是“>>>”。要实现算术右移应注意，被移位的对象必须定义是 reg 类型，但是在 sra 指令，被移位的对象操作数 A 为输入信号，不能定义为 reg 类型。因此，必须引入 reg 类型中间变量 A_reg，引入 reg 类型的中间变量 A_reg 后，就可对 A_reg 进行算术右移操作。

```
reg signed[31:0] A_reg;
always @(*)  begin  A_reg=A;  end
```

（2）Verilog 代码：

```
//除加法外均处于减法状态，低电平加法
wire Binvert;
assign Binvert=~(ALUCode==0);
//d0_d1 加减
wire [31:0] d0_d1;
adder_32bits adder_32bits_ALU (
  .a(A),
  .b({32{Binvert}} ^ B),
  .ci(Binvert),
  .s(d0_d1),
  .co()
);
//d2 B的值
wire [31:0] d2;
assign d2=B;
//d3 与门
wire [31:0] d3;
assign d3=A&B;
//d4 异或门
wire [31:0] d4;
assign d4=A^B;
//d5 或门
wire [31:0] d5;
assign d5=A|B;

//d6 左移
wire [31:0] d6;
//A最多移动32位，只需要B的低五位
assign d6=A<<B[4:0];
//d7 右移
wire [31:0] d7;
assign d7=A>>B[4:0];

//d8 算术右移
wire [31:0] d8;
reg signed [31:0] A_reg;
always @(*) begin A_reg=A; end
assign d8=A_reg>>>B[4:0];

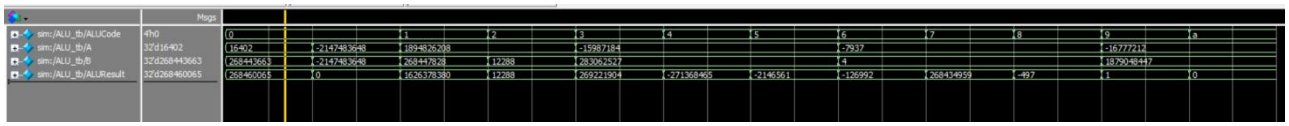
//d9 有符号小于 slt
wire [31:0] d9;
assign d9=A[31] && (~B[31]) || (A[31]~^B[31]) && d0_d1[31];
//d10 无符号小于 sltu
wire [31:0] d10;
assign d10=(~A[31]) && B[31] || (A[31]~^B[31]) && d0_d1[31];

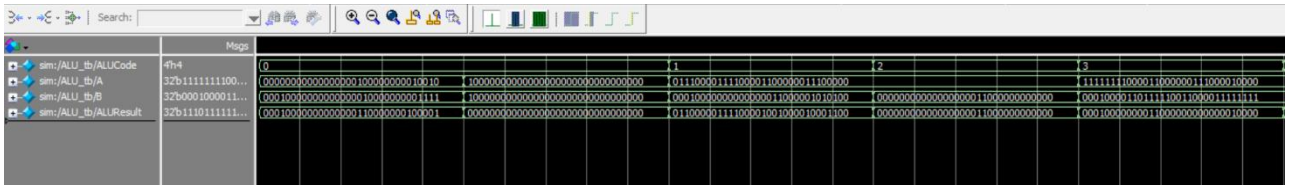
reg [31:0] ALU_out;
always @(*) begin
  case (ALUCode)
      alu_add: ALU_out=d0_d1;
      alu_sub: ALU_out=d0_d1;
      alu_lui: ALU_out=d2;
      alu_and: ALU_out=d3;
      alu_xor: ALU_out=d4;
      alu_or: ALU_out=d5;
      alu_sll: ALU_out=d6;
      alu_srl: ALU_out=d7;
      alu_sra: ALU_out=d8;
      alu_slt: ALU_out=d9;
      alu_sltu: ALU_out=d10;
      default: ALU_out=d0_d1 ; //其他为加
  endcase
end

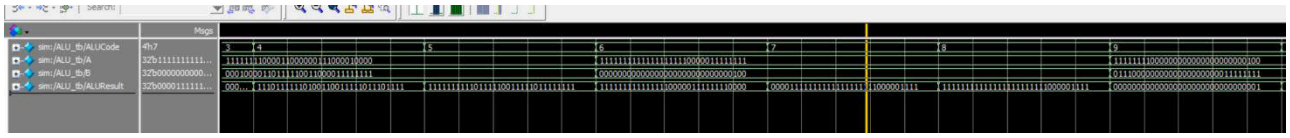
assign ALUResult=ALU_out;
```

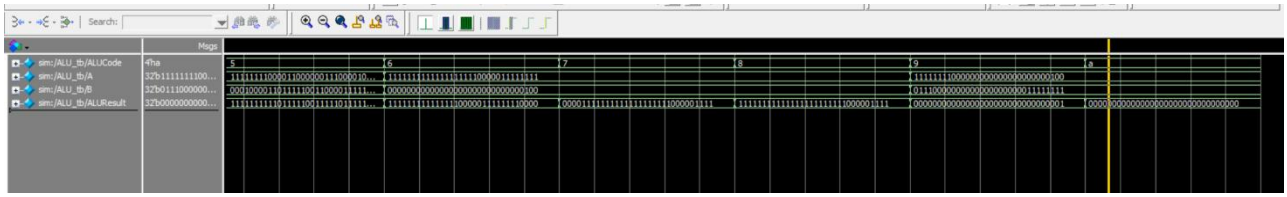
（3）仿真波形分析：

①仿真波形：





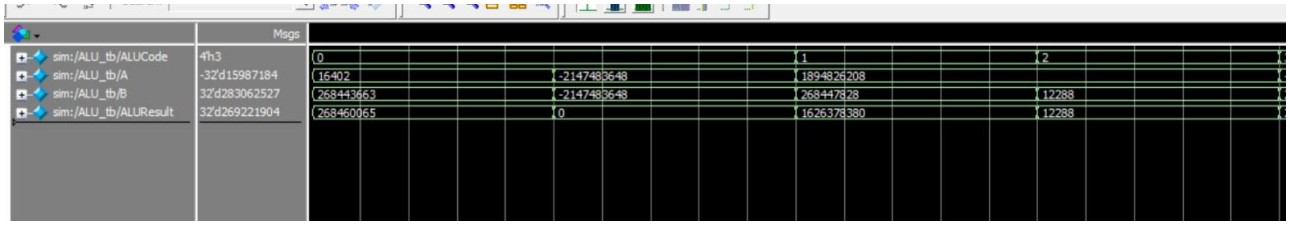




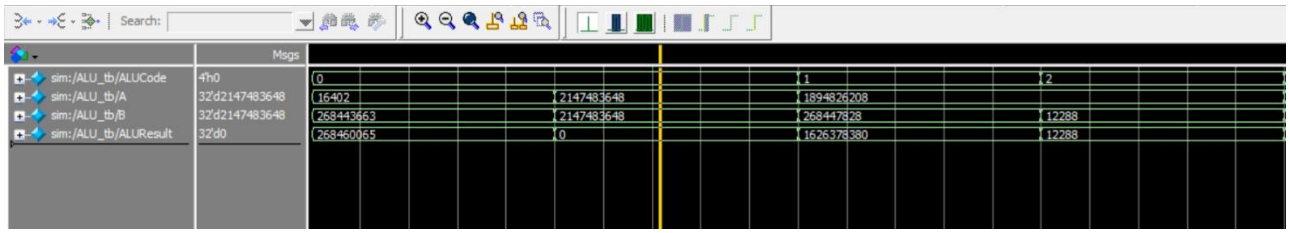
②波形结果分析：

1）ALU：加、减、输出 B

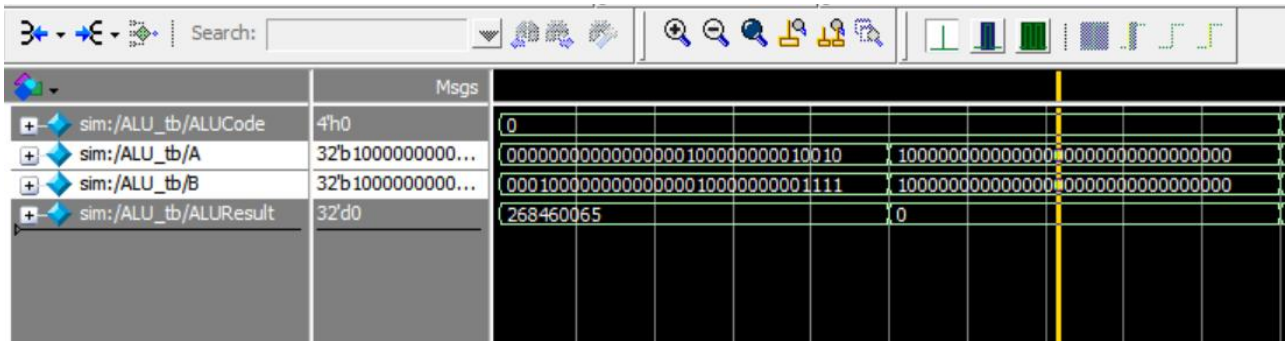
Decimal：



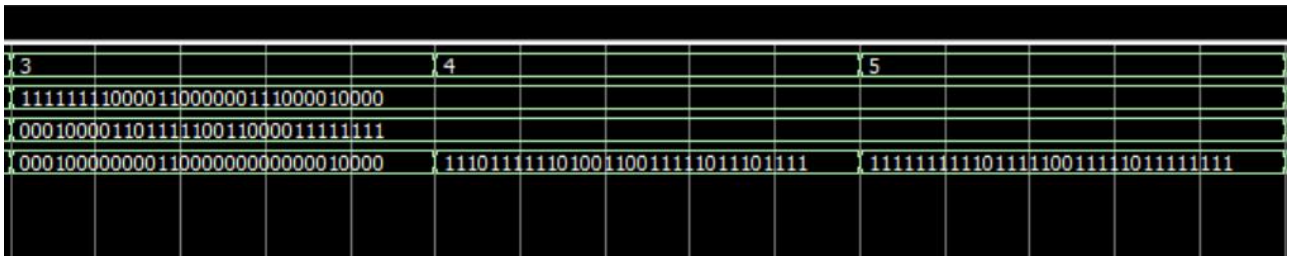

Unsigned：




从结果可以看出：code=0，加法模式，有符号十进制 16402+268443663=268460065，十进制 2147483648+2147483648=0 是因为溢出了，可以从下图看出：code=1，减法模式，十进制 1894826208-268447828=1626378380，符合；code=2，直接输出 B，输入为 12288，result 也是 12288，符合

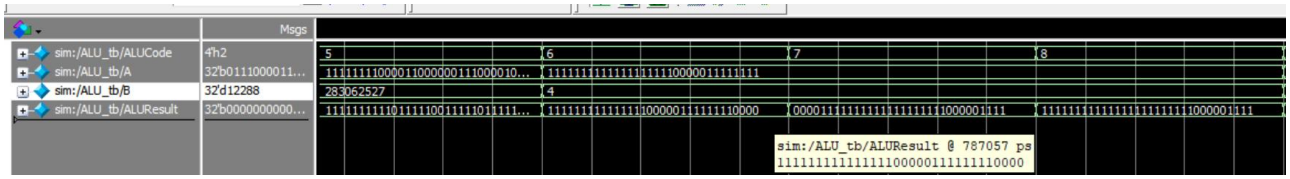



2）ALU：与、异或、或



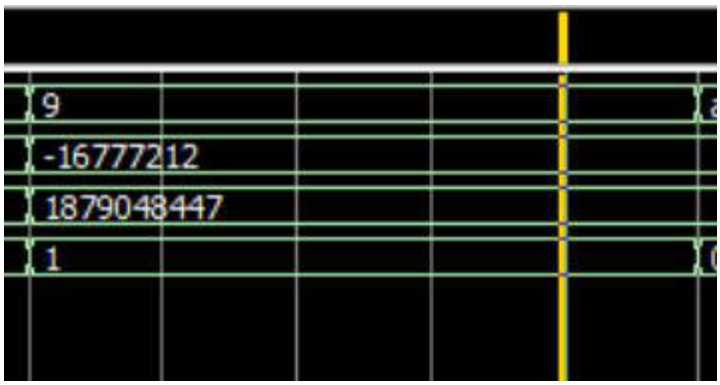

从结果可以看出，code=3、4、5 时，逻辑功能均符合结果

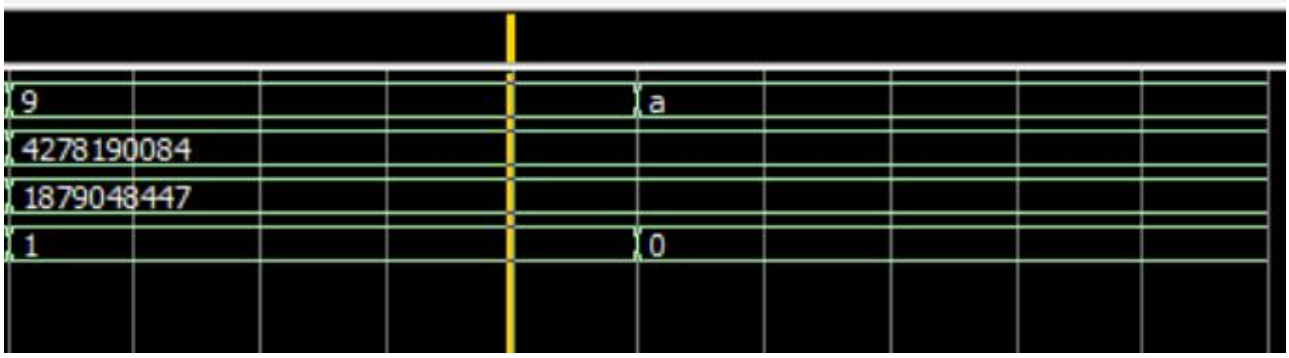
3）ALU：左右移位，算数移位




从结果可以看出，code=6、7、8 时，由于是对 A 进行移位，所以二进制 A，十进制 B，所以是移位 4，均符合结果

4）ALU：大小比较






从结果可以看出，code=9 时，A<B，因此输出结果为 1；code=10（16 进制为 a），A>B，因此输出结果是 0，均符合结果

## 2．数据前推电路的设计

（1）原理：

操作数 A 和 B 分别由数据选择器决定，数据选择器地址信号 ForwardA、ForwardB 的含义如表 30.9 所示

表 30.9 前推电路输出信号的含义

| 地址 | 操作数来源 | 说明 |
|---|---|---|
| ForwardA=2'b00 | rs1Data_ex | 操作数 A 来自寄存器堆 |
| ForwardA=2'b01 | RegWriteData_wb | 操作数 A 来自二阶数据相关的转发数据 |
| ForwardA=2'b10 | ALUResult_mem | 操作数 A 来自一阶数据相关的转发数据 |
| ForwardB=2'b00 | rs2Data_ex | 操作数 B 来自寄存器堆 |
| ForwardB=2'b01 | RegWriteData_wb | 操作数 B 来自二阶数据相关的转发数据 |
| ForwardB=2'b10 | ALUResult_mem | 操作数 B 来自一阶数据相关的转发数据 |

由前面介绍的一、二阶数据相关判断条件，不难得到

$$\begin{cases} \text{ForwardA}[0] = \text{RegWrite_wb} \,\&\, \&(\text{rdAddr_wb} != 0) \,\&\, \& \\ \qquad (\text{rdAddr_mem} != \text{rs1Addr_ex}) \,\&\, \& \\ \qquad (\text{rdAddr_wb} == \text{rs1Addr_ex}) \\ \text{ForwardA}[1] = \text{RegWrite_mem} \,\&\, \&(\text{rdAddr_mem} != 0) \,\&\, \& \\ \qquad (\text{rdAddr_mem} == \text{rs1Addr_ex}) \end{cases} \tag{30.18}$$

$$\begin{cases} \text{ForwardB}[0] = \text{RegWrite_wb} \,\&\, \&(\text{rdAddr_wb} != 0) \,\&\, \& \\ \qquad (\text{rd_mem} != \text{rs2Addr_ex}) \,\&\, \& \\ \qquad (\text{rdAddr_wb} == \text{rs2Addr_ex}) \\ \text{ForwardB}[1] = \text{RegWrite_mem} \,\&\, \&(\text{rdAddr_mem} != 0) \,\&\, \& \\ \qquad (\text{rdAddr_mem} == \text{rs2Addr_ex}) \end{cases} \tag{30.19}$$

（2）Verilog 代码：

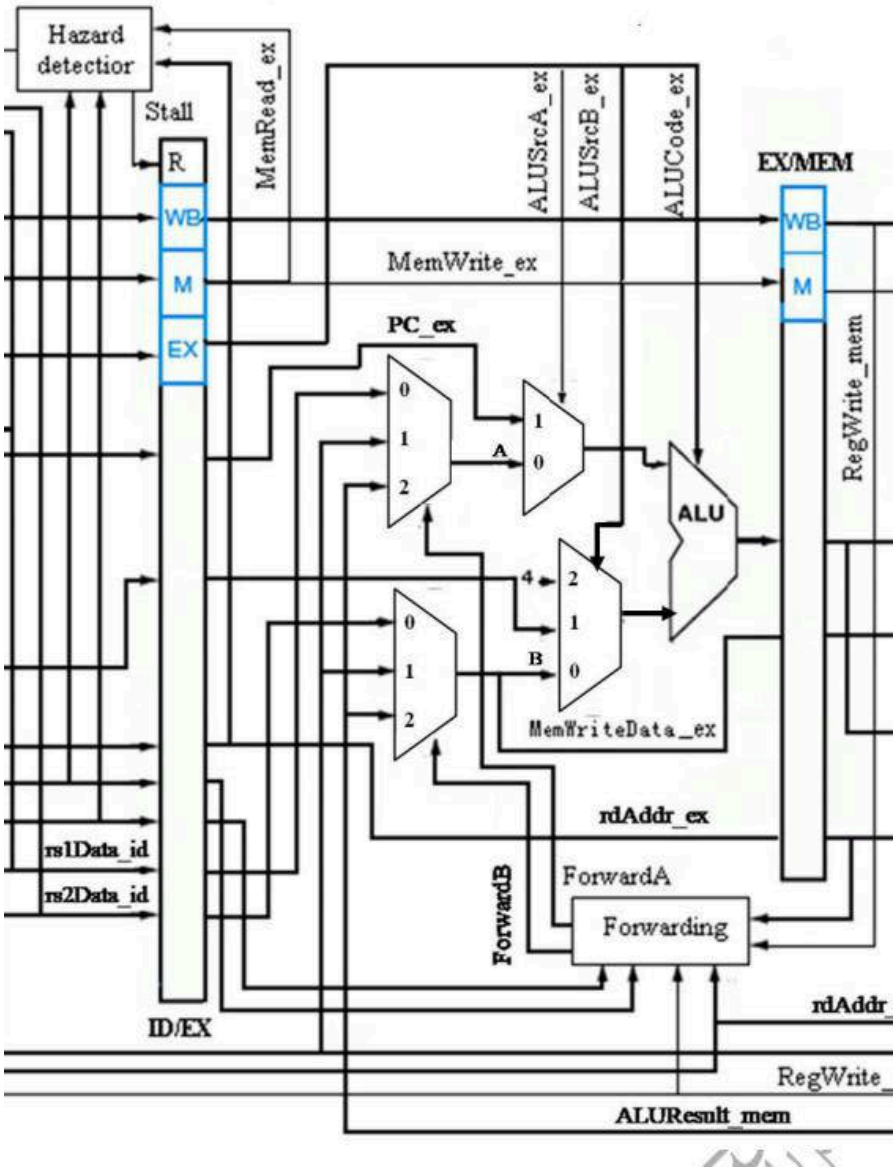
```
//数据前推电路Forwarding
wire [1:0]ForwardA,ForwardB;
assign ForwardA[0]=RegWrite_wb && (rdAddr_wb!=0) && (rdAddr_mem!=rs1Addr_ex) && (rdAddr_wb==rs1Addr_ex);
assign ForwardA[1]=RegWrite_mem && (rdAddr_mem!=0) && (rdAddr_mem==rs1Addr_ex);
assign ForwardB[0]=RegWrite_wb && (rdAddr_wb!=0) && (rdAddr_mem!=rs2Addr_ex) && (rdAddr_wb==rs2Addr_ex);
assign ForwardB[1]=RegWrite_mem && (rdAddr_mem!=0) && (rdAddr_mem==rs2Addr_ex);
```

## 3．顶层代码 EX 设计

（1）原理框图：

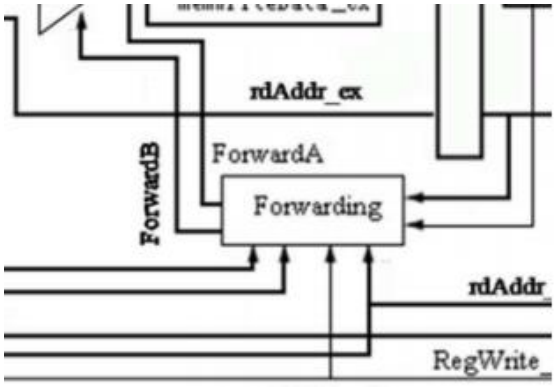
输入端口：ALUCode_ex, ALUSrcA_ex, ALUSrcB_ex,Imm_ex, rs1Addr_ex, rs2Addr_ex, rs1Data_ex, rs2Data_ex, PC_ex, RegWriteData_wb, ALUResult_mem,rdAddr_mem, rdAddr_wb, RegWrite_mem, RegWrite_wb

输出端口：ALUResult_ex, MemWriteData_ex, ALU_A, ALU_B



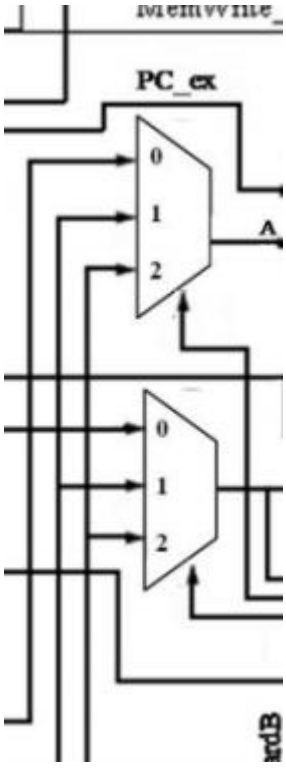
（2）Verilog 代码：

①Forwarding 部分：负责产生选通信号



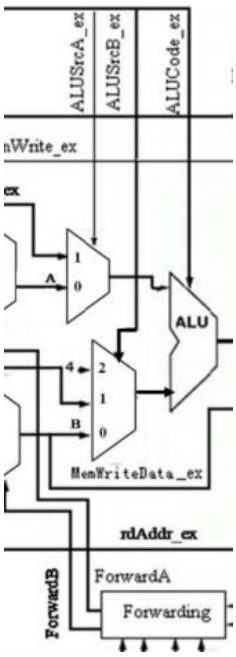

```
//数据前推电路Forwarding
wire [1:0]ForwardA,ForwardB;
assign ForwardA[0]=RegWrite_wb && (rdAddr_wb!=0) && (rdAddr_mem!=rs1Addr_ex) && (rdAddr_wb==rs1Addr_ex);
assign ForwardA[1]=RegWrite_mem && (rdAddr_mem!=0) && (rdAddr_mem==rs1Addr_ex);
assign ForwardB[0]=RegWrite_wb && (rdAddr_wb!=0) && (rdAddr_mem!=rs2Addr_ex) && (rdAddr_wb==rs2Addr_ex);
assign ForwardB[1]=RegWrite_mem && (rdAddr_mem!=0) && (rdAddr_mem==rs2Addr_ex);
```

②第一级 MUX，产生数据 A、B




```
//Forwarding_mux
wire [31:0] A,B;
mux_3to1 #(.n(32)) mux_Forward_A_ex(.out(A), .in0(rs1Data_ex), .in1(RegWriteData_wb), .in2(ALUResult_mem), .addr(ForwardA));
mux_3to1 #(.n(32)) mux_Forward_B_ex(.out(B), .in0(rs2Data_ex), .in1(RegWriteData_wb), .in2(ALUResult_mem), .addr(ForwardB));
```

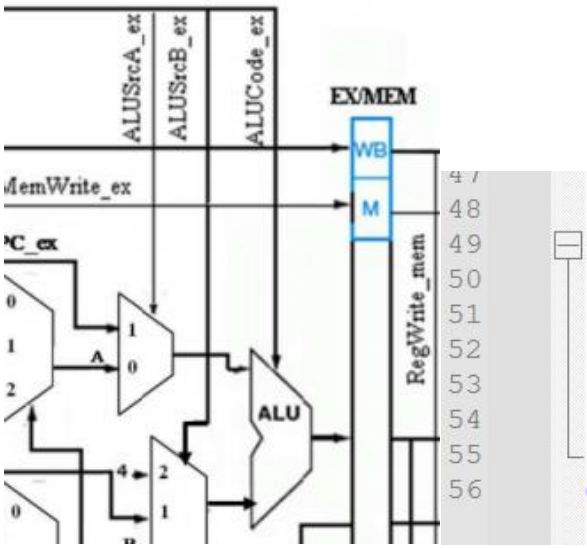
③第二级 MUX，产生进入 ALU 的数据 ALU_A，ALU_B




```
mux_3to1 #(.n(32)) mux_Forward_B_ex(.out(B), .in0(rs2Data_ex), .in1(RegWriteData_wb), .in2(ALUResult_mem), .addr(Forward

//ALU_mux
assign MemWriteData_ex=B;
mux_2to1 #(.n(32)) mux_ALUSrcA_ex(.out(ALU_A), .in0(A), .in1(PC_ex), .addr(ALUSrcA_ex));
mux_3to1 #(.n(32)) mux_ALUSrcB_ex(.out(ALU_B), .in0(B), .in1(Imm_ex), .in2(32'h00000004), .addr(ALUSrcB_ex));
```

④根据产生的 ALU_A，ALU_B，调用 ALU 模块




```
48      //ALU
49      ALU ALU_ex(
50          .ALUCode(ALUCode_ex),
51          .A(ALU_A),
52          .B(ALU_B),
53
54          .ALUResult(ALUResult_ex)
55      );
56  endmodule
```

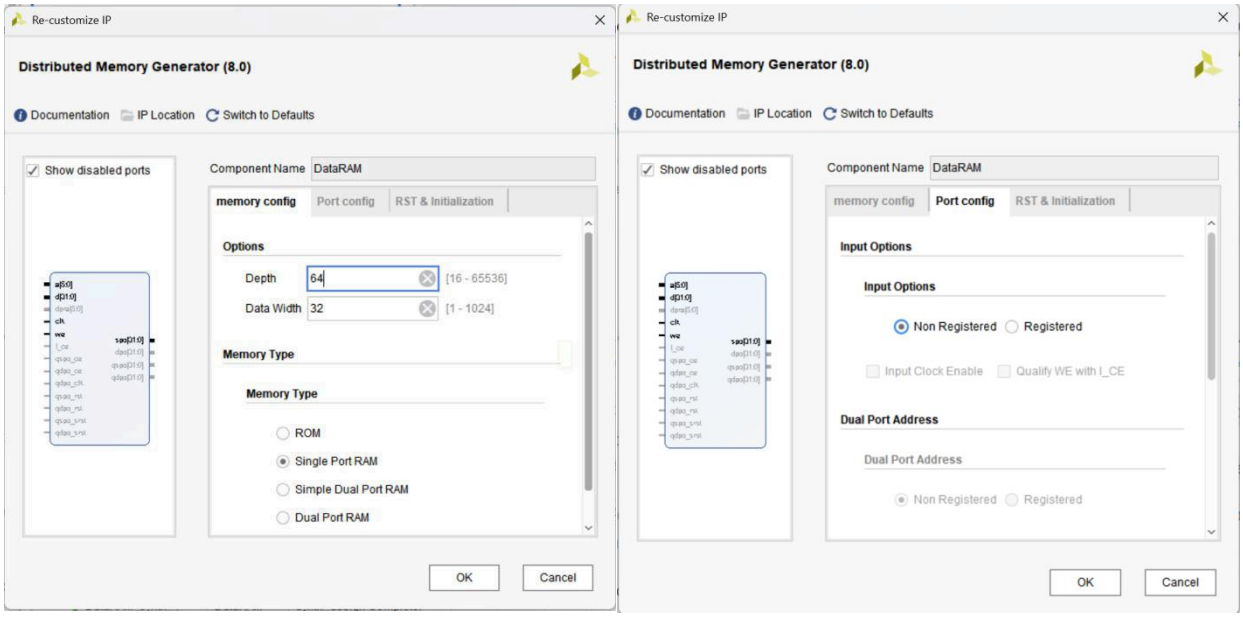
## （四）RISC-V：数据存储器模块(DataRAM)的设计

1．原理：

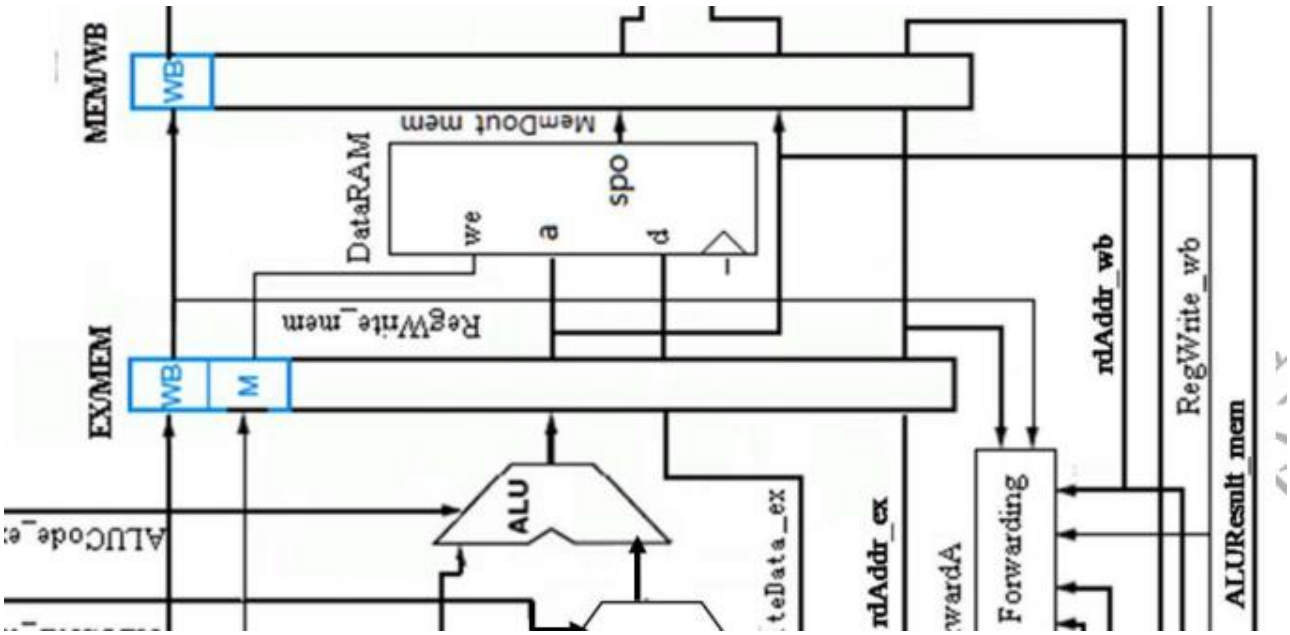
数据存储器可用 Xilinx 的 IP 内核实现。考虑到 FPGA 的资源，数据存储器可设计为容量为 64x32 bit 的单端口 RAM，输出采用组合输出(Non Registered)。

由于数据存储器容量为 64x32 bit，故存储器地址共 6 位，与 ALUResult mem[7:2]连接。

2．过程：




3．Verilog 代码：在 Risc5CPU 顶层代码中，作为 MEM 级使用




```
//MEM
wire [31:0] MemDout_mem;
DataRAM DataRAM_inst(
    .a(ALUResult_mem[7:2]),
    .d(MemWriteData_mem),
    .clk(clk),
    .we(MemWrite_mem),
    .spo(MemDout_mem)
);
```

## （五）RISC-V：取指令级模块(IF)的设计

1．原理：

IF 模块由指令指针寄存器(PC)、指令存储器子模块(Instruction ROM)、指令指针选择器(MUX)和一个 32 位加法器组成，IF 模块接口信息如表 30.10 所示。

表 30.10 IF 模块的输入/输出引脚说明

| 引脚名称 | 方向 | 说明 |
|---|---|---|
| clk | Input | 系统时钟 |
| reset | | 系统复位信号，高电平有效 |
| Branch | | 条件分支指令的条件判断结果 |
| Jump | | 无条件分支指令的条件判断结果 |
| IFWrite | | 流水线阻塞信号 |
| JumpAddr[31:0] | | 分支地址 |
| Instruction [31:0] | Output | 指令机器码 |
| IF_flush | | 流水线清空信号 |
| PC [31:0] | | PC 值 |

由图 30.3 可看出，指令存储器为组合存储器，可用 Verilog HDL,设计一个查找表阵列 ROM。考虑到 FPGA 的资源，该 ROM 容量可设计为 64x32bit。作者提供一个指令存储器模块 InstructionROM.v

输入端口：clk, reset, Branch,Jump, IFWrite, JumpAddr

输出端口：Instruction_if,PC,IF_flush

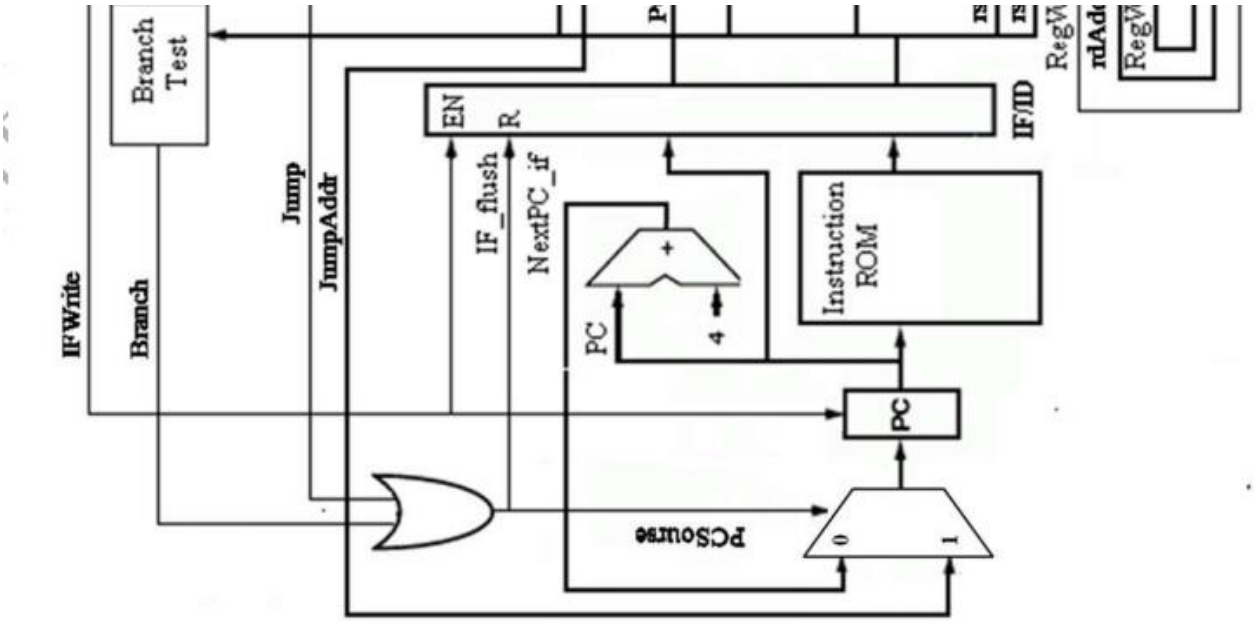
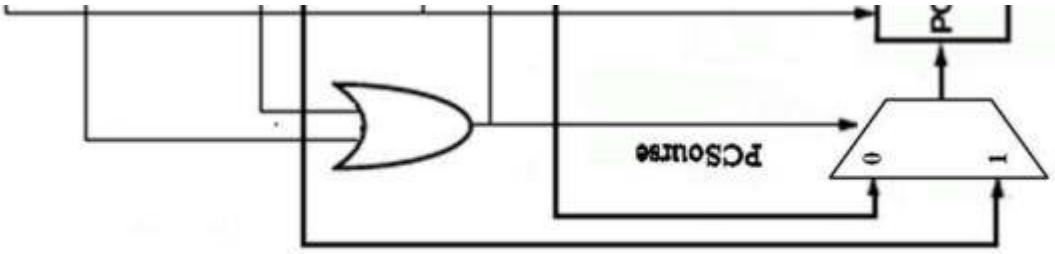



图 30.3 流水线 RISC-V 微处理器的原理框图

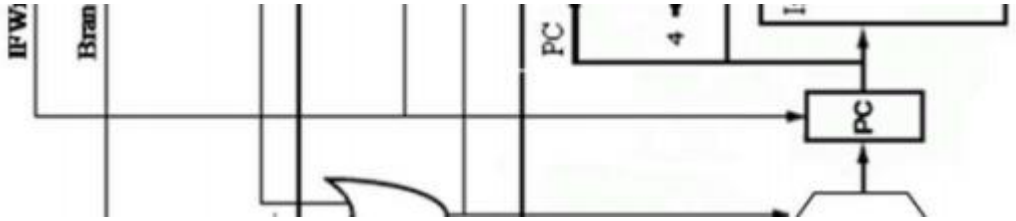
2．Verilog 代码：

（1）指令指针选择器(MUX)：



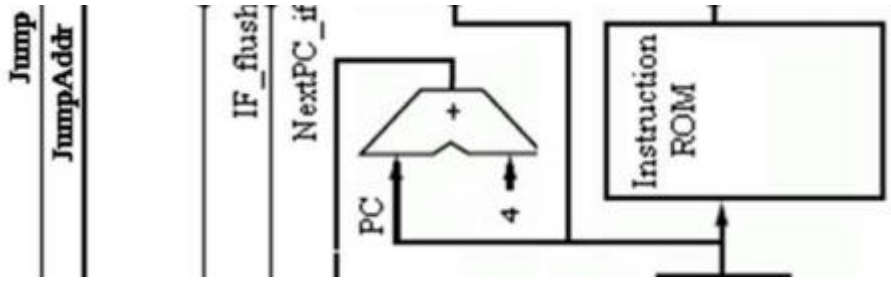

```
//指令指针寄存
wire PCsource;
assign PCsource=Jump || Branch;
assign IF_flush=PCsource;
wire [31:0] NextPC_if;
mux_2to1 #(.n(32)) mux_PC_if(.out(NextPC_if), .in0(PC + 32'h00000004), .in1(JumpAddr), .addr(PCsource));
```

（2）指令指针寄存器：




```
//处理PC，复位信号，写入信号
reg [31:0] PC;
always @(posedge clk) begin
    if (reset) begin
        PC <= 32'b0;
    end else if (IFWrite) begin
        PC <= NextPC_if;
    end else begin
        PC <= PC; //保持
    end
end
```

（3）指令存储器子模块(Instruction ROM)：




```
//读取指令存储器
InstructionROM InstructionROM_if(
    .addr(PC[7:2]),

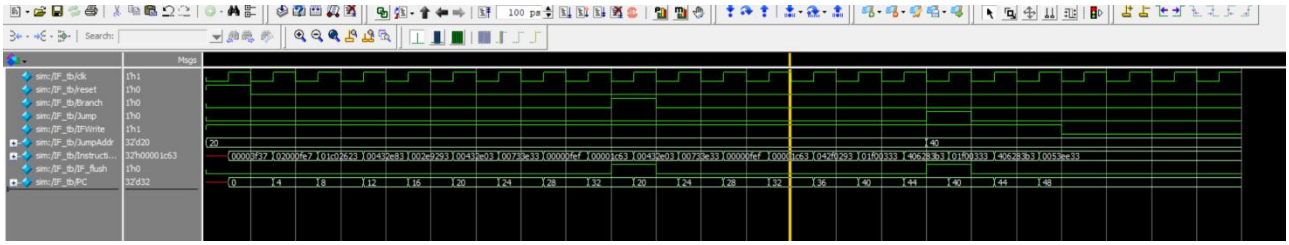
    .dout(Instruction_if)
);
```

```
    always @(*)
        case (addr)
            6'd0:   dout=32'h0000_3f37 ;//           lui X30,0x3000
            6'd1:   dout=32'h0200_0fE7 ;//           jalr X31 later(X0)
            6'd2:   dout=32'h01c0_2623 ;// earlier: sw  X28, 0C(X0)
            6'd3:   dout=32'h0043_2e83 ;//           lw  X29, 4(X6)
            6'd4:   dout=32'h002e_9293 ;//           sll  X5, X29, 2
            6'd5:   dout=32'h0043_2e03 ;//           lw   X28, 4(X6)
            6'd6:   dout=32'h0073_3e33 ;//           sltu X28,X6,X7
            6'd7:   dout=32'h0000_0fef ;//  done:   jal X31,done
            6'd8:   dout=32'h0000_1c63 ;//  later:  bne X0, X0, end         // 分支条件不成立
            6'd9:   dout=32'h042f_0293;//           addi X5, X30, 42
            6'd10:  dout=32'h01f0_0333 ;//          add  X6, X0, X31
            6'd11:  dout=32'h4062_83b3 ;//           sub X7, X5, X6
            6'd12:  dout=32'h0053_ee33 ;//          or   X28, X7, X5
            6'd13:  dout=32'hfc00_0ae3 ;//          beq X0, X0, earlier
            6'd14: dout=32'h00000000 ;// end:   nop
            default:dout=32'h00000000 ;//nop
        endcase
endmodule
```

PC 使用的是字节地址，这意味着 PC 的值表示的是内存中的第几个“字节”。一条标准的 RISC-V 指令长度是 32 位（即 4 个字节），为了读取一整条指令，PC 的值通常是 4 的倍数，InstructionROM 中的 case(addr) 使用索引 0, 1, 2... 来访问第 1 条、第 2 条、第 3 条指令。将 PC 的“字节地址”转换为 ROM 需要的“字索引”，我们需要将 PC 除以 4。InstructionROM 中 addr 只需要[5:0]6 位，因此输入的是 PC[7:2]

3．仿真波形：

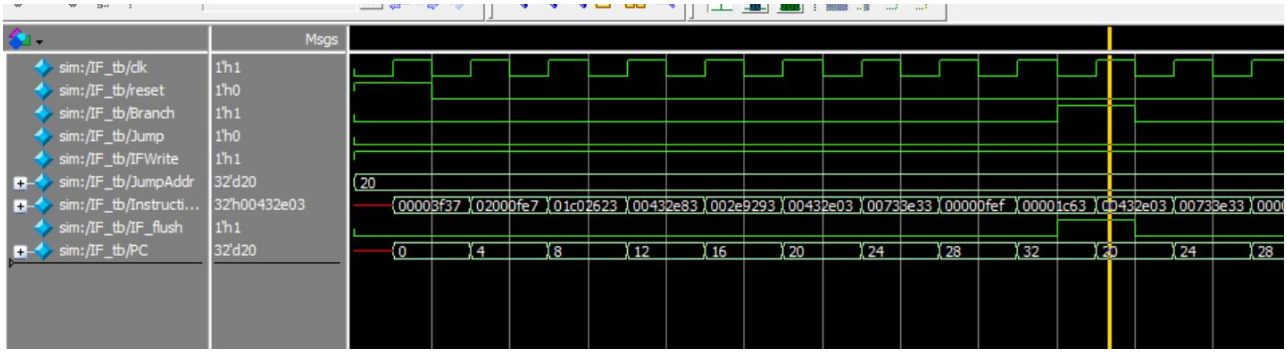
（1）波形图片：

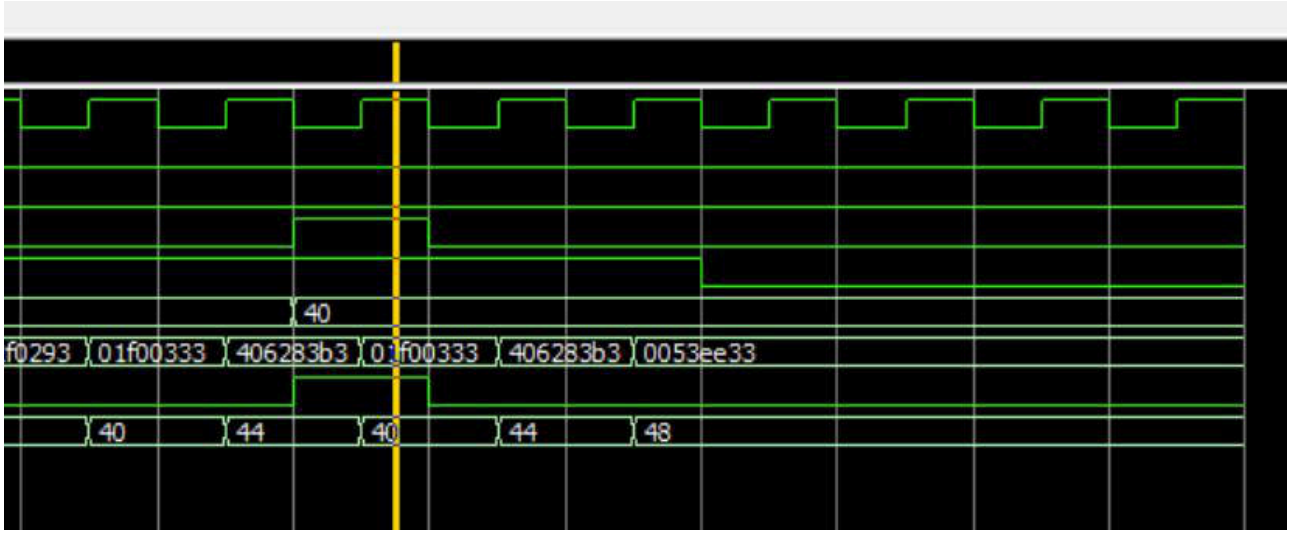


（2）结果分析：

①PC 整体递增，并且取到的指令也是正确的，符合 InstructionROM

②当出现 Branch 信号时，会读取 JumpAddr 的信息，然后在下一个时钟上升沿将 PC 地址改写为 JumpAddr 的地址，并且使得 IF_flush 为高电平，冲刷掉错误的流水线






从结果可以看出，Branch=1 时，IF_flush=1，并且下个时钟沿上升沿取到 JumpAddr 的值，然后赋值给 PC，符合 IF 的功能要求

## （六）RISC-V：流水线寄存器的设计

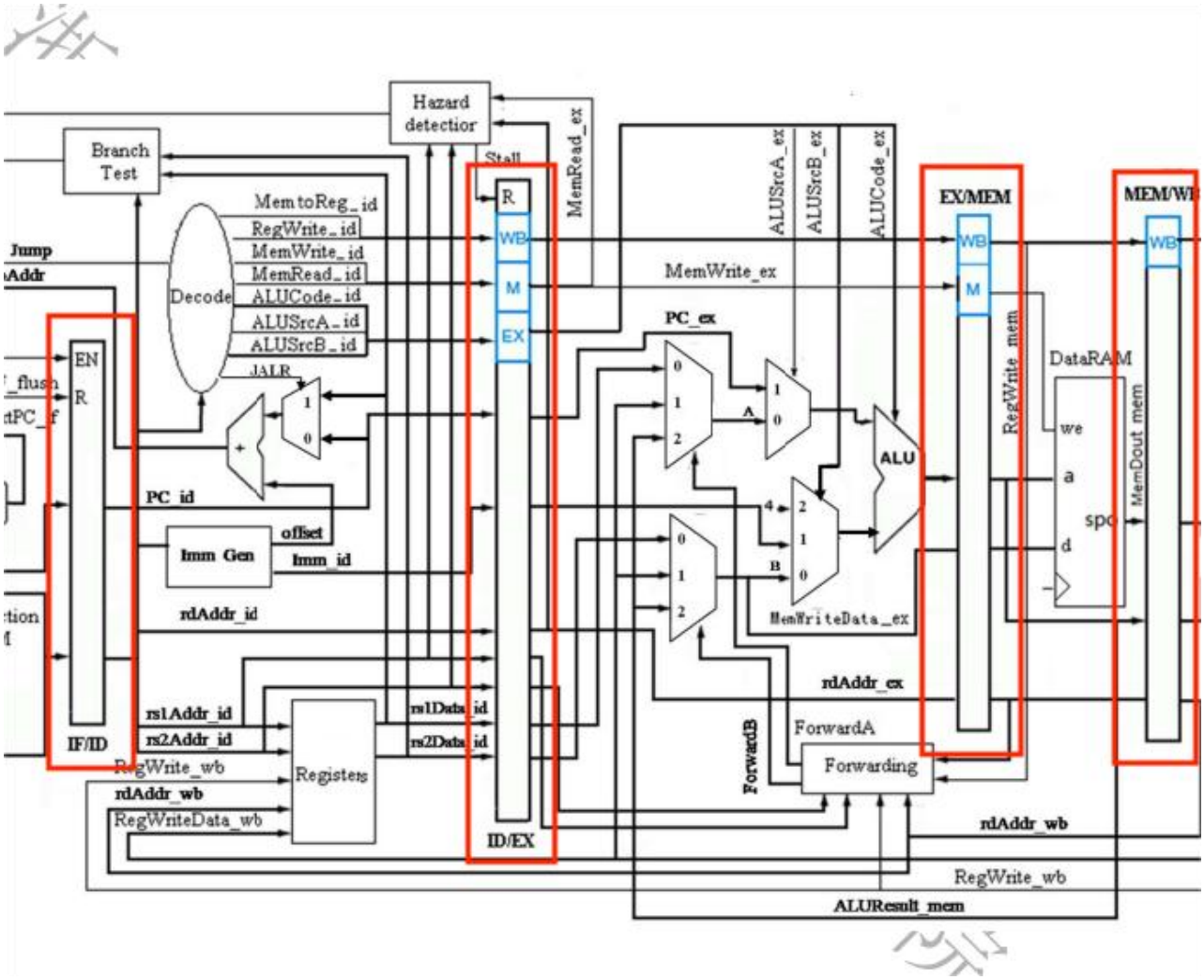
1．原理：

流水线寄存器负责将流水线的各部分分开，共有 IF/ID、ID/EX、EX/MEM、MEM/WB 四组，对四组流水线寄存器要求不完全相同，因此设计也有不同考虑。

EX/MEM、MEM/WB 两组流水线寄存器只是普通的 D 型寄存器。

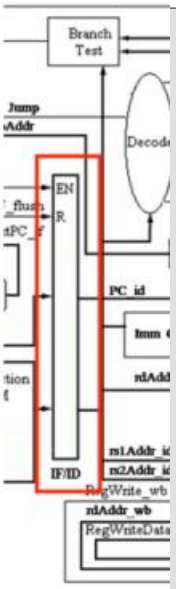
当流水线发生数据冒险时，需要清空 ID/EX 流水线寄存器而插入一个气泡，因此 ID/EX 流水线寄存器是一个带同步清零功能的 D 型寄存器。

当流水线发生数据冒险时，需要阻塞 ID 流水线寄存器;若跳转指令或分支成立，则还需要清空 ID/EX 流水线寄存器。因此，IF/ID 流水线寄存器除同步清零功能外，还需要具有保持功能(即具有使能 EN 信号输入)。




## 2. Verilog 代码：

### （1）IF/ID 寄存器：



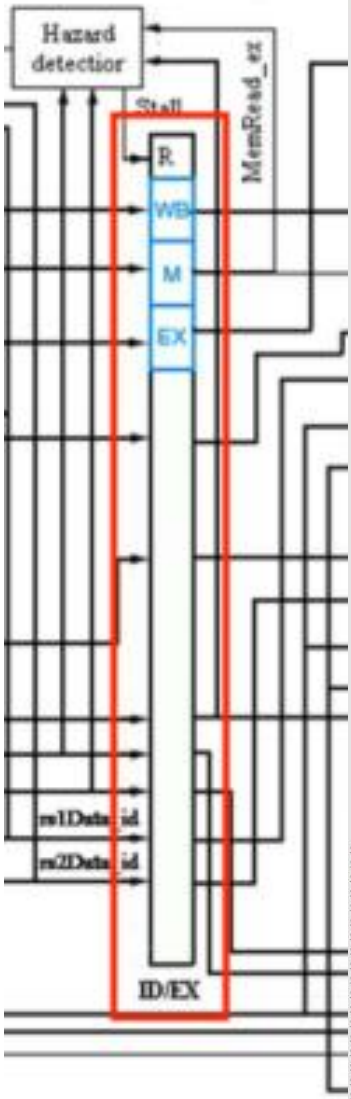
```
//IF/ID Reg
module IF_ID(clk,EN,R,PC_if,Instruction_if,PC_id,Instruction_id);
    //input
    input clk;
    input EN; //使能端
    input R;  //阻塞与清空
    input [31:0] PC_if;
    input [31:0] Instruction_if;

    //output
    output reg [31:0] PC_id;
    output reg [31:0] Instruction_id;

    always @(posedge clk) begin
        if (R) begin
            Instruction_id <= 32'b0;
            PC_id <= 32'b0;
        end
        else if (EN !== 1'b0) begin
            Instruction_id <= Instruction_if;
            PC_id <= PC_if;
        end
        else begin
            Instruction_id <= Instruction_id; // 保持
            PC_id <= PC_id;
        end
    end

endmodule
```

（2）ID/EX 寄存器：




```
//ID/EX Reg
module ID_EX(clk,R,MemtoReg_id,RegWrite_id,MemWrite_id,MemRead_id,ALUCode_id,ALUSrcA_id,ALUSrcB_id,PC_id,Imm_id,
             rdAddr_id,rs1Addr_id,rs2Addr_id,rs1Data_id,rs2Data_id,MemtoReg_ex,RegWrite_ex,MemWrite_ex,MemRead_ex,
             ALUCode_ex,ALUSrcA_ex,ALUSrcB_ex,PC_ex,Imm_ex,rdAddr_ex,rs1Addr_ex,rs2Addr_ex,rs1Data_ex,rs2Data_ex);

    //input
    input clk;
    input R;//冒险检测

    input MemtoReg_id;//Decode信号输出
    input RegWrite_id;
    input MemWrite_id;
    input MemRead_id;
    input [3:0] ALUCode_id;
    input ALUSrcA_id;
    input [1:0] ALUSrcB_id;

    input [31:0] PC_id;//PC

    input [31:0] Imm_id;//Imm

    input [4:0] rdAddr_id;//寄存器地址
    input [4:0] rs1Addr_id;
    input [4:0] rs2Addr_id;

    input [31:0] rs1Data_id; //数据
    input [31:0] rs2Data_id;

    //output
    output reg MemtoReg_ex;//WB
    output reg RegWrite_ex;
    output reg MemWrite_ex;//MEM
    output reg MemRead_ex;
    output reg [3:0] ALUCode_ex;//EX
    output reg ALUSrcA_ex;
    output reg [1:0] ALUSrcB_ex;

    output reg [31:0] PC_ex;//PC

    output reg [31:0] Imm_ex;//Imm
    output reg [4:0] rdAddr_ex;//寄存器地址
    output reg [4:0] rs1Addr_ex;
    output reg [4:0] rs2Addr_ex;

    output reg [31:0] rs1Data_ex; //数据
    output reg [31:0] rs2Data_ex;

    always @(posedge clk) begin
        if (R) begin//复位
            MemtoReg_ex <= 1'b0;
            RegWrite_ex <= 1'b0;
            MemWrite_ex <= 1'b0;
            MemRead_ex <= 1'b0;

            ALUSrcA_ex <= 1'b0;
            ALUSrcB_ex <= 2'b0;
            ALUCode_ex <= 4'b0;

            PC_ex <= 32'b0;
            Imm_ex <= 32'b0;

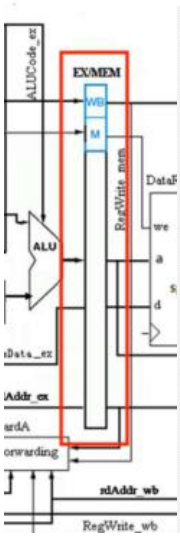
            rdAddr_ex <= 5'b0;
            rs1Addr_ex <= 5'b0;
            rs2Addr_ex <= 5'b0;
            rs1Data_ex <= 32'b0;
            rs2Data_ex <= 32'b0;
        end
        else begin
            MemtoReg_ex <= MemtoReg_id;
            RegWrite_ex <= RegWrite_id;
            MemWrite_ex <= MemWrite_id;
            MemRead_ex <= MemRead_id;

            ALUSrcA_ex <= ALUSrcA_id;
            ALUSrcB_ex <= ALUSrcB_id;
            ALUCode_ex <= ALUCode_id;

            PC_ex <= PC_id;
            Imm_ex <= Imm_id;

            rdAddr_ex <= rdAddr_id;
            rs1Addr_ex <= rs1Addr_id;
            rs2Addr_ex <= rs2Addr_id;
            rs1Data_ex <= rs1Data_id;
            rs2Data_ex <= rs2Data_id;
        end
    end
endmodule
```

（3）EX/MEM 寄存器：



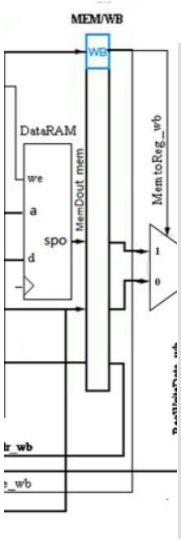

```
//EX/MEM Reg
module EX_MEM(clk,MemtoReg_ex,RegWrite_ex,MemWrite_ex,ALUResult_ex,MemWriteData_ex,rdAddr_ex,MemtoReg_mem,
              RegWrite_mem,MemWrite_mem,ALUResult_mem,MemWriteData_mem,rdAddr_mem);
    //input
    input clk;
    input MemtoReg_ex;//WB
    input RegWrite_ex;
    input MemWrite_ex;//MEM
    input [31:0] ALUResult_ex;//ALU
    input [31:0] MemWriteData_ex;
    input [4:0] rdAddr_ex;

    //output
    output reg MemtoReg_mem;//WB
    output reg RegWrite_mem;
    output reg MemWrite_mem;//MEM
    output reg [31:0] ALUResult_mem;
    output reg [31:0] MemWriteData_mem;
    output reg [4:0] rdAddr_mem;

    always @(posedge clk) begin
        MemtoReg_mem <= MemtoReg_ex;//WB
        RegWrite_mem <= RegWrite_ex;
        MemWrite_mem <= MemWrite_ex;//MEM
        ALUResult_mem <= ALUResult_ex;
        MemWriteData_mem <= MemWriteData_ex;
        rdAddr_mem <= rdAddr_ex;
    end

endmodule
```

（4）MEM/WB 寄存器：




```
//MEM/WB Reg
module MEM_WB(clk,MemtoReg_mem,RegWrite_mem,ALUResult_mem,MemDout_mem,rdAddr_mem,MemtoReg_wb,RegWrite_wb,
              ALUResult_wb,MemDout_wb,rdAddr_wb);
    input clk;
    input MemtoReg_mem;//WB
    input RegWrite_mem;
    input [31:0] ALUResult_mem;
    input [31:0] MemDout_mem;//MEM
    input [4:0] rdAddr_mem;

    //output
    output reg MemtoReg_wb;//WB
    output reg RegWrite_wb;
    output reg [31:0] ALUResult_wb;
    output reg [31:0] MemDout_wb;
    output reg [4:0] rdAddr_wb;

    always @(posedge clk) begin
        MemtoReg_wb <= MemtoReg_mem;
        RegWrite_wb <= RegWrite_mem;
        ALUResult_wb <= ALUResult_mem;
        MemDout_wb <= MemDout_mem;
        rdAddr_wb <= rdAddr_mem;
    end

endmodule
```

## （七）RISC-V：顶层文件的设计

1．原理：

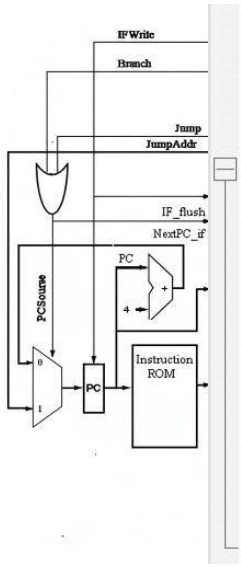
按照图 30.3 所示的原理框图连接各模块即可。为了测试方便，可将关键变量输出，关键变量有:指令指针 PC、指令码 Instruction íd、流水线插入气泡标志 Stal、分支标志 JumpFlag 即{Jump, Branch}、ALU 输入输出(ALU A、ALU B、ALUResult ex)和数据存储器的输出 MemDout mem。

顶层文件的端口列表描述如下：

```
module Risc5CPU(clk, reset, JumpFlag, Instruction_id, ALU_A,
                ALU_B, ALUResult_ex, PC, MemDout_mem, Stall);
    input clk;
    input reset;
    output[1:0] JumpFlag;
    output [31:0] Instruction_id;
    output [31:0] ALU_A;
    output [31:0] ALU_B;
    output [31:0] ALUResult_ex;
    output [31:0] PC;
    output [31:0] MemDout_mem;
    output Stall;
```

2. Verilog 代码：

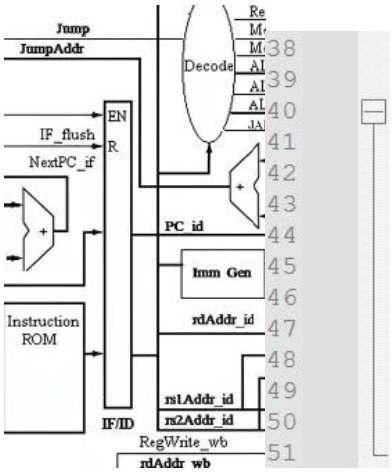
（1）IF 阶段：




```
//IF阶段
wire Branch,Jump,IFWrite,IF_flush;
assign JumpFlag[0]=Branch;
assign JumpFlag[1]=Jump;
wire [31:0] JumpAddr,Instruction_if;
IF1 IF1_inst(
    //input
    .clk(clk),
    .reset(reset),
    .Branch(Branch),
    .Jump(Jump),
    .IFWrite(IFWrite),
    .JumpAddr(JumpAddr),

    //output
    .Instruction_if(Instruction_if),
    .PC(PC),
    .IF_flush(IF_flush)
);
```

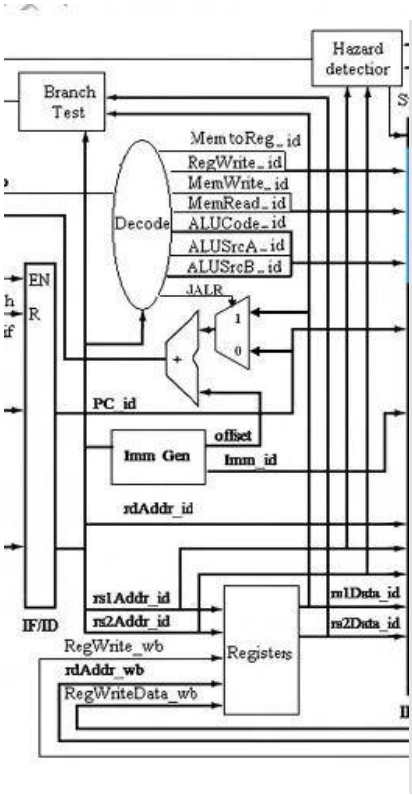
（2）IF/ID 寄存器：




```
//IF/ID
wire [31:0] PC_id;
IF_ID IF_ID_inst(
    //input
    .clk(clk),
    .EN(IFWrite),
    .R(IF_flush),
    .PC_if(PC),
    .Instruction_if(Instruction_if),

    //output
    .PC_id(PC_id),
    .Instruction_id(Instruction_id)
);
```

（3）ID 阶段：

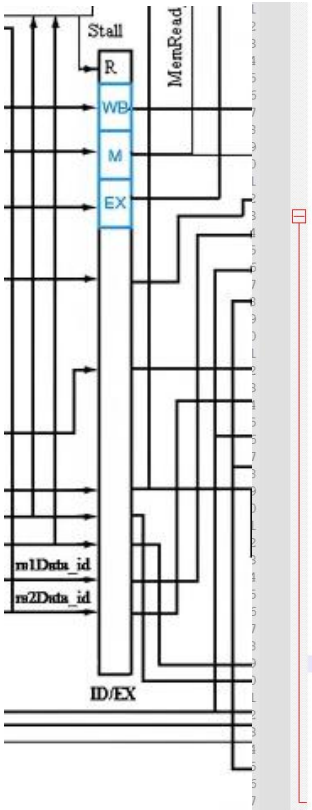



```
//ID阶段
wire RegWrite_wb;
wire [4:0] rdAddr_wb;
wire [31:0] RegWriteData_wb;
wire MemRead_ex;
wire [4:0] rdAddr_ex;

wire MemtoReg_id;
wire RegWrite_id;
wire MemWrite_id;
wire MemRead_id;
wire [3:0] ALUCode_id;
wire ALUSrcA_id;
wire [1:0]ALUSrcB_id;
wire [31:0] Imm_id;
wire [31:0] rs1Data_id,rs2Data_id;
wire[4:0] rs1Addr_id,rs2Addr_id,rdAddr_id;
ID ID_inst(
    //input
    .clk(clk),
    .Instruction_id(Instruction_id),
    .PC_id(PC_id),
    .RegWrite_wb(RegWrite_wb),
    .rdAddr_wb(rdAddr_wb),
    .RegWriteData_wb(RegWriteData_wb),
    .MemRead_ex(MemRead_ex),
    .rdAddr_ex(rdAddr_ex),

    //output
    .MemtoReg_id(MemtoReg_id),
    .RegWrite_id(RegWrite_id),
    .MemWrite_id(MemWrite_id),
    .MemRead_id(MemRead_id),
    .ALUCode_id(ALUCode_id),
    .ALUSrcA_id(ALUSrcA_id),
    .ALUSrcB_id(ALUSrcB_id),
    .Stall(Stall),
    .Branch(Branch),
    .Jump(Jump),
    .IFWrite(IFWrite),
    .JumpAddr(JumpAddr),
    .Imm_id(Imm_id),
    .rs1Data_id(rs1Data_id),
    .rs2Data_id(rs2Data_id),
    .rs1Addr_id(rs1Addr_id),
    .rs2Addr_id(rs2Addr_id),
    .rdAddr_id(rdAddr_id)
);
```

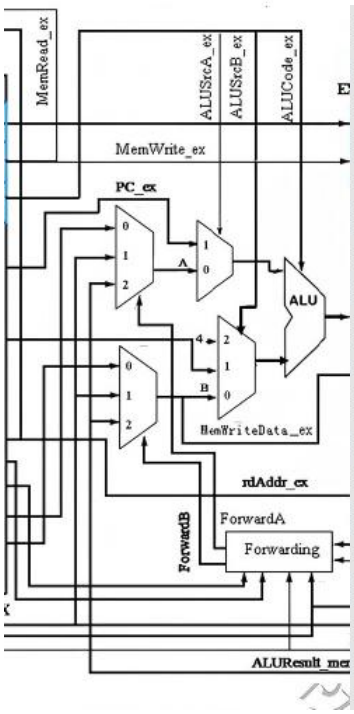
（4）ID/EX 寄存器：




```
wire MemtoReg_ex;
wire RegWrite_ex;
wire MemWrite_ex;
wire [3:0] ALUCode_ex;
wire ALUSrcA_ex;
wire [1:0] ALUSrcB_ex;
wire [31:0] PC_ex;
wire [31:0] Imm_ex;
wire [4:0] rs1Addr_ex;
wire [4:0] rs2Addr_ex;
wire [31:0] rs1Data_ex;
wire [31:0] rs2Data_ex;
ID_EX ID_EX_inst(
    //input
    .clk(clk),
    .R(Stall),
    .MemtoReg_id(MemtoReg_id),
    .RegWrite_id(RegWrite_id),
    .MemWrite_id(MemWrite_id),
    .MemRead_id(MemRead_id),
    .ALUCode_id(ALUCode_id),
    .ALUSrcA_id(ALUSrcA_id),
    .ALUSrcB_id(ALUSrcB_id),
    .PC_id(PC_id),
    .Imm_id(Imm_id),
    .rdAddr_id(rdAddr_id),
    .rs1Addr_id(rs1Addr_id),
    .rs2Addr_id(rs2Addr_id),
    .rs1Data_id(rs1Data_id),
    .rs2Data_id(rs2Data_id),

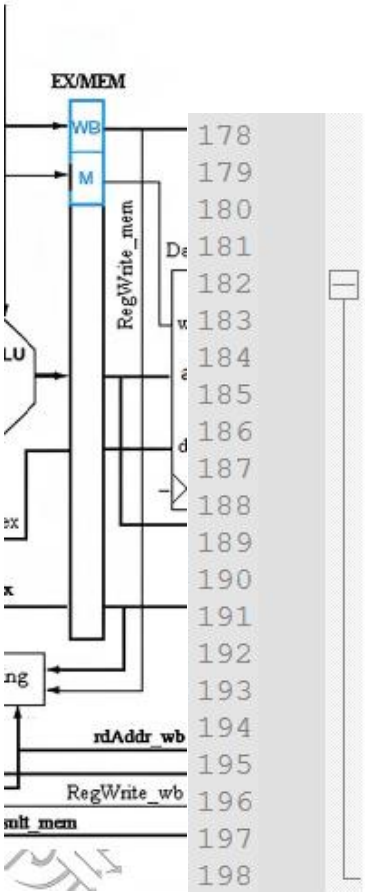
    //output
    .MemtoReg_ex(MemtoReg_ex),
    .RegWrite_ex(RegWrite_ex),
    .MemWrite_ex(MemWrite_ex),
    .MemRead_ex(MemRead_ex),
    .ALUCode_ex(ALUCode_ex),
    .ALUSrcA_ex(ALUSrcA_ex),
    .ALUSrcB_ex(ALUSrcB_ex),
    .PC_ex(PC_ex),
    .Imm_ex(Imm_ex),
    .rdAddr_ex(rdAddr_ex),
    .rs1Addr_ex(rs1Addr_ex),
    .rs2Addr_ex(rs2Addr_ex),
    .rs1Data_ex(rs1Data_ex),
    .rs2Data_ex(rs2Data_ex)
);
```

（5）EX 阶段：



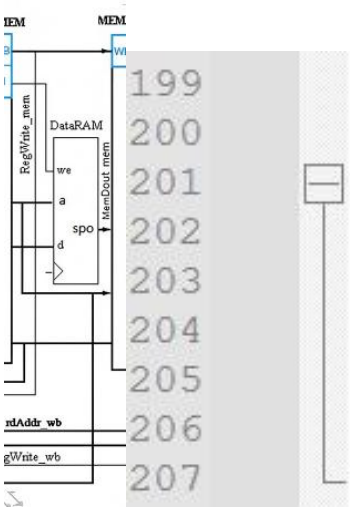

```
148     //EX阶段
149     wire [31:0] MemWriteData_ex;
150     wire [31:0] ALUResult_mem;
151     wire [4:0] rdAddr_mem;
152     wire RegWrite_mem;
153
154     EX EX_inst(
155         //input
156         .ALUCode_ex(ALUCode_ex),
157         .ALUSrcA_ex(ALUSrcA_ex),
158         .ALUSrcB_ex(ALUSrcB_ex),
159         .Imm_ex(Imm_ex),
160         .rs1Addr_ex(rs1Addr_ex),
161         .rs2Addr_ex(rs2Addr_ex),
162         .rs1Data_ex(rs1Data_ex),
163         .rs2Data_ex(rs2Data_ex),
164         .PC_ex(PC_ex),
165         .RegWriteData_wb(RegWriteData_wb),
166         .ALUResult_mem(ALUResult_mem),
167         .rdAddr_mem(rdAddr_mem),
168         .rdAddr_wb(rdAddr_wb),
169         .RegWrite_mem(RegWrite_mem),
170         .RegWrite_wb(RegWrite_wb),
171
172         //output
173         .ALUResult_ex(ALUResult_ex),
174         .MemWriteData_ex(MemWriteData_ex),
175         .ALU_A(ALU_A),
176         .ALU_B(ALU_B)
177     );
```

（6）EX/MEM 寄存器：




```
178     //EX/Mem
179     wire MemtoReg_mem;
180     wire MemWrite_mem;
181     wire [31:0] MemWriteData_mem;
182     EX_MEM EX_MEM_inst(
183         //input
184         .clk(clk),
185         .MemtoReg_ex(MemtoReg_ex),
186         .RegWrite_ex(RegWrite_ex),
187         .MemWrite_ex(MemWrite_ex),
188         .ALUResult_ex(ALUResult_ex),
189         .MemWriteData_ex(MemWriteData_ex),
190         .rdAddr_ex(rdAddr_ex),
191         //output
192         .MemtoReg_mem(MemtoReg_mem),
193         .RegWrite_mem(RegWrite_mem),
194         .MemWrite_mem(MemWrite_mem),
195         .ALUResult_mem(ALUResult_mem),
196         .MemWriteData_mem(MemWriteData_mem),
197         .rdAddr_mem(rdAddr_mem)
198     );
```

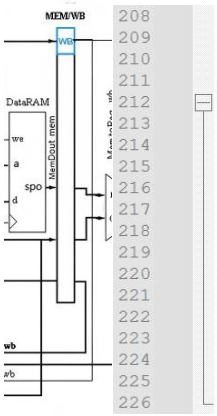
（7）MEM 阶段：




```
199
200
201
202
203
204
205
206
207
```

```
//MEM
wire [31:0] MemDout_mem;
DataRAM DataRAM_inst(
    .a(ALUResult_mem[7:2]),
    .d(MemWriteData_mem),
    .clk(clk),
    .we(MemWrite_mem),
    .spo(MemDout_mem)
);
```

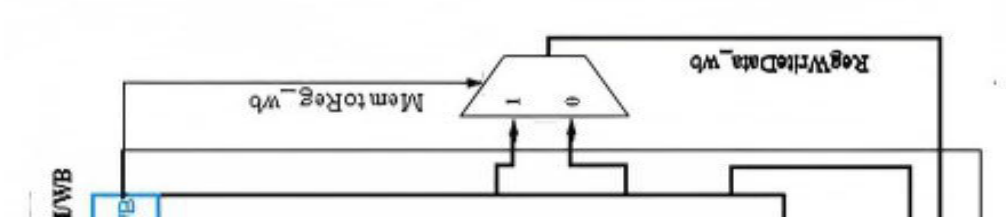
（8）MEM/WB 寄存器：




```
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
```

```
//MEM/WB
wire MemtoReg_wb;
wire [31:0] ALUResult_wb;
wire [31:0 ]MemDout_wb;
MEM_WB MEM_WB_inst(
    //input
    .clk(clk),
    .MemtoReg_mem(MemtoReg_mem),
    .RegWrite_mem(RegWrite_mem),
    .ALUResult_mem(ALUResult_mem),
    .MemDout_mem(MemDout_mem),
    .rdAddr_mem(rdAddr_mem),
    //output
    .MemtoReg_wb(MemtoReg_wb),
    .RegWrite_wb(RegWrite_wb),
    .ALUResult_wb(ALUResult_wb),
    .MemDout_wb(MemDout_wb),
    .rdAddr_wb(rdAddr_wb)
);
```

（9）WB 阶段：

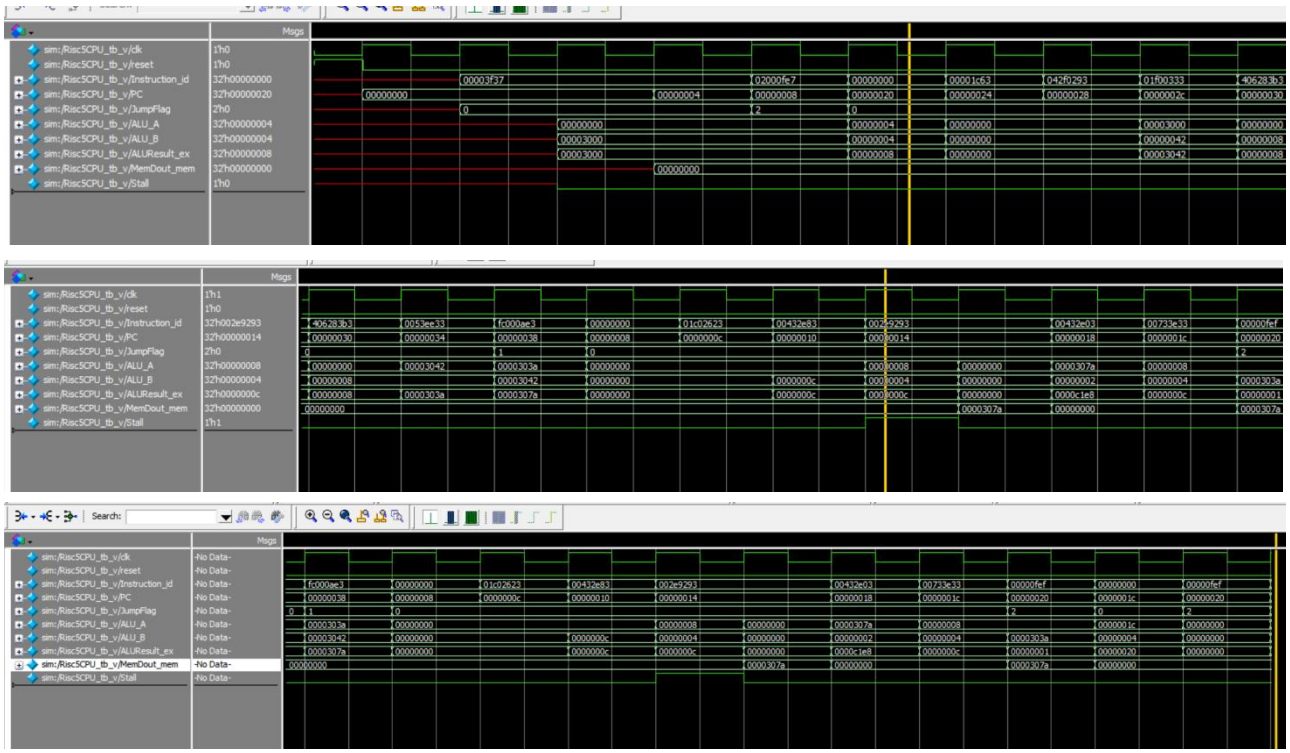



```
225
226
227
228
229
230
```

```
    .rdAddr_wb(rdAddr_wb)
);
//WB
mux_2to1 #(.n(32)) mux_WB(.out(RegWriteData_wb), .in0(ALUResult_wb), .in1(MemDout_wb), .addr(MemtoReg_wb));
endmodule
```

3．仿真波形与结果分析：

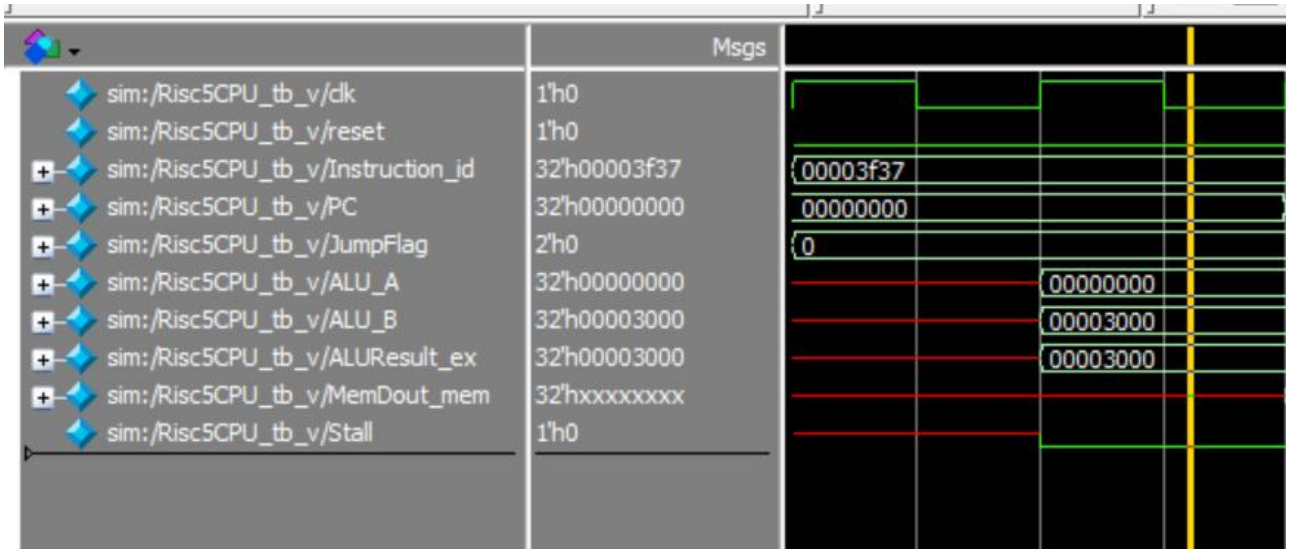
（1）波形结果：



（2）结果分析：

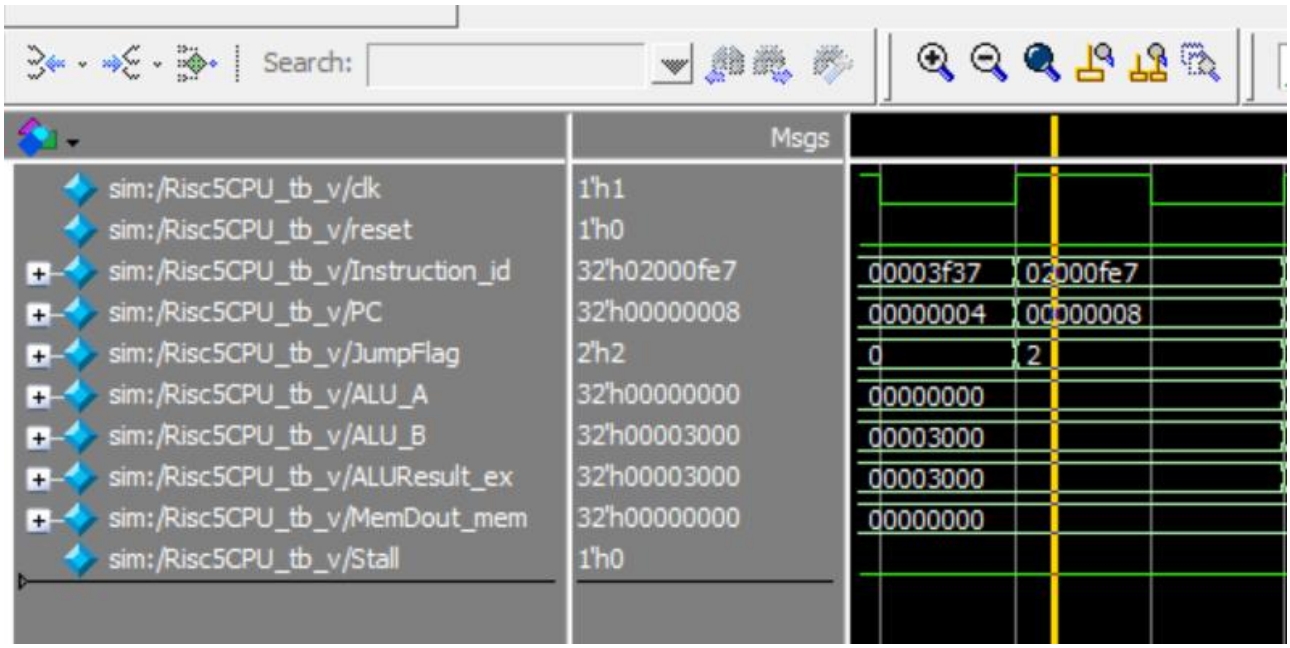
依次分命令分析，可以看出符合讲义给出的“测试程序的运行结果”图

①Lui x30,0x3000：



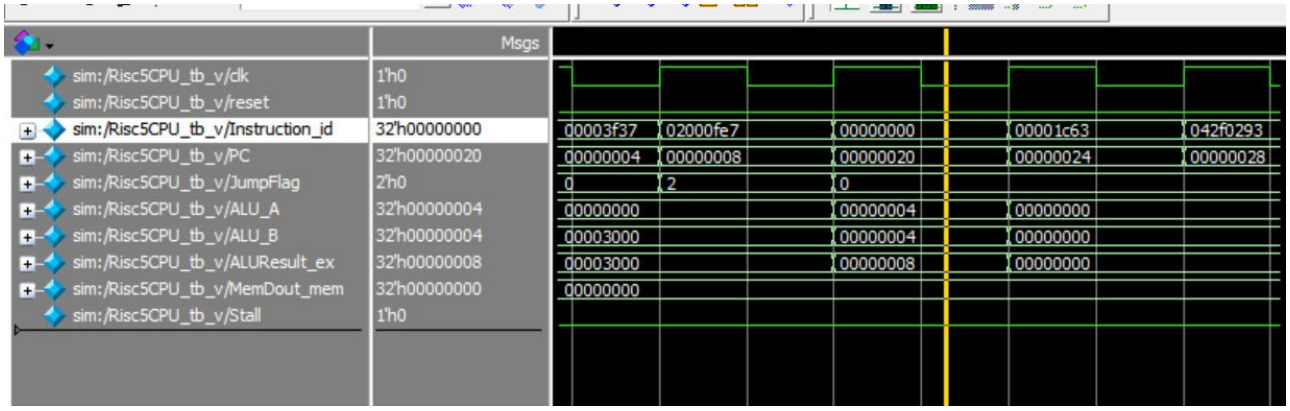

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 4 | 00003f37 | 0 | 0 | - | - | - | - |

②jalr x31,later(x0)：



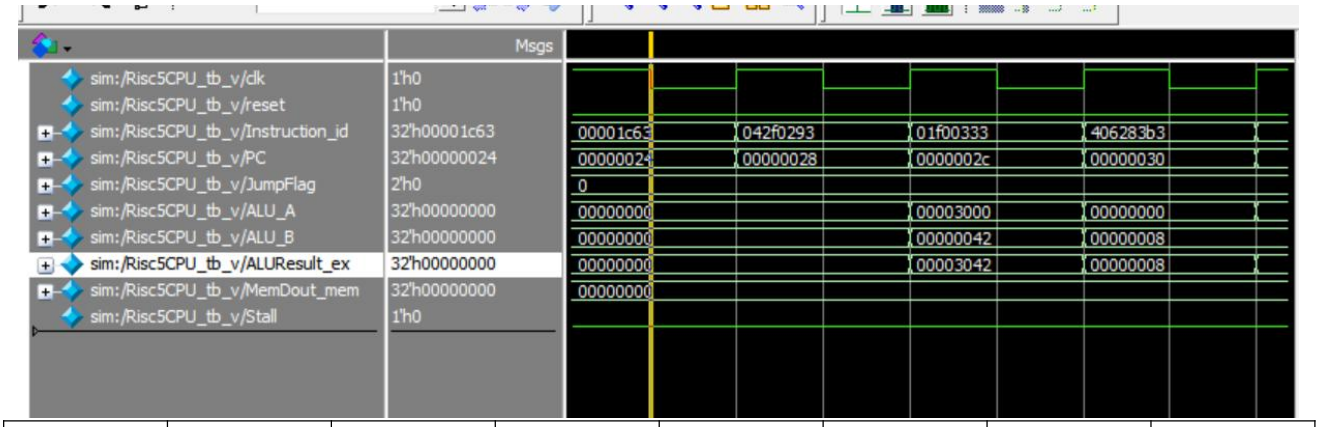

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 8 | 02000fe7 | 2 | 0 | - | 3000 | 3000 | - |

③nop：



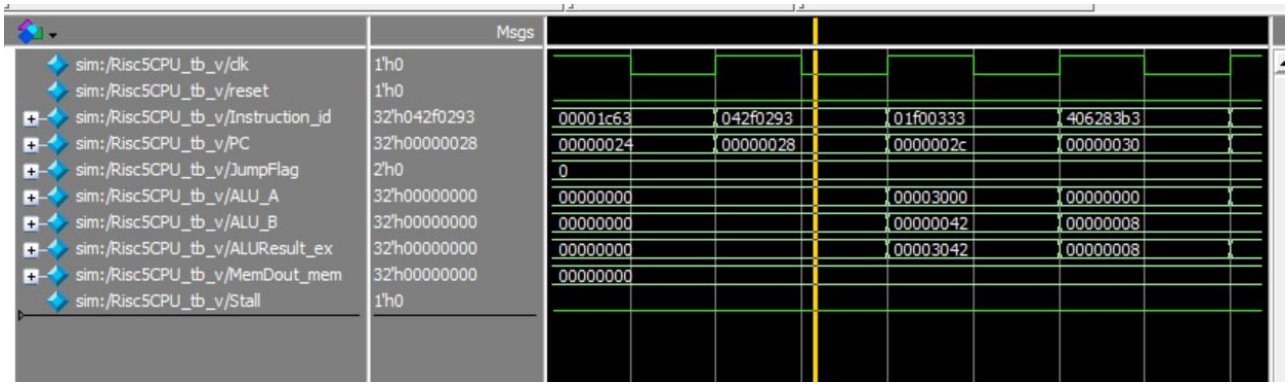

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 20 | 0 | 0 | 0 | 4 | 4 | 8 | - |

④Bne x0,x0,end：



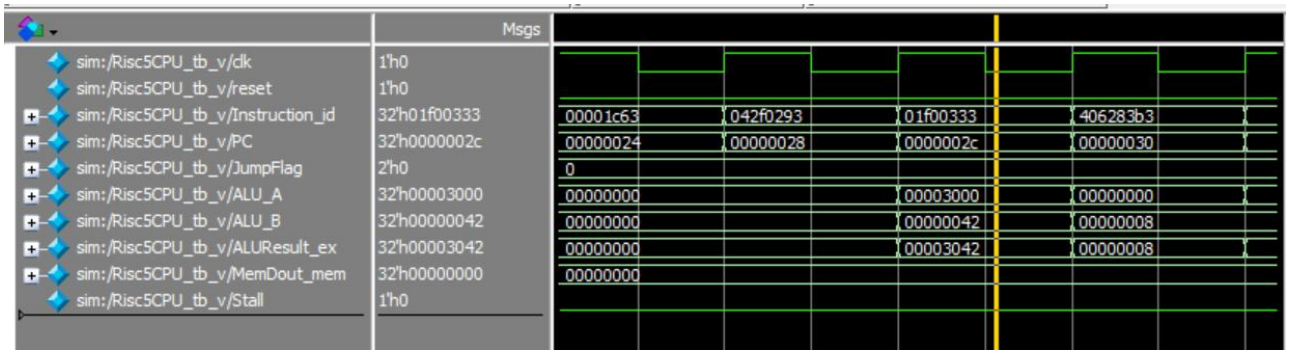

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 24 | 00001c63 | 0 | 0 | - | - | - | - |

⑤Addi x5,x30,0x42：



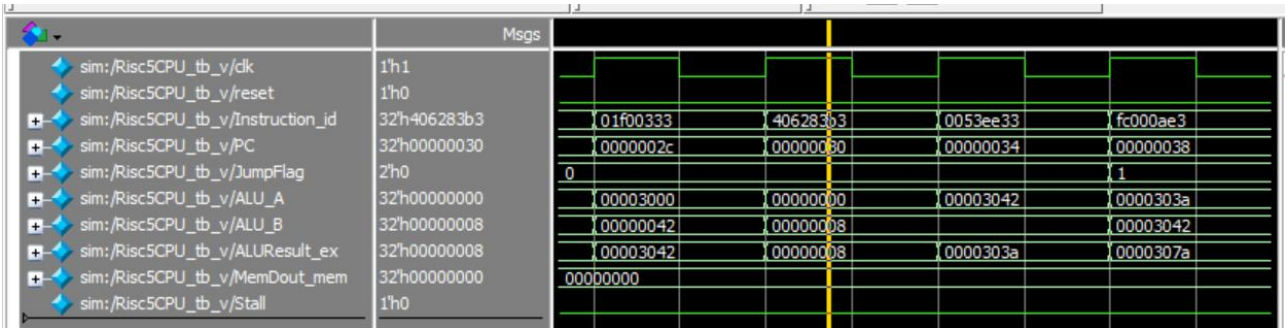

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 28 | 042f0293 | 0 | 0 | - | - | - | - |

⑥Add x6,x0,x31：



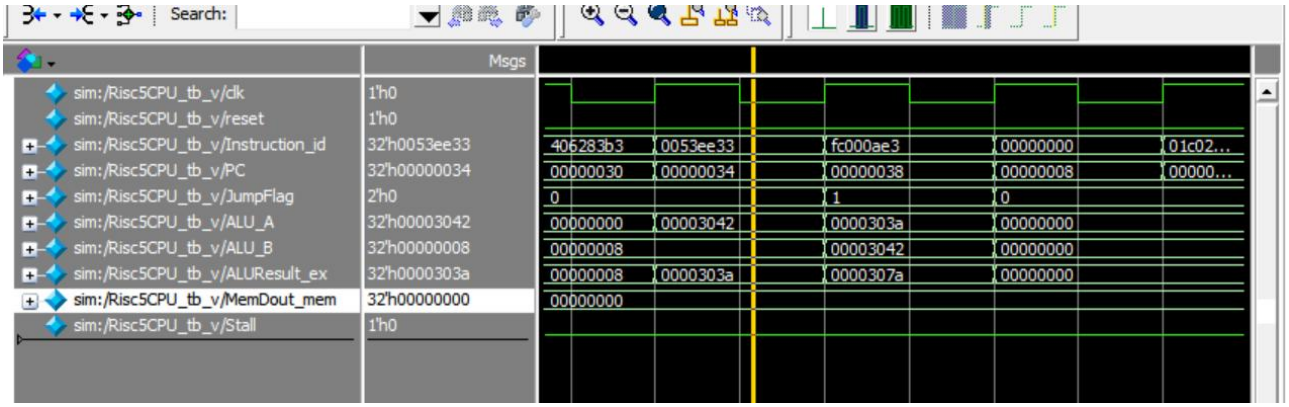

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 2c | 01f00333 | 0 | 0 | 3000 | 42 | 3042 | - |

⑦Sub x7,x5,x6：



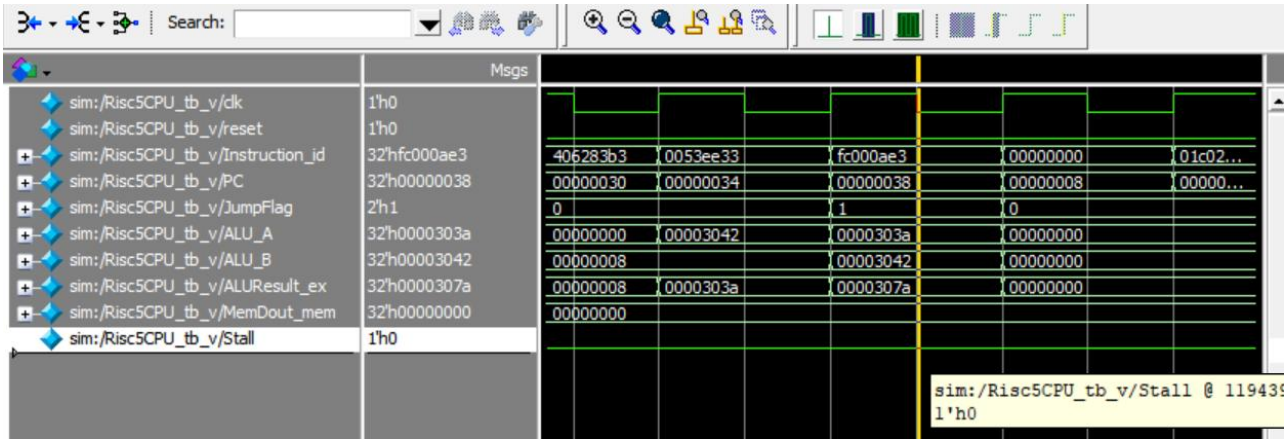

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 30 | 406283b3 | 0 | 0 | 0 | 8 | 8 | - |

⑧Or x28,x7,x5：



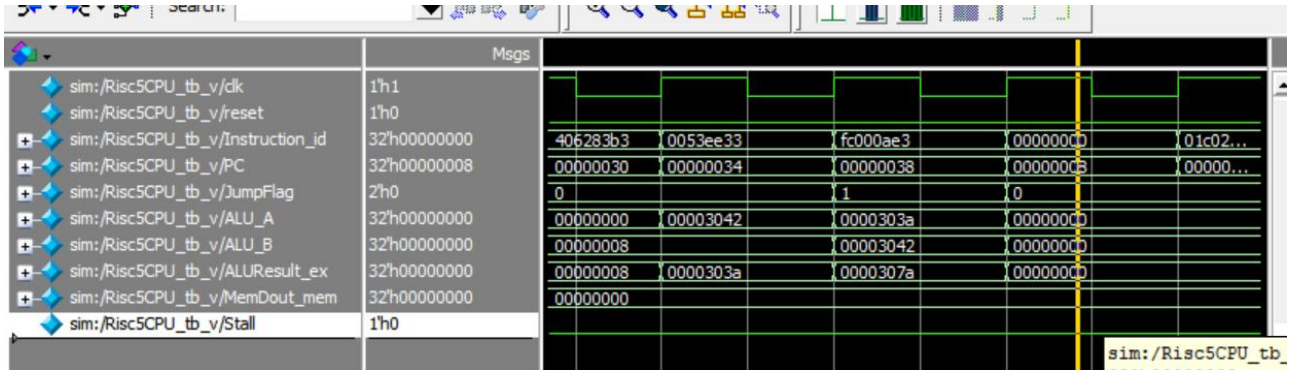

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 34 | 0053ee33 | 0 | 0 | 3042 | 8 | 303a | - |

⑨Beq x0,x0,earlier：



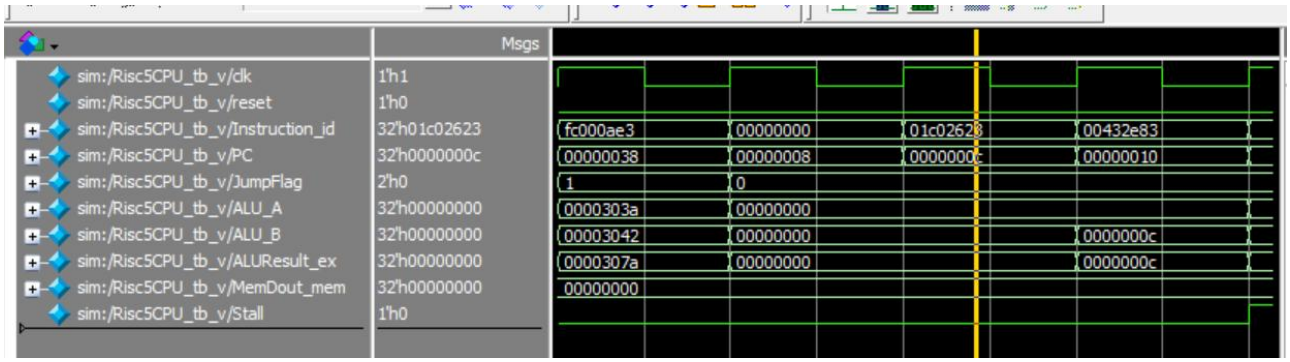

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 38 | Fc000ae3 | 1 | 0 | 303a | 3042 | 307a | - |

⑩Nop:



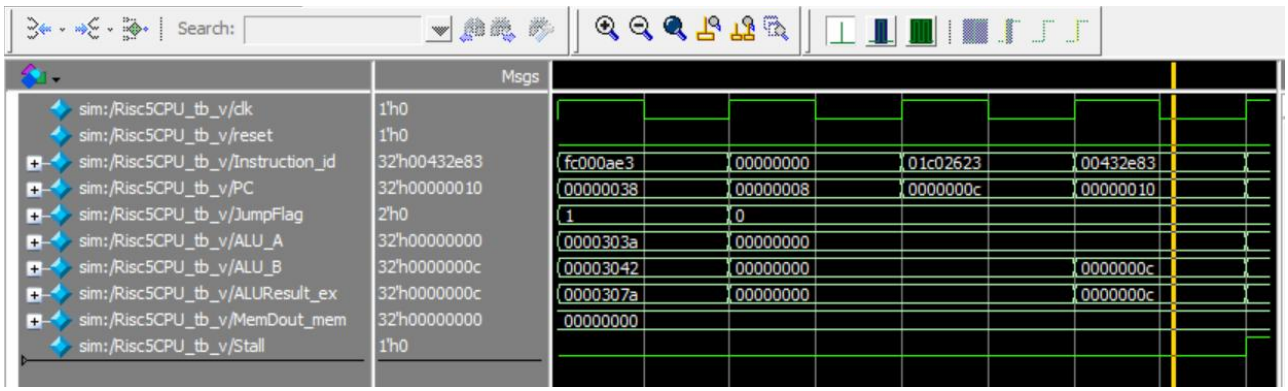

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 8 | 0 | 0 | 0 | - | - | - | - |

⑪Sw x28,0x0c(x0):



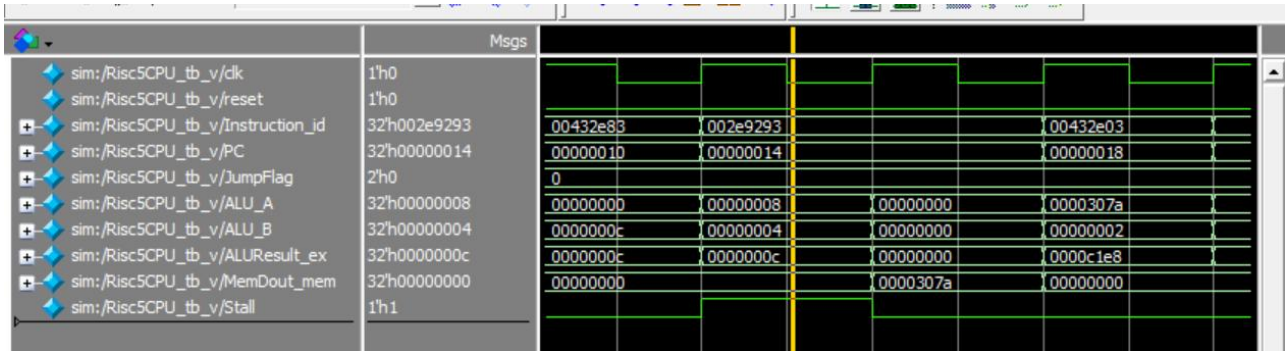

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| C | 001c2623 | 0 | 0 | - | - | - | - |

⑫Lw x29,4（x6）：



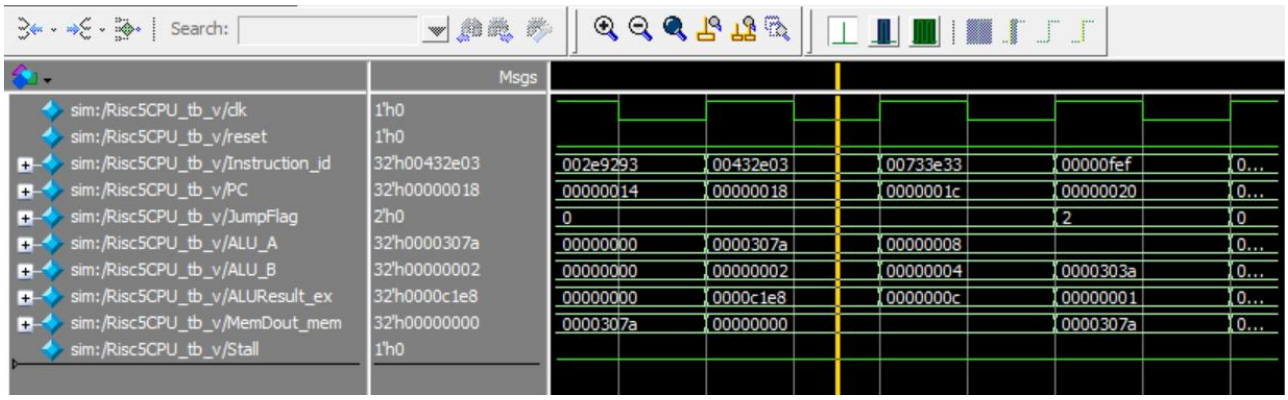

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 10 | 00432e83 | 0 | 0 | 0 | C | C | - |

⑬Slli x5，x29，2：



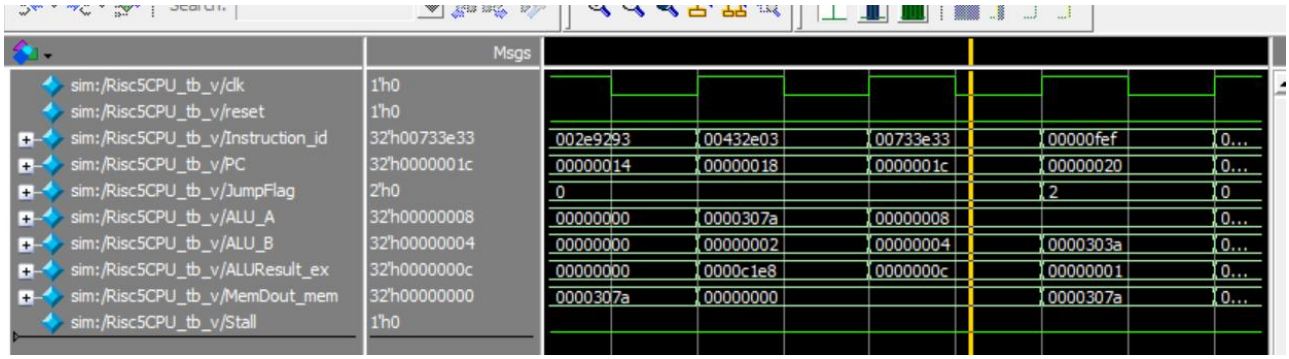

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 14 | 002e9293 | 0 | 1 | 8 | 4 | c | - |
| | | 0 | 0 | - | - | - | 307a |

⑭Lw x28,4（x6）：



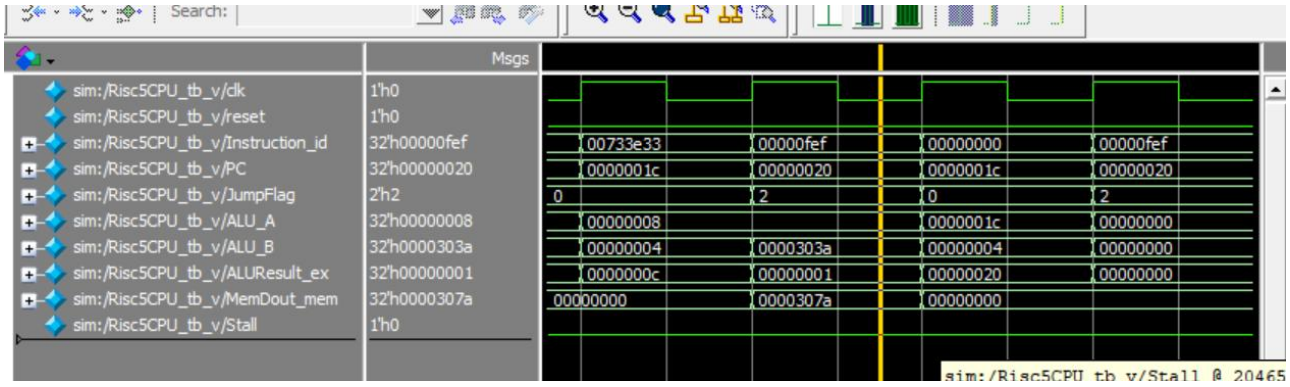

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 18 | 00432e03 | 0 | 0 | 307a | 2 | Cle8 | - |

⑮Sltu x28，x6，x7：



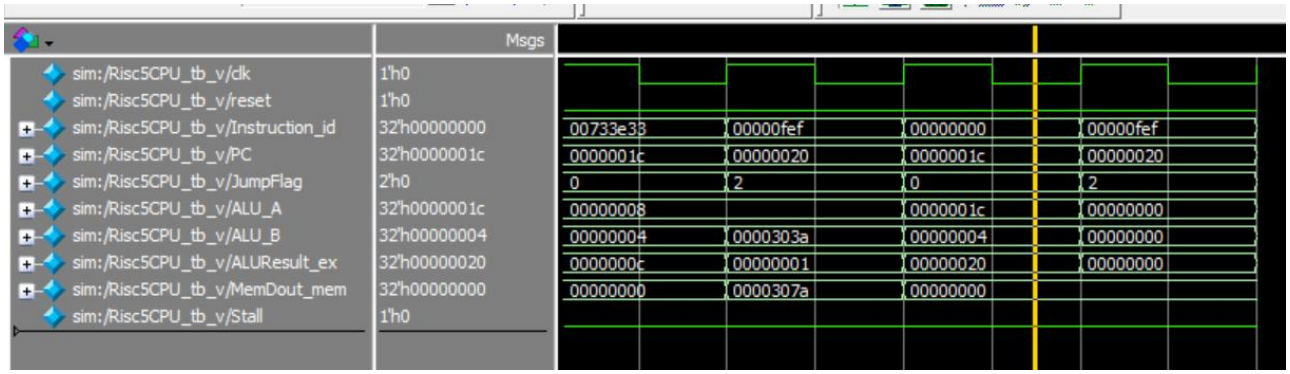

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 1c | 00733e33 | 0 | 0 | 8 | 4 | c | - |

⑯Jar x31 done：



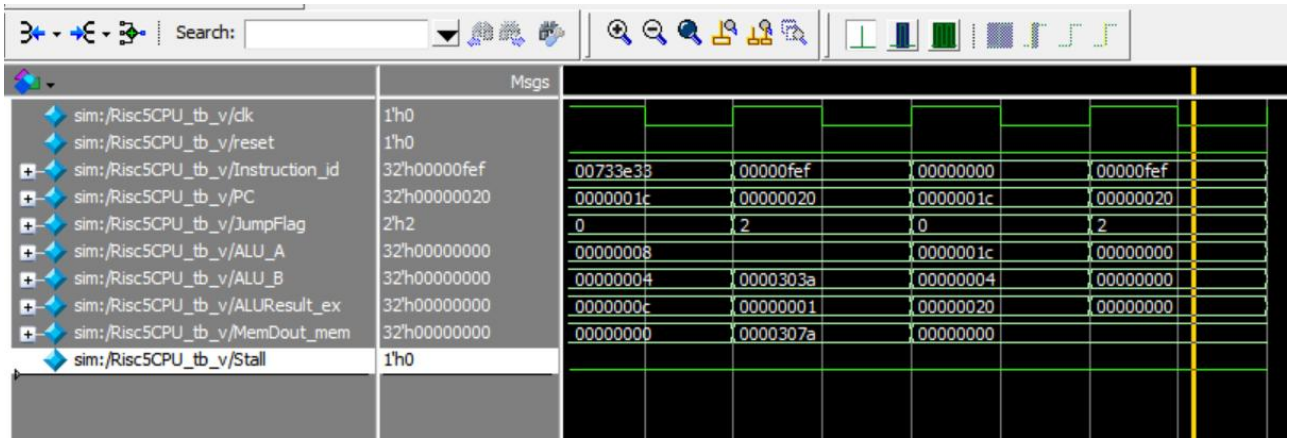

| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 20 | 0000fef | 2 | 0 | 8 | 303a | 1 | 307a |

⑰Nop：




| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 1c | 0 | 0 | 0 | 1c | 4 | 20 | - |

⑱Jar x31 done：




| PC | ID | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout |
|---|---|---|---|---|---|---|---|
| 20 | 0000fef | 2 | 0 | - | - | - | - |

表 30.11 测试程序的运行结果

| reset | clk | PC | Instruction (ID) | JumpFlag | Stall | ALU_A | ALU_B | ALUResult | MemDout (MEM) |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | - |
| | 2 | 4 | 00003f37 (lui) | 0 | 0 | - | - | - | - |
| | 3 | 8 | 02000fe7 (jalr) | 2 | 0 | - | 3000 | 3000 | - |
| | 4 | 20 | 0 | 0 | 0 | 4 | 4 | 8 | - |
| | 5 | 24 | 00001c63 (bne) | 0 | 0 | -- | - | - | - |
| | 6 | 28 | 042f0293 (addi) | 0 | 0 | - | - | - | - |
| | 7 | 2c | 01f00333 (add) | 0 | 0 | 3000 | 42 | 3042 | - |
| | 8 | 30 | 406283b3 (sub) | 0 | 0 | 0 | 8 | 8 | - |
| | 9 | 34 | 0053ee33 (or) | 0 | 0 | 3042 | 8 | 303a | - |
| 0 | 10 | 38 | fc000ae3 (beq) | 1 | 0 | 303a | 3042 | 307a | - |
| | 11 | 8 | 0 | 0 | 0 | - | - | - | - |
| | 12 | c | 01c02623 (sw) | 0 | 0 | - | - | - | - |
| | 13 | 10 | 00432e83 (lw) | 0 | 0 | 0 | c | c | - |
| | 14 | 14 | 002e9293 (sll) | 0 | 1 | 8 | 4 | c | - |
| | 15 | | | 0 | 0 | - | - | - | 307a |
| | 16 | 18 | 00432e03 (lw) | 0 | 0 | 307a | 2 | c1e8 | - |
| | 17 | 1c | 00733e33 (sltu) | 0 | 0 | 8 | 4 | c | - |
| | 18 | 20 | 0000fef (jal) | 2 | 0 | 8 | 303a | 1 | 307a |
| | 19 | 1c | 0 | 0 | 0 | 1c | 4 | 20 | - |
| | 20 | 20 | 0000fef (jal) | 2 | 0 | - | - | - | - |
| | 21 | 1c | 0 | 0 | 0 | 1c | 4 | 20 | - |

信息与电子工程学院

（3）结果解释：

①指令流执行轨迹：

1）启动与初次跳转：程序从 0x0 开始，执行 lui。在 clk=3 时执行 jalr（PC=8），随后跳转至 later 标签所在的地址 0x24

2）分支判断：在 clk=5 处遇到 bne X0, X0, end。由于 X0 = X0，条件不成立，程序继续顺序执行 addi、add、sub 和 or

3）循环回溯：在 clk=10 处执行 beq X0, X0, earlier。由于条件成立，JumpFlag 置为 1，PC 跳回至 earlier 标签所在的地址 0xc

4）最终阶段：程序执行 sw、lw、slli 等指令，最后进入 done: jal X1, done 的死循环中（PC 在 0x20 和 0x1c 之间切换）

②控制冒险：

1）跳转延迟槽与冲刷：观察 clk=4、clk=11 和 clk=19，可以发现 Instruction (ID) 列出现了“0”或无效指令。这是因为当 jalr、beq 或 jal 确定发生跳转时，流水线已经预取了错误路径上的下一条指令。处理器通过将 JumpFlag 置位并清空译码阶段的指令

2）跳转目标计算：在 clk=10 的 beq 指令，ALUResult 计算出跳转目标地址，迫使下一周期的 PC 改变

③数据冒险：

1）Load-Use 冲突与 Stall：观察 clk=14，Stall 信号置为 1· 前一条指令是 lw X29, 4(X6)（clk=13），紧接着的指令是 slli X5, X29, 2。而 slli 需要使用 X29 的值，但 lw 指令的数据直到访存阶段（MEM）结束才能写回。仅靠 Forwarding 无法消除此类冲突，因此插入了一个周期的 Stall，使 slli 在 clk=15 才正式进入执行阶段

2）Forwarding 的应用： 在 clk=8 和 clk=9 处，sub 指令计算出的 303a 立即被作为 or 指令的输入 ALU_A。这说明处理器实现了 EX-to-EX Forwarding，即不需要写回寄存器堆，直接将上一个周期的 ALU 结果传递给当前周期

④模块功能验证：

1）ALU 模块：clk=7 时，add 操作将 3000 (X30) 与 42 (立即数经过处理) 相加，得到 3042，符合预期

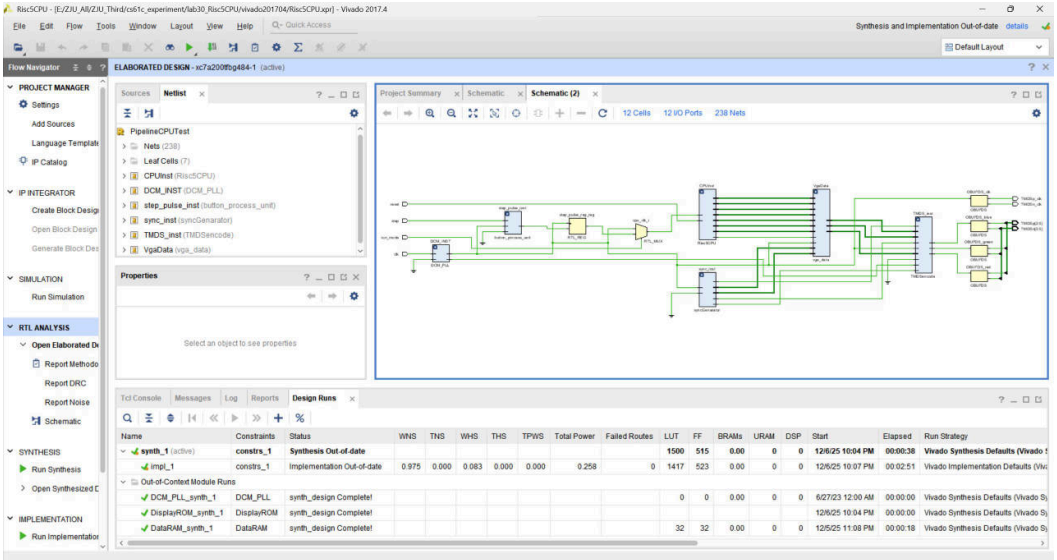
2）DataRAM 模块：访存操作，clk=15 时，MemDout 输出了 307a，对应 lw 指令从内存取出的数据，随后该数据在下一周期参与了 slli 的运算

3）Pipeline 模块：此表需要按指令阅读，译码出的 PC 是处于 IF 阶段，Decode 处于 ID 阶段，ALU_A，ALU_B，ALU_Result 处于 EX 阶段，Mem 处于 MEM 阶段，因此需要阶梯去看，才是具体到一个指令中，每一阶段运行的结果和返回的值

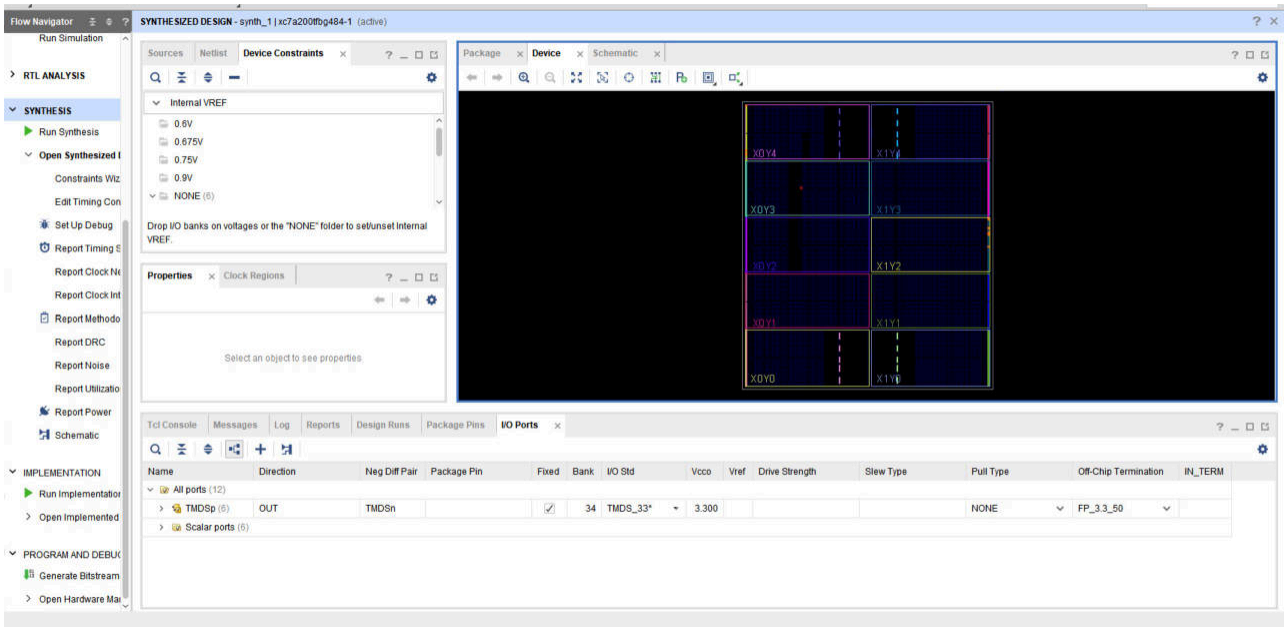
# 四、Vivado 工程实现和下载

## 1．vivado 工程文件建立

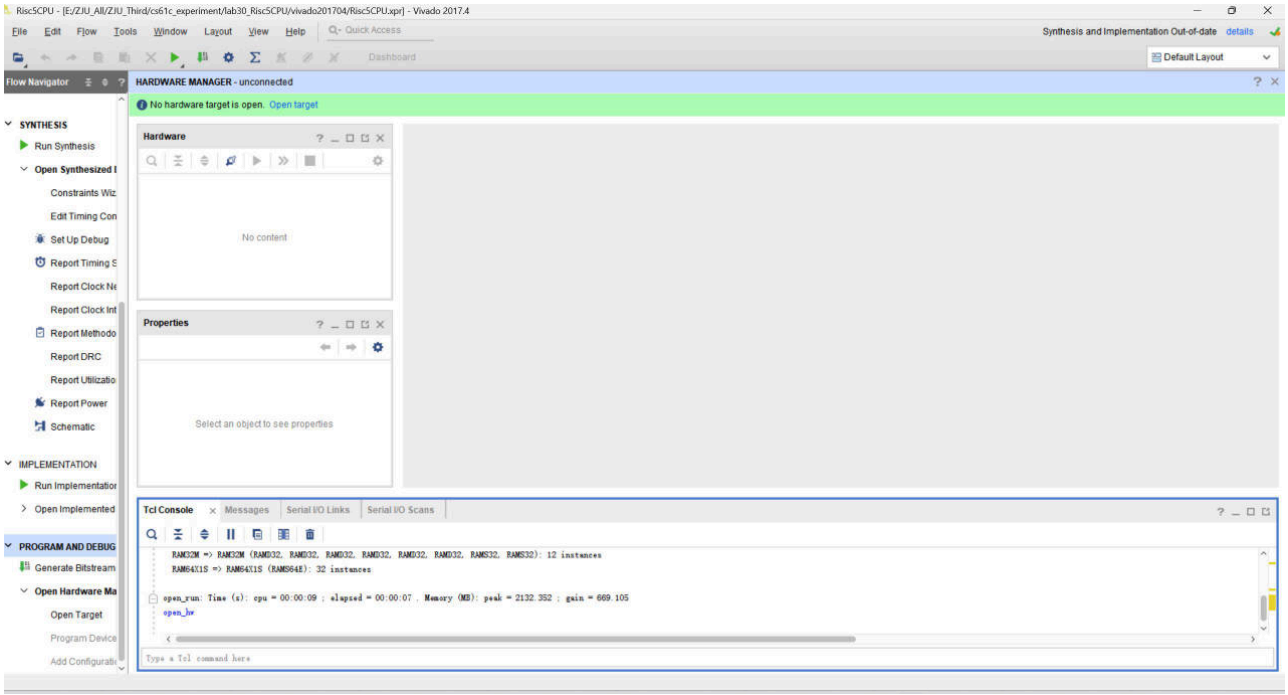
（1）RTL 综合



（2）实现




（3）下载并连接开发板




## 2．开发板测试：

（1）连接带有 HDMI 接口的显示器，进行测试。首先将 SW0 置于低电平，使 RISC-V CPU 工作在“单步”运行模式。复位后，每按一下上边按键，RISC-V CPU 运行一步，记录下显示器上的结果

（2）HDMI 接口显示器 VGA 显示格式

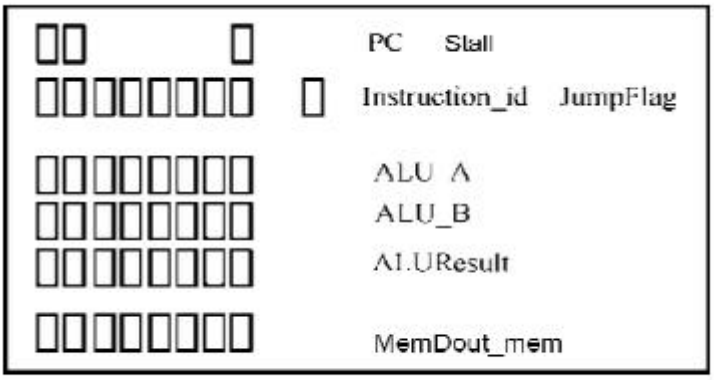



图 30.11　VGA 显示格式



# 五、拓展功能

## （一）功能 1：要求设计的微处理器还能运行 lb、lh、ld、lbu、lhu、lwu、sb、sh 或 sd 等字节、半字 和双字数据传送指令；

### 1．原理及 Verilog 代码

（1）ID 修改：LW、SW 类指令，lb，lh，ld，lbu，lhu，lwu，sb，sh ，sd 等命令主要是通过 funct3 来区分的，因此在 ID 阶段，需要将这些 funct3 从 instruction 中翻译出来，然后传给后面的 MEM 阶段

```
84  //*****************************************************************************
85      wire [6:0]      op;
86      wire            funct6_7;
87      wire [2:0]      funct3;
88      assign op           = Instruction[6:0];
89      assign funct6_7     = Instruction[30];
90      assign funct3       = Instruction[14:12];
91      assign Load_funct3  = funct3;
92      assign Store_funct3 = funct3;
93
```

（2）MEM 修改：原来的五级流水线 CPU 的 MEM 主要是通过 DataRAM 的 IP 内核实现，现在需要拓展 MEM 功能，选择 byte 或者 halfword 的写入或读出即可：

```
218     //MEM
219     reg [31:0] d_reg;
220     //根据Store_funct3_mem选择写入数据
221     always @(*) begin
222       case (Store_funct3_mem)
223           3'b000: d_reg = {24'b0, MemWriteData_mem[7:0]};//3'b000; sb
224           3'b001: d_reg = {16'b0, MemWriteData_mem[15:0]};//3'b001;sh
225           3'b010: d_reg = MemWriteData_mem;//3'b010; sw
226           default: d_reg = MemWriteData_mem;//默认为sw
227       endcase
228     end
229
230     wire [31:0] MemDout_mem_wire;//暂存lw取出的数据，后续根据指令改变格式
231     DataRAM DataRAM_inst(
232         .a(ALUResult_mem[7:2]),
233         .d(d_reg),
234         .clk(clk),
235         .we(MemWrite_mem),
236         .spo(MemDout_mem_wire)
237     );
238
239     reg [31:0] MemDout_mem_reg;
240     always @(*) begin
241       case (Load_funct3_mem)
242           3'b000: MemDout_mem_reg = {{24{MemDout_mem_wire[7]}}, MemDout_mem_wire[7:0]};//3'b000; lb
243           3'b001: MemDout_mem_reg = {{16{MemDout_mem_wire[15]}}, MemDout_mem_wire[15:0]};//3'b001; lh
244           3'b010: MemDout_mem_reg = MemDout_mem_wire;//3'b010; lw
245           3'b100: MemDout_mem_reg = {24'b0, MemDout_mem_wire[7:0]};//3'b100; lbu
246           3'b101: MemDout_mem_reg = {16'b0, MemDout_mem_wire[15:0]};//3'b101;lhu
247           default: MemDout_mem_reg = MemDout_mem_wire; //默认为lw
248       endcase
249     end
250
251     assign MemDout_mem = MemDout_mem_reg;
252     //MEM/WB
```

（3）InstructionROM 修改：为了验证功能，修改 Instruction

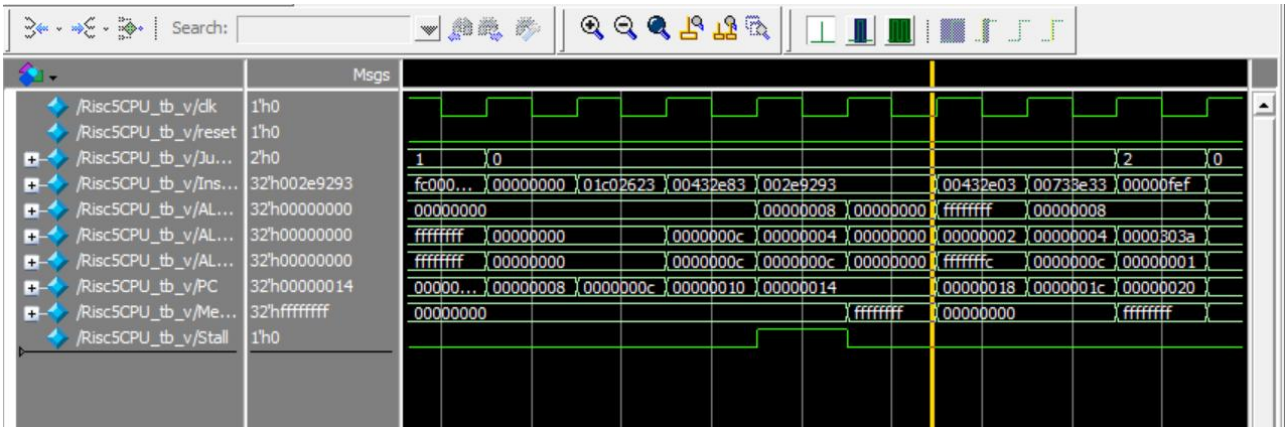
```
module InstructionROM(addr,dout);
    input [5 : 0] addr;
    output [31 : 0] dout;
    reg [31 : 0] dout;
    always @(*)
        case (addr)
            6'd0:   dout=32'h0000_3f37 ;//           lui X30,0x3000
            6'd1:   dout=32'h0200_0fE7 ;//           jalr X31 later(X0)
            6'd2:   dout=32'h01c0_2623 ;// earlier: sw  X28, 0C(X0)
            //6'd2:   dout=32'h01c0_0623 ;// earlier: sb  X28, 0C(X0)
            //6'd2:   dout=32'h01c0_1623 ;// earlier: sh  X28, 0C(X0)

            //6'd3:   dout=32'h0043_2e83 ;//           lw  X29, 4(X6)
            //6'd3:   dout=32'h0043_1e83 ;//           lh  X29, 4(X6)
            // 6'd3:   dout=32'h0043_0e83 ;//           lb  X29, 4(X6)
            //6'd3:   dout=32'h0043_4e83 ;//           lbu  X29, 4(X6)
            6'd3:   dout=32'h0043_5e83 ;//           lhu  X29, 4(X6)
            6'd4:   dout=32'h002e_9293 ;//           sll  X5, X29, 2
            6'd5:   dout=32'h0043_2e03 ;//           lw   X28, 4(X6)

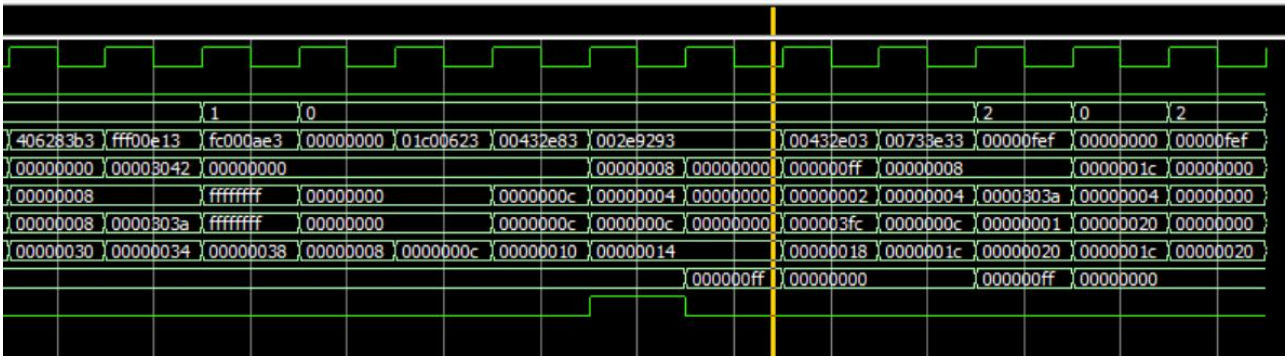
            6'd6:   dout=32'h0073_3e33 ;//           sltu X28,X6,X7
            6'd7:   dout=32'h0000_0fef ;//  done:    jal X31,done
            6'd8:   dout=32'h0000_1c63 ;//  later:   bne X0, X0, end          // 分支条件不成立
            6'd9:   dout=32'h042f_0293;//            addi X5, X30, 42
            6'd10:  dout=32'h01f0_0333 ;//           add  X6, X0, X31
            6'd11:  dout=32'h4062_83b3 ;//           sub X7, X5, X6
            6'd12:  dout=32'hFFF0_0E13 ; // addi x28, x0, -1  (x28 = 0xFFFFFFFF)
            //6'd12:  dout=32'h0800_0E13 ; // x28=1000_0000
            6'd13:  dout=32'hfc00_0ae3 ;//           beq X0, X0, earlier
            6'd14: dout=32'h00000000 ;// end:   nop
            default:dout=32'h00000000 ;//nop
        endcase
endmodule
```

## 2．仿真及结果分析：

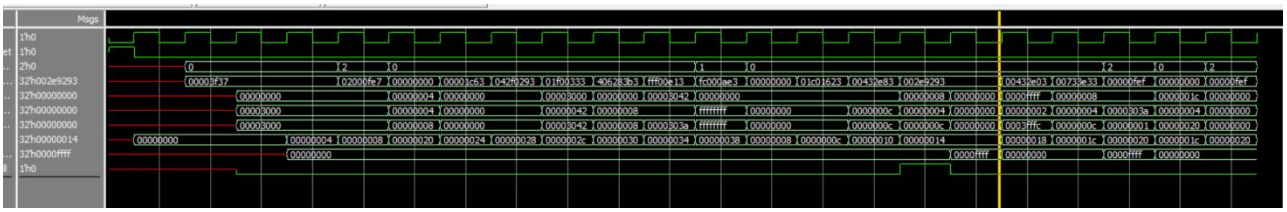
（1）使用 sw 和 lw 指令的原始结果，为了区分不同的指令，修改 x28 为-1，从结果可以看出 MemDout 的结果是 ffff_ffff，符合结果



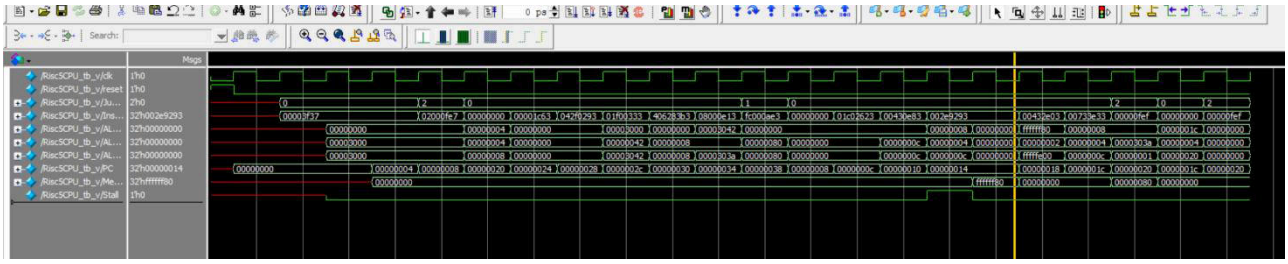

（2）使用 sb 和 lw 指令：由于 x28=-1，store byte 只存储了低 8bits，所以结果应该是 0000_00ff，从结果可以看出 MemDout 的结果是 0000_00ff，符合结果



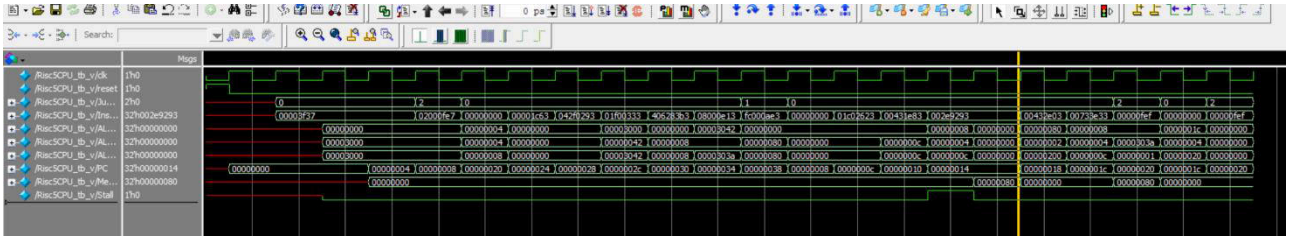
（3）使用 sh 和 lw 指令:由于 x28=-1，store half word 只存储了低 16bits，所以结果应该是 0000_ffff,从结果可以看出 MemDout 的结果是 0000_ffff，符合结果



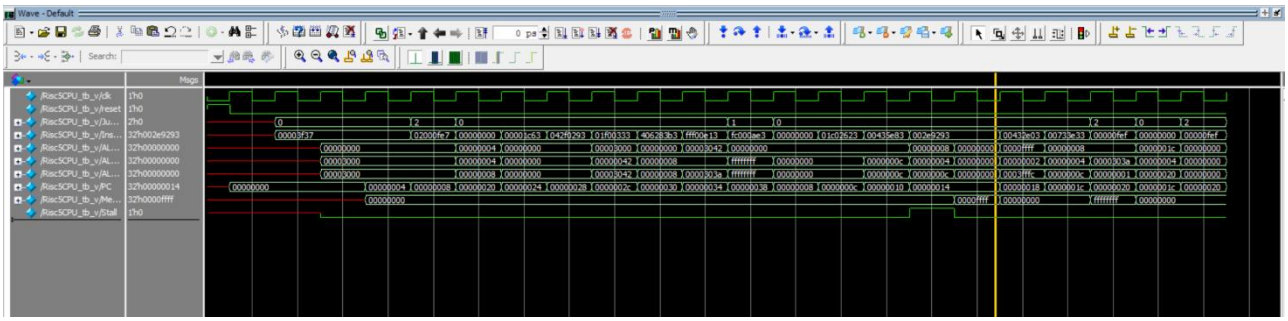
（4）使用 sw 和 lb：为了区分不同的指令，修改 x28 为 128 即 0000_0000_0000_0000_0000_0000_1000_0000，lb 只读取了低 8bits，用 1 补齐，所以结果应该是 ffff_ff80：



（5）使用 sw 和 lbu:修改 x28 为 128 即 0000_0000_0000_0000_0000_0000_1000_0000，lbu 只读取了低 8bits，用 0 补齐，所以结果应该是 0000_0080：



（6）使用 sw 和 lhu：修改 x28 为-1 即 ffff_ffff，lhu 只读取了低 16bits，用 0 补齐，所以结果应该是 0000_ffff：



## （二）功能 2：CPU 增加异常（exception）、自陷（trap）、中断（interrupt）等处理方案

### 1. 原理及 Verilog 代码

（1）新增 CSR 异常处理模块：

```
module CSR(clk,reset,PC,interrupt,illegal_inst,is_mret,mepc,trap,trap_addr);
    input clk, reset;
    input [31:0] PC;       // 当前指令的PC
    input interrupt;       // 外部中断请求（如按键）
    input illegal_inst;    // 非法指令信号
    input is_mret;         // mret指令信号

    output reg [31:0] mepc;     // 输出 mepc 的值用于返回
    output trap;                // 发生中断/异常
    output [31:0] trap_addr;    // 异常处理程序的入口地址


    reg [31:0] mcause; // 记录原因 (1:中断, 0:非法指令)

    // 固定的异常入口地址
    localparam TRAP_VECTOR_ADDR = 32'h0000_0004;

    // trap!
    assign trap = illegal_inst | interrupt;
    assign trap_addr = TRAP_VECTOR_ADDR;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            mepc <= 0;
            mcause <= 0;
        end
        else begin
            // 优先级：复位 > 外部中断 > 内部异常
            if (interrupt) begin
                mepc <= PC;         // 保存断点
                mcause <= 32'd1;            // 标记为中断
            end
            else if (illegal_inst) begin
                mepc <= PC;
                mcause <= 32'd0;            // 标记为非法指令
            end
        end
    end
endmodule
```

（2）修改 ID 阶段，增加识别是否是未知指令以及是否是 mret 指令

```
35
36      output is_mret;        // 新增端口
37      output illegal_inst;   // 新增端口
38
39      //识别 mret 指令
40      assign is_mret = (Instruction_id == 32'h30200073);
41      // 判断非法指令
42      reg illegal_inst_reg;
43      always @(*) begin
44          case(Instruction_id[6:0])
45              7'h33: illegal_inst_reg = 0; // R-type
46              7'h13: illegal_inst_reg = 0; // I-type
47              7'h03: illegal_inst_reg = 0; //lw
48              7'h23: illegal_inst_reg = 0; //sw
49              7'h37: illegal_inst_reg = 0; //lui
50              7'h17: illegal_inst_reg = 0; //auipc
51              7'h6F: illegal_inst_reg = 0; //jal
52              7'h67: illegal_inst_reg = 0; //JALR
53              7'h63: illegal_inst_reg = 0; //SB_type
54              7'h73: illegal_inst_reg = 0; // SYSTEM (比如 mret)
55              7'h00: illegal_inst_reg = 0; // 跳转等待nop
56              default: illegal_inst_reg = 1; // 没见过的指令就是非法指令
57          endcase
58      end
59      assign illegal_inst = illegal_inst_reg;
60
```

（3）修改 IF 阶段，增加 PC 修改逻辑，如果 trap 则跳到异常处理指令，如果读到了 mret 则跳回断点位置

```
24      //指令指针寄存
25      wire PCsource;
26      assign PCsource=Jump || Branch;
27      assign IF_flush=PCsource || trap || is_mret;
28      wire [31:0] NextPC_if;
29      assign NextPC_if =
30            (trap == 1) ? trap_addr :       // 1. 发生异常，跳到入口 (0x0004)
31            (is_mret == 1) ? mepc :         // 2. 执行 mret，跳回断点
32            (PCsource) ? JumpAddr :    // 3. 原有的分支/跳转
33            (PC + 4);                  // 4. 顺序执行
34
```

（4）顶层文件 CPU 调用 CSR 模块，修改端口

```
19      //CSR阶段
20      wire interrupt = 0;
21      wire illegal_inst_id;
22      wire is_mret_id;
23      wire trap;
24      wire [31:0] trap_addr;
25      wire [31:0] mepc;
26      wire [31:0] PC_id;
27
28      CSR CSR_inst(
29          .clk(clk),
30          .reset(reset),
31          .PC(PC_id),
32          .interrupt(interrupt),
33          .illegal_inst(illegal_inst_id),
34          .is_mret(is_mret_id),
35
36          .mepc(mepc),
37          .trap(trap),
38          .trap_addr(trap_addr)
39      );
```

（5）instructionROM 修改，来检验功能是否正确实现

```
module InstructionROM(addr,dout);
    input [5 : 0] addr;
    output [31 : 0] dout;
    reg [31 : 0] dout;
    always @(*)
        case (addr)
            // 0x00: 复位后执行的第一条指令。跳过异常处理程序，直接跳到 0x10 (Main)
            // jal x0, 16 (offset=0x10) -> 机器码 0x0100006f
            6'd0:   dout=32'h0100_006f ;

            // 异常处理程序 (Trap Handler)
            // 硬件固定入口地址 0x04 (CSR.v 中定义的 TRAP_VECTOR_ADDR)
            // addi x31, x0, 1
            6'd1:   dout=32'h0010_0f93 ;

            // 0x08: 从异常返回。恢复 PC = mepc
            // mret (System指令)
            6'd2:   dout=32'h3020_0073 ;

            // 0x0C: 填充空指令
            6'd3:   dout=32'h0000_0000 ; // nop

            // 主程序 (Main)
            // 0x10: 正常初始化一些数据
            // addi x1, x0, 5
            6'd4:   dout=32'h0050_0093 ;

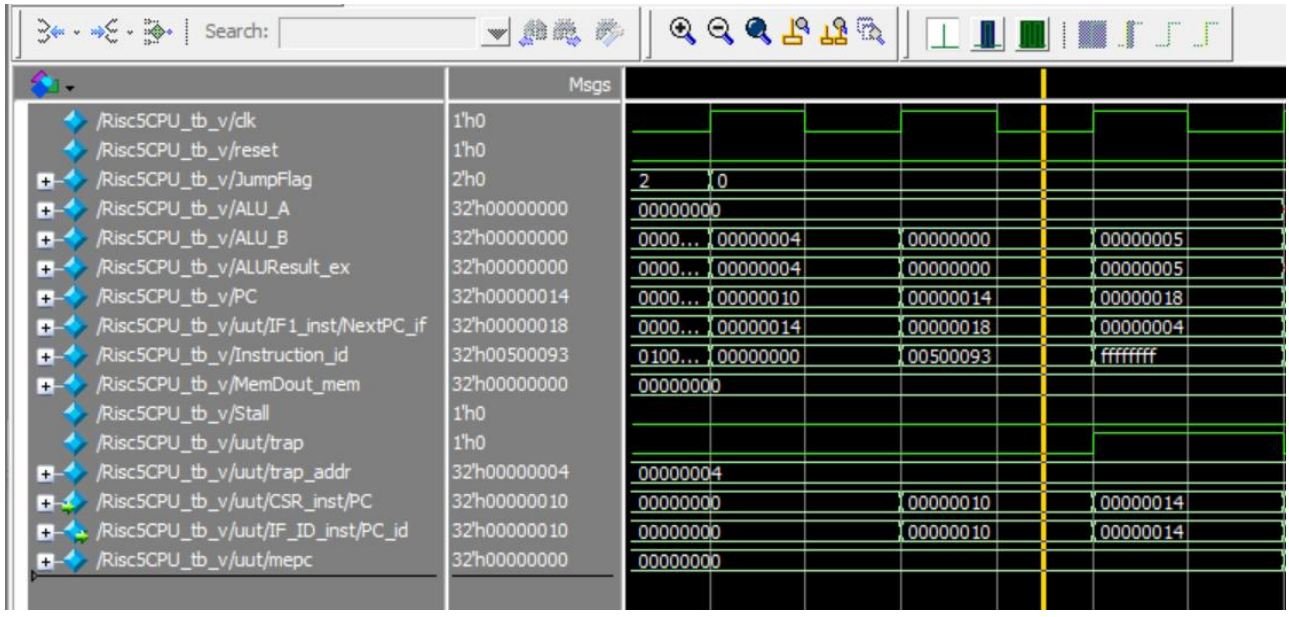
            // 0x14:【测试核心】
            // 0xFFFFFFFF 不是 RISC-V 的合法指令，Opcode 全是 1
            // 预期：CPU 应该立即把 PC 变成 0x04，并把 0x14 存入 mepc
            6'd5:   dout=32'hFFFF_FFFF ;

            // 0x18: 正常情况下这条指令会被跳过
            6'd6:   dout=32'h0020_0113 ; // addi x2, x0, 2

            // 结束
            default: dout=32'h0000_0000 ; // nop
        endcase
endmodule
```

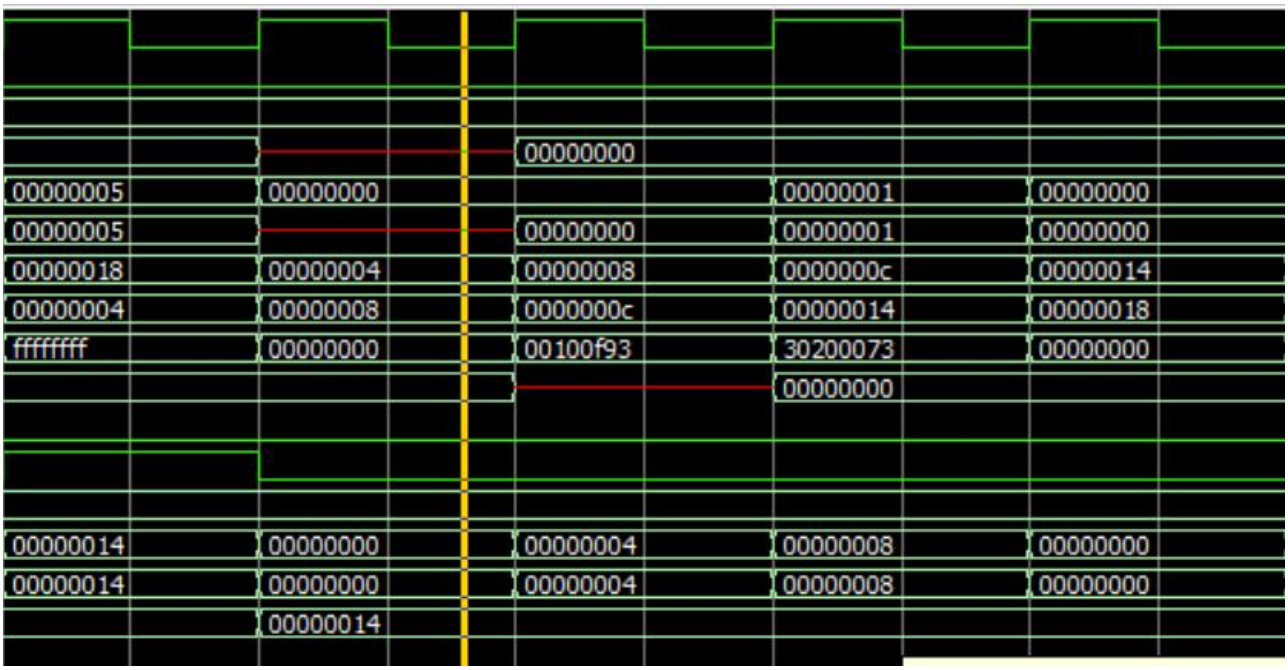
## 2．原波形仿真结果分析

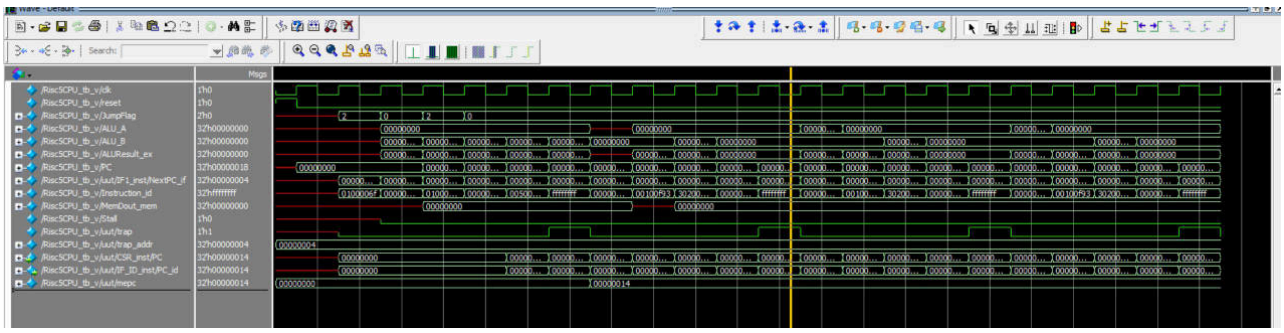
（1）执行第一条指令后跳转到 0x10，即将执行非法指令




（2）执行非法指令后跳转到 0x04 即提前设定的异常处理程序 6'd1: dout=32'h0010_0f93 ;然后下一步执行 mret，6'd2: dout=32'h3020_0073；异常发生时，mepc

指向的是导致异常的那条指令。如果软件处理程序（Trap Handler）没有修改 mepc 的值，那么 mret 就会返回去再次执行那条错误的指令，从而导致死循环。因此基本符合功能要求



（3）总体思路

①核心增设模块：CSR 寄存器堆 (Control and Status Registers)RISC-V 的异常处理依赖于一组专用的控制状态寄存器。需要在 ID 级（译码级） 或 WB 级（回写级）旁路增设一个 CSR 模块。

1）mtvec (Machine Trap-Vector Base-Address): 存放异常处理程序的入口地址。当异常发生时，PC 将跳转到这里。

2）mepc (Machine Exception PC): 存放遇到异常的那条指令的地址（或下一条），用于异常处理结束后 mret 返回。

3）mcause (Machine Cause): 记录导致异常或中断的原因（例如：非法指令是 2，断点是 3，外部中断是 11 等）。mstatus (Machine Status): 核心是 MIE 位（全局中断使能），用于控制

中断的开启与关闭。

4）mie / mip: 分别用于控制和显示具体的中断类型（如定时器中断、外部中断等）。

②异常与中断的检测逻辑 (Detection Logic)异常可能发生在流水线的不同阶段，中断则是异步发生的。需要收集各阶段的异常信号。

1）ID 级检测：非法指令异常 (Illegal Instruction): 译码器无法识别 opcode。系统调用 (ECALL): 用户程序主动请求自陷

2）EX/MEM 级检测：地址不对齐 (Misaligned Access): 在 load/store 或跳转时，目标地址不是 4 字节（或 2 字节）对齐。存储器访问错误：访问了非法内存区域（可选）

3）外部中断信号：来自开发板按键或定时器的信号，通常在 WB 级 退休前或 IF 级 取指前进行采样判断

③数据通路与控制逻辑的修改

1）PC 更新逻辑 (IF Stage)：目前的 PC 更新逻辑主要由 PCSource 控制顺序执行或跳转。修改：需要增加一个多路选择器输入。逻辑：当检测到 Trap 信号有效时，PC 的下一值强制置为 mtvec 寄存器的值。

2）流水线冲刷 (Pipeline Flush)：当异常或中断发生时，流水线中已经取出的后续指令必须被作废，否则会执行错误的操作。修改：利用现有的 IF_flush 信号 和 Stall 机制。逻辑：一旦触发 Trap：产生全局 Flush 信号，清空 IF/ID、ID/EX、EX/MEM 寄存器（将其变为 NOP）。确保不会有副作用（如内存写入 MemWrite 或寄存器写入 RegWrite）发生。

3） CSR 指令支持：需要增加对 CSR 读写指令的支持（如 csrrw, csrrs）以及特权指令 mret。mret (Machine Return): 是一种特殊的跳转指令。执行动作：PC <= mepc，同时恢复 mstatus 中的中断使能位。

## 六、实验过程中遇到的问题和解决方案

### 1．原 CPU 代码编写时：

（1）第一次根据讲义编写完后，遇到的第一个问题就是，几乎全部是红线！即几乎全部是不定态，只有第一个 PC 为 0，此时我发现最重要的问题就是，明明 IF 模块是正确的，为什么一条指令都没有取到呢，这就导致指令一个都没有往后传递，此时我发现 IFwrite 是不定态，这就导致了 IF/ID 的使能端 EN 无法使得 IF/ID 工作，我想起了当时数电实验也碰到过这个问题，应该是因为 modelsim 本身的版本问题，一开始我写的是 if (EN )则将指令传

递，但是一开始 IFwrite 确实是没有值传来的，所以是不定态，这个版本的 modelsim 这么写无法处理不定态！所以我改写为了 if (EN !== 1'b0)，只要不是 0，即 EN 是 1 或者不定态，都执行后面的指令，这样后面的很多线都不再是红线了！

（2）在 debug 过程中，由于流水线寄存器有很多的端口，尤其是 ID/EX 寄存器，我有一个写错名称了，即后缀 id 和 ex 写反了，导致结果一开始是错误的，所以在写端口时一定要仔细检查！

（3）由于我“擅自”编写了 mux3，而没有选择 ifelse 作为数据选择器，mux3 的 addr 我一开始只写了一位，因为我直接复制了 mux2 的端口，多写了 in2，所以导致后面 ALU 的结果不是很正确，没有选择到第三个端口，后续我改写了 Addr 应该得有两位才能选择三个数据的来源

（4）Debug 过程中，即和正确仿真结果对照时，我一开始很有疑问，几乎有点看不懂这个表的结果为什么是这样的，后来我理清了，PC 对应的是 IF 阶段，instruction 对应的是 ID 的 Decode 阶段，ALUA，ALUB，ALUResult 的是 EX 阶段的，MemDout 的是 MEM 阶段，所以要以**流水线**的思路去看，PC 对应的该指令的 instruction 应该看下一行，而在执行的运算阶段，应该看再下一行才是这条指令的运行结果

（5）Modelsim 仿真阶段，一开始有点忘记了 IP 内核是怎么导入仿真的，后续查看了原来 IP 内核的实验，是要先导入 IP 的 sim_netlist，然后 add project 将 tb 文件导入仿真即可

2．编写拓展功能：

（1）写拓展功能时，我通过查询资料，确认添加 sb、lb 等功能还是比较好拓展的，只需要多引入一个判断信号即可，但是我在编写拓展功能时遇到的最大的问题就是，instructionROM 的设计，这个让我非常苦恼，因此最后我选择了，在已经提供的 instructionROM 上进行修改

（2）第二个功能的设计也非常非常让我困惑，这个 instructionROM 不能直接套用，需要自己去写，在这个功能编写时，我遇到最大的问题就是，时序逻辑的问题，如 CSR 如果其中部分变量采取了 always 语句去赋值，会导致时序混乱，得到了错误的值

# 七、思考题

1．如下面两条指令，条件分支指令试图读取上一条指令的目标寄存器，插入气泡或数据转发都无法解决流水线冲突问题。为什么在大多 CPU 架构中，都不去解决这一问题?这一问题应在什么层面中解决?

```
lw   X28, 04(X6)
beq  X28,x29,Loop
```

（1）根本原因在于数据准备好与数据需要被使用的时间点不匹配：

①LW 指令：lw 指令需要从数据存储器（Data RAM）中读取数据。这个操作发生在流水线的 MEM 阶段。意味着直到 MEM 阶段结束，寄存器 X28 的新值才真正进入内部。

②BEQ：为了减少分支预测失败带来的开销，本实验设计的 CPU 将分支比较逻辑提前到了 ID（译码）阶段 。这意味着 beq 指令在 ID 阶段就需要读取 X28 的值进行比较。

③当 beq 指令处于 ID 阶段（需要数据）时，上一条 lw 指令仅仅处于 EX 阶段（正在计算地址），数据还在内存里，根本没有被读出来。即使有（Forwarding）也无法解决：数据转发通常是将 ALU 的计算结果（在 EX 阶段产生）直接传给下一条指令。但 lw 的数据不是算出来的，而是读出来的，必须要等 MEM 阶段完成。

（2）虽然硬件可以通过暂停流水线（Stall/插入气泡）来解决这个问题，但是代价过高：

①性能损失：为了让 beq 在 ID 阶段拿到 lw 在 MEM 阶段取回的数据，流水线需要暂停 2 个时钟周期；

②硬件复杂度：在 ID 阶段检测这种特定的“加载后紧跟分支”的冒险，并控制流水线回退或冻结多个周期，会显著增加冒险检测单元（Hazard Detector）的逻辑复杂度

（3）硬件保持简单，复杂的依赖关系交给软件处理

①编译器调度：编译器会检测到这种依赖关系，并尝试在 lw 和 beq 之间插入一条不相关的指令（填充延迟槽）

②插入 NOP：如果找不到有用的指令来填充，编译器会直接在 lw 和 beq 之间插入一个或多个 nop，确保 beq 执行时 lw 已经完成了访存操作

# 八、实验总结

本实验通过对 RISC-V 五级流水线处理器的仿真测试，验证了处理器在处理复杂指令流时的正确性与高效性。实验的核心意义在于深入理解了流水线架构下指令并行执行与资源冲突之间的平衡。

首先，在控制冒险方面，通过对 jal、jalr 及 beq 指令的跟踪，我观察到处理器能够识别跳转指令并及时发出 JumpFlag 信号。实验结果证明，通过“清空延迟槽指令（Flush）”的策略，处理器有效避免了由于程序流改变导致的指令误执行问题。

其次，在数据冒险处理上，实验清晰地展示了“转发（Forwarding）”与“停顿（Stall）”的协同工作：对于普通的寄存器依赖，转发机制保证了流水线无需等待写回即可继续执行；而对于 lw 指令后的 Load-Use 冲突，硬件自动插入的一个周期 Stall 信号（如 clk=14 所示）则是保证数据一致性的必要手段。

最后，通过量化分析 ALU 运算结果与内存访存数据，验证了数据通路（Datapath）设计的严谨性。本次实验不仅巩固了我对流水线五大阶段（取指、译码、执行、访存、写回）的理论认知，更让我通过波形与数据表分析，直观掌握了现代处理器主频优化与逻辑复杂性之间的权衡艺术。这为后续研究更复杂的乱序执行或多核架构打下了坚实的实践基础。